

Quality Software Design

SOFTWARE DESIGN GUIDES

AUTHOR	RANDALL MAAS
OVERVIEW	This guide describes the design of high-quality software for embedded systems. The intent is to promote well-founded, justified designs and confidence in their operation. It provides guides, checklists and templates.
BENEFITS	Improve the quality of source code: its maintainability, testability, <i>etc.</i> Prevent potential defects Smoother, shorter design / release cycles Better software products
TEMPLATES	Design documentation templates Design review checklists Software Risk Analysis Templates Bug reporting template Coding Style guides for C and C++ Bug defect type classification Code review checklists Code quality rubric

RANDALL MAAS has spent decades in Washington and Minnesota. He consults in embedded systems development, especially medical devices. Before that he did a lot of other things... like everyone else in the software industry. He is also interested in geophysical models, formal semantics, model theory and compilers.

You can contact him at randym@randym.name.

LinkedIn: <http://www.linkedin.com/pub/randall-maas/9/838/8b1>

PREFACE	1
SPECIFICATIONS.....	3
OVERVIEW OF SOFTWARE DESIGN QUALITY	5
PROCESS	9
SPECIFICATION.....	21
REQUIREMENTS CHECKLISTS	33
SOFTWARE RISK ANALYSIS	37
SOFTWARE DESIGN & DOCUMENTATION.....	43
DESIGN OVERVIEW & WRITING TIPS	45
ARCHITECTURE TEMPLATE	51
SOFTWARE ARCHITECTURE RISK ANALYSIS	59
DETAILED DESIGN	62
COMMUNICATION PROTOCOL TEMPLATE.....	80
PROGRAMMER DOCUMENTATION	88
SOFTWARE MODULE DOCUMENTATION TEMPLATE.....	92
DESIGN REVIEW CHECKLISTS	102
SOFTWARE DETAILED DESIGN RISK ANALYSIS	110
SOURCE CODE CRAFTSMANSHIP	114
OVERVIEW OF SOURCE CODE WORKMANSHIP.....	116
C/C++ CODING STYLE	118

CODE INSPECTIONS AND REVIEWS	147
CODE INSPECTION & REVIEWS CHECKLISTS	151
APPENDICES.....	161
ABBREVIATIONS, ACRONYMS, GLOSSARY	163
PRODUCT STANDARDS.....	169
FLOATING-POINT PRECISION	172
BUG REPORT TEMPLATE	173
TYPES OF DEFECTS	179
CODE-COMPLETE REQUIREMENTS REVIEW CHECKLISTS	186
CODE-COMPLETE DESIGN REVIEW CHECKLISTS	188
DESIGN REVIEW RUBRIC	198
CODE-COMPLETE CODE REVIEW CHECKLISTS.....	202
CODE REVIEW RUBRIC.....	219
ARM CORTEX-M SPECIFICS	225
HARDWARE FIRMWARE INTEGRATION TESTS	230
REFERENCES & RESOURCES	237

FIGURE 1: THE HIERARCHY OF SYSTEMS & SUBSYSTEMS	5
FIGURE 2: LEVELS OF ABSTRACTION IN DEVELOPMENT PROCESS	6
FIGURE 3: LEVELS OF ABSTRACTION IN DEVELOPMENT PROCESS	10
FIGURE 4: WHERE KEY FUNCTIONS & REQUIREMENTS ARE IDENTIFIED IN THE PROCESS	14
FIGURE 5: EXAMPLE DIAGRAM WITH CALLOUTS.....	22
FIGURE 6: EXAMPLE STATE FLOW DIAGRAM	24
FIGURE 7: TYPICAL REQUIREMENT TEXT	27
FIGURE 8: STRUCTURE OF A BROAD DESIGN WITH MODERATE FAN OUT.....	47
FIGURE 9: STRUCTURE OF A MID-SIZE DESIGN, WITH HIGH FAN OUT	47
FIGURE 10: BASIC FLOW STRUCTURE OF THE SOFTWARE	52
FIGURE 11: PROCESSOR WITH A SUPERVISOR PROCESSOR.....	53

FIGURE 12: EXAMPLE STATE FLOW DIAGRAM	54
FIGURE 13: SOFTWARE ITEMS, MAJOR FUNCTIONALITY GROUPS	55
FIGURE 14: BASIC STRUCTURE DIAGRAM OF THE SOFTWARE	63
FIGURE 15: BASIC STRATIFIED DIAGRAM OF THE SOFTWARE MODULES.....	65
FIGURE 16: BASIC SEPARATION INTO THREADS.	66
FIGURE 17: SEPARATION INTO THREADS & INTERRUPTS TO DRIVE HARDWARE	66
FIGURE 18: BASIC THREAD STRUCTURE	67
FIGURE 19: TYPICAL INSTRUMENTATION STRUCTURAL DIAGRAM	67
FIGURE 20: TYPICAL INSTRUMENTATION LOOP	68
FIGURE 21: DMA DRIVEN LINEAR INPUT AND OUTPUT	69
FIGURE 22: TYPICAL COMMUNICATION STACK	70
FIGURE 23: DMA DRIVEN COMMUNICATION	70
FIGURE 24: TYPICAL STORAGE STACK.....	71
FIGURE 25: DMA DRIVEN STORAGE	72
FIGURE 26: TYPICAL FIELD-ORIENTED CONTROL OF MOTOR SPEED	72
FIGURE 27: SEGMENTATION OF MEMORY WITH CANARIES.....	74
FIGURE 28: OVERVIEW OF BUFFERS WITH CANARIES	74
FIGURE 29: OVERVIEW OF THE STACK STRUCTURE WITH CANARIES	75
FIGURE 30: UNIT AND SUBSYSTEM TEST CONFIGURATION.....	76
FIGURE 31: WHITE BOX TEST STATION CONFIGURATION.....	77
FIGURE 32: SEQUENCE FOR READING PORTION OF THE XYZ DATA	81
FIGURE 33: THE XYZ DATA RETRIEVAL ALGORITHM	82
FIGURE 34: LOGICAL OVERVIEW OF THE COMMUNICATION STACK OVERVIEW	83
FIGURE 35: THE FORMAT OF THE COMMAND/QUERY AND RESPONSE MESSAGES.....	84
FIGURE 36: READ COMMAND SEQUENCE ON SUCCESS	86
FIGURE 37: READ COMMAND WITH ERROR RESPONSE	86
FIGURE 38: THE CONFIGURATION OF THE PRODUCTION FIRMWARE	89
FIGURE 39: HOW .H AND .C FILES RELATED TO A MODULE	91
FIGURE 40: OVERVIEW OF THE FOO MODULE.....	93
FIGURE 41: DETAILED MODULE ORGANIZATION	98
FIGURE 42: HOW .H AND .C FILES RELATED TO A MODULE	121
FIGURE 43: OVERVIEW OF BUFFERS WITH CANARIES	134
FIGURE 44: TYPICAL PROCEDURE TEMPLATE.....	135
FIGURE 45: PRIORITIZED INTERRUPTS AND EXCEPTIONS	228
TABLE 1: ISO/IEC 25010 MODEL OF SOFTWARE QUALITY.....	6
TABLE 2: MCCALL MODEL OF SOFTWARE QUALITY.....	6
TABLE 3: INPUTS FOR EACH KIND OF RISK ANALYSIS	17
TABLE 4: EXAMPLE TABLE OF DIAGRAM ELEMENTS.....	23
TABLE 5: TABLE TEMPLATE TO DESCRIBE A STATE.....	24
TABLE 6: DECOMPOSITION OF EXAMPLE REQUIREMENT	28
TABLE 7: SUMMATION OF THE TYPICAL REQUIREMENTS PATTERNS.....	29
TABLE 8: VALUE ACCURACY RISKS	38
TABLE 9: HAZARD PROBABILITY LEVELS BASED ON MIL-STD 882	38
TABLE 10: AN EXAMPLE RISK ACCEPTABILITY MATRIX DETERMINING RISK ACCEPTABILITY	38
TABLE 11: MESSAGE CAPACITY RISKS	39
TABLE 12: TIMING CAPACITY RISKS	39
TABLE 13: SOFTWARE FUNCTION RISKS.....	40
TABLE 14: SOFTWARE ROBUSTNESS RISKS	40
TABLE 15: SOFTWARE CRITICAL SECTIONS RISKS	41
TABLE 16: UNAUTHORIZED USE RISKS.....	41
TABLE 17: THE SOFTWARE DESIGN ELEMENTS	53
TABLE 18: THE EXTERNAL ELEMENTS.....	53
TABLE 19: EXAMPLE SOFTWARE ITEMS DESCRIPTION AND KEY RESPONSIBILITIES	55
TABLE 20: TIMING CAPACITY RISKS	60
TABLE 21: SOFTWARE FUNCTION RISKS.....	61
TABLE 22: THE STRUCTURAL DIAGRAM ELEMENTS	63
TABLE 23: THE EXTERNAL ELEMENTS	63

TABLE 24: SUMMARY OF THE READ DATA COMMAND	85
TABLE 25: PARAMETERS FOR READ COMMAND.....	85
TABLE 26: PARAMETERS FOR READ RESPONSE	85
TABLE 27: SUMMARY OF C MODULE PREFIXES.....	89
TABLE 28: TOP-LEVEL FOLDERS IN THE PROJECT FILE DIRECTORY	90
TABLE 29: SOURCE CODE FOLDERS IN THE PROJECT FILE DIRECTORY	90
TABLE 30: FOO STRUCTURES	95
TABLE 31: FOO _T STRUCTURE	95
TABLE 32: FOO VARIABLES	95
TABLE 33: MODULE CLASSES	96
TABLE 34: FOO CLASS STRUCTURE	96
TABLE 35: FOO METHODS	96
TABLE 36: FOO INTERFACE PROCEDURES.....	97
TABLE 37: MODULE FILES.....	99
TABLE 38: CONFIGURATION OF THE FOO MODULE	100
TABLE 39: SOFTWARE FUNCTION RISKS.....	111
TABLE 40: SUFFIXES FOR CONFIGURATION MACROS AND VARIABLES.....	121
TABLE 41: THE PREFERRED TYPES FOR QUANTITY, BY DIMENSION	131
TABLE 42: THE PREFERRED INTEGER TYPE FOR A GIVEN SIZE	131
TABLE 43: COMMON ACRONYMS AND ABBREVIATIONS.....	163
TABLE 44: GLOSSARY OF COMMON TERMS AND PHRASES.....	164
TABLE 45: SAFETY STANDARDS AND WHERE THEY ADAPT FROM	171
TABLE 46: FLOAT RANGE.....	172
TABLE 47: ACCURACY OF INTEGER VALUES REPRESENTED AS A FLOAT	172
TABLE 48: READABILITY RUBRIC	198
TABLE 49: DOCUMENTATION ORGANIZATION AND CLARITY RUBRIC	199
TABLE 50: IMPLEMENTATION RUBRIC.....	200
TABLE 51: READABILITY RUBRIC	219
TABLE 52: COMMENTS AND DOCUMENTATION RUBRIC.....	220
TABLE 53: IMPLEMENTATION RUBRIC.....	221
TABLE 54: ERROR HANDLING RUBRIC	224
TABLE 55: BEHAVIOUR RUBRIC	224

Preface

This guide aims to provide relevant tools to support creating quality software. It tries to do so in a manner that the reader may apply to their projects. Why create such a thing? As a consultant who has seen many client development organizations, I've found that few have the material that I present here. None has any guidelines on good software designs, design reviews and hazard analysis of software. Many lack coding style guide, code review guidance, and bug reporting standards. If they do have code guidelines, it is sparse and could do so much more to improve quality.

This is a guide will only cover the quality of software design and the workmanship of source code. It does not cover:

- Writing software requirements
- Testing of the software
- Debugging the software
- Project and development management
- Planning, scheduling or budgeting

1. ORGANIZATION OF THIS DOCUMENT

This guide is written in 3 parts, with the broadest up front, and the most specific or esoteric toward the rear.

- CHAPTER 1: PREFACE. This chapter describes the other chapters.

PART I: SPECIFICATIONS.

- CHAPTER 2: OVERVIEW OF SOFTWARE DESIGN QUALITY. Introduces what is meant by quality.
- CHAPTER 3: PROCESS.
- CHAPTER 4: SOFTWARE SPECIFICATIONS. Provides an outline for the structure and contents of a specification.
- CHAPTER 5: REQUIREMENTS CHECKLISTS. This chapter provides checklists for reviewing requirements.
- CHAPTER 6: SOFTWARE RISK ANALYSIS.

PART II: SOFTWARE DESIGN & DOCUMENTATION. This part provides guides for software design and its documentation

- CHAPTER 7: DESIGN OVERVIEW & WRITING TIPS.
- CHAPTER 8: GUIDELINES FOR ARCHITECTURE. Provides guidelines for high-level designs.
- CHAPTER 9: SOFTWARE ARCHITECTURE RISK ANALYSIS.
- CHAPTER 10: GUIDELINES FOR DETAILED DESIGNS (e.g. major subsystems or “stacks”).
- CHAPTER 11: PROTOCOL DOCUMENTATION TEMPLATE.

- CHAPTER 12: PROGRAMMER DOCUMENTATION. The software documentation primarily for software developers, often created with *doxygen*, *docfx*, and markdown.
- CHAPTER 13: SOFTWARE MODULE DOCUMENTATION TEMPLATE. Provides a guide for detailed design documentation of a module.
- CHAPTER 14: DESIGN REVIEWS CHECKLISTS. Provides checklists for reviewing a design.
- CHAPTER 15: SOFTWARE DETAILED DESIGN RISK ANALYSIS. Describes reviewing software for hazard analysis.

PART III: SOURCE CODE CRAFTSMANSHIP. This part provides source code workmanship guides

- CHAPTER 16: OVERVIEW OF SOURCE CODE WORKMANSHIP.
- CHAPTER 17: C/C++ CODING STYLE used for C & C++ source code.
- CHAPTER 18: CODE INSPECTION & REVIEWS. Describes code reviews.
- CHAPTER 19: CODE INSPECTION & REVIEWS CHECKLISTS. Provides checklists for reviewing source code.

APPENDICES: The appendices provide extra material

- APPENDIX A: ABBREVIATIONS, ACRONYMS, & GLOSSARY. This appendix provides a gloss of terms, abbreviations, and acronyms.
- APPENDIX B: PRODUCT STANDARDS. This appendix provides supplemental information on standards and how product standards are organized
- APPENDIX C: FLOATING POINT PRECISION. This appendix recaps the limits of floating-point precision.
- APPENDIX D: BUG REPORTING TEMPLATE. A template (and guidelines) for reporting bugs
- APPENDIX E: TYPES OF DEFECTS. This appendix provides a classification of different kinds of software defects that are typically encountered.
- APPENDIX F: CODE COMPLETE REQUIREMENTS REVIEW CHECKLISTS. This appendix reproduces checklists from *Code Complete, 2nd Ed* relevant to requirements reviews.
- APPENDIX G: CODE COMPLETE DESIGN REVIEW CHECKLISTS. This appendix reproduces checklists from *Code Complete, 2nd Ed* that are relevant to design reviews.
- APPENDIX H: DESIGN REVIEW RUBRIC. This appendix provides rubrics relevant in assessing the design and its documentation.
- APPENDIX I: CODE COMPLETE CODE REVIEW CHECKLISTS. This appendix reproduces checklists from *Code Complete, 2nd Ed* that are relevant to code reviews.
- APPENDIX J: SOFTWARE REVIEW RUBRIC. This appendix provides rubrics relevant in assessing software workmanship.
- APPENDIX K: ARM CORTEX-M SPECIFICS. Technical tips and design information too low-level for a detailed design document.
- APPENDIX L: HARDWARE-FIRMWARE INTEGRATION TESTS.

REFERENCES AND RESOURCES. This provides further reading and referenced documents.

PART I

Specifications

This first part provides guides for software design and its documentation

- OVERVIEW OF SOFTWARE DESIGN QUALITY. Introduces what is meant by quality.
- PROCESS
- SOFTWARE SPECIFICATIONS. Provides an outline for the structure and contents of a specification.
- REQUIREMENTS CHECKLISTS. This chapter provides checklists for reviewing requirements.
- SOFTWARE RISK ANALYSIS.

[This page is intentionally left blank for purposes of double-sided printing]

“The project development people seemed to be a special breed of programmers whose incomprehensibility was matched only by their desire to document in a level of detail that baffled the minds of ordinary folk.”
– NSA Cryptolog, 1979 June

CHAPTER 2

Overview of Software Design Quality

This chapter promotes good software quality:

- Software quality overview
- Where do bugs come from?
- How quality software can be achieved
- A tip on staffing

2. OVERVIEW

Software lives as part of a system within a product. Typical embedded software can be described as fit into a hierarchy of systems and subsystems:

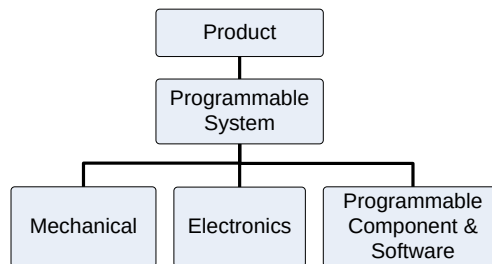


Figure 1: The hierarchy of systems & subsystems

There is the “final” *product* itself, with a portion – sometimes large, sometimes small – that is the *programmable system*. This system has mechanical and electronic subsystems, as well as the *programmable component* (usually a microcontroller) that is executing the software that will be discussing through this guidebook.

The diagram below synthesizes the levels of abstraction in the normative software development process. Guidance documents help the work to be performed be done quickly, and with appropriate craftsmanship. The tests and reviews help catch errors and improve the construction of the software.

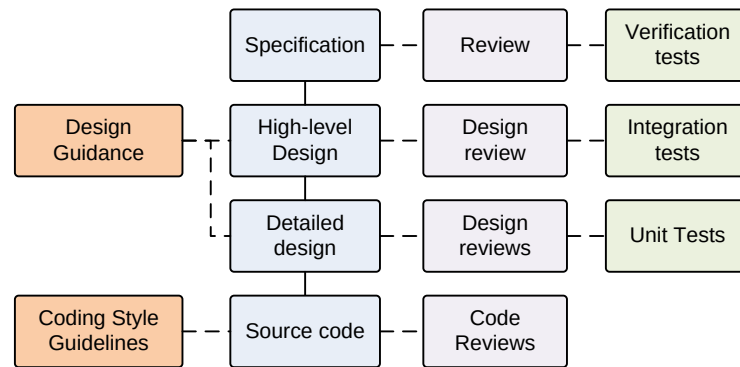


Figure 2: Levels of abstraction in development process

Review checklists & rubrics should be a dual (twin) to the coding style. Everything in one should be in the other.

3. SOFTWARE QUALITY OVERVIEW

It may be helpful to provide a brief overview of what “software quality” is. ISO/IEC 25010 model of software quality is one useful way to organize the areas of quality:

Quality factor	Quality Criteria
Functionality	Completeness, Correctness, Appropriateness
Performance & Efficiency	Time behavior, Resource utilization, Capacity
Compatibility	Interoperability
Usability	Appropriateness, Recognisability, Learnability, Operability, User error protection, Aesthetics, Accessibility
Reliability	Maturity, Availability, Fault tolerance, Recoverability
Security	Confidentiality, Integrity, Non-repudiation, Accountability, Authenticity
Maintainability	Analyzability, Modifiability, Modularity, Reusability, Testability
Portability	Adaptability, Installability, Replaceability

Table 1: ISO/IEC 25010 model of software quality

McCall’s model is another way to organize the areas of quality. It maps each top-level area of quality to a more specific quality.

Quality factor	Quality Criteria
Correctness	Traceability, Completeness, Consistency
Reliability	Consistency, Accuracy, Error tolerance
Efficiency	Execution efficiency, Storage efficiency
Integrity	Access control, Access audit
Usability	Operability, Training, Communicativeness
Maintainability	Simplicity, Conciseness, Self-descriptiveness, Modularity
Testability	Simplicity, Instrumentation, Self-descriptiveness, Modularity
Flexibility	Simplicity, Expandability, Generality, Modularity

Table 2: McCall model of software quality

<i>Portability</i>	Simplicity, Software system independence, Machine independence
<i>Reusability</i>	Simplicity, Generality, Modularity, Software system independence, Machine independence
<i>Interoperability</i>	Modularity, Communications commonality, Data commonality

These same metrics apply to the *programmable system*, and perhaps the product overall.

3.1. WHERE DO BUGS & DEFECTS COME FROM?

Where do the bugs & defects come from?

- The wrong requirements – that the product and programmable system was designed to the wrong set of rules.
- Operation action and input – inconsistent settings, out of range entries, and so forth. These indicate insufficient requirements about the constraints on the user interface.
- Poor design – a design is unsound, an algorithm has too high of computational complexity, bottlenecks & contention for resources, prioritization issues, etc.
- Edge case circumstances, such as race conditions and overloading of processing resources.
- Programmer mistakes, such as language mistakes, or incorrect use of hardware – use of disabled peripherals, bad parameters, index out of range, hardware exceptions, divide by zero, etc. These are often in the form of “exceptions” and “assert” failures.
- Hardware components may have shifted values; connections break.
- Environmental conditions – such as a component being used out of its operating range, a low battery, and so forth.

It is important to note: the software can perform with high quality, and the programmable system low quality. This can come from the wrong requirements, at any level.

3.2. HOW QUALITY SOFTWARE CAN BE ACHIEVED

Steps to quality software include recognizing that

- It is an acquired, disciplined art.
- It requires practice, diligence and assessment
- Organizations must teach how to write quality code.
- The organization must value quality software in order for the individual to value it
- The development organization has a culture of accountability and commitment
- There is encouragement for respectful, frank, rational conversations about failures
- Information, activities and agreements are explicitly communicated (rather than tacit and assumed)
- A study and critique of other projects – whether they be internal, competitors or public ones – to understand good and bad patterns.

3.3. TESTING

Testing

- Has an important role in quality
- Most often removes the “easy” and frequent bugs
- Won’t find subtle timing bugs and edge cases. It can help regression test to ensure that specific occurrences do not recur
- Doesn't improve workmanship

4. A TIP ON STAFFING

This guide generally does not address development process – plans, schedules, sequencing, staffing, and so on. However, here are some opinionated tips:

1. Assign leadership to those who care about the quality. In any organization, there is a leader somewhere who capitates the quality – even if there is one who drives a minimum quality standard. It doesn’t matter if the quality is something aesthetic (like being stylish & usable), or a process quality (like being maintainable and traceable), or other quality aspects.
2. Work with people who value the development artifacts they are creating and the processes they work in. My experience has been that people who dislike writing or reading documentation will create poor documentation and the hate shines thru.
3. Encourage *gracious professionalism*¹ where the staff is fiercely driven, seeks mutual gain, are intensely respectful and kind
4. Reduce stress. Faux urgency and cranking up the time pressure is a too common managerial technique. Meeting regular shipment schedules or quality goals is a long marathon.

In short, *care* and *drive* (or *passion*, *internal motivation*, *pride*).

5. REFERENCES AND RESOURCES

ISO/IEC FDIS 25010:2011, “*Systems and software engineering - Systems and software Quality Requirements and Evaluation (SQuaRE) - System and software quality models*” 2011

IEEE Std 730-2014, *IEEE Standard for Software Quality Assurance Processes*, 2014

IEEE Std 1061-1998, *IEEE Standard for a Software Quality Metrics Methodology*

IoT-SQH-00304, *Software Synergy Software Quality Handbook*, Renesas Electronics, Rev 3
2018 Jun

<https://www.renesas.com/us/en/img/products/synergy/software/ssp/synergy-quality-handbook.pdf>

This is an excellent outline of Renesas’s Software Quality Process.

¹ Coined by Dr Woodie Flowers, registered trademark of FIRST

CHAPTER 3

Process

This chapter describes the software development process:

- Process, specifications, and requirements
- The role of standards & certification
- System engineering
- Development plan
- Risk analysis
- Testing, Verification, Validation, and Testing

6. PROCESS

An organization always has processes – implicit or explicit – it engages to achieve goals. An explicit process spells out activities, and the artifacts to create (and more importantly information to be captured in the artifacts). These activities – creation of specifications, review, implementation, and test – are to drive clarity of thinking around the *actual* problem to solve, and in terms of *essential outcomes*. Rigorous processes attempt to assure that:

- the project will succeed, *project assurance*
- the schedule will be reasonably met,
- the cost of development is acceptable,
- the product is acceptable & performs as intended *design assurance*
- the product does not pose an unacceptable risk of harm
- the product is well made
- the product can be kept in use / operation for a period, including revising and maintaining the product.

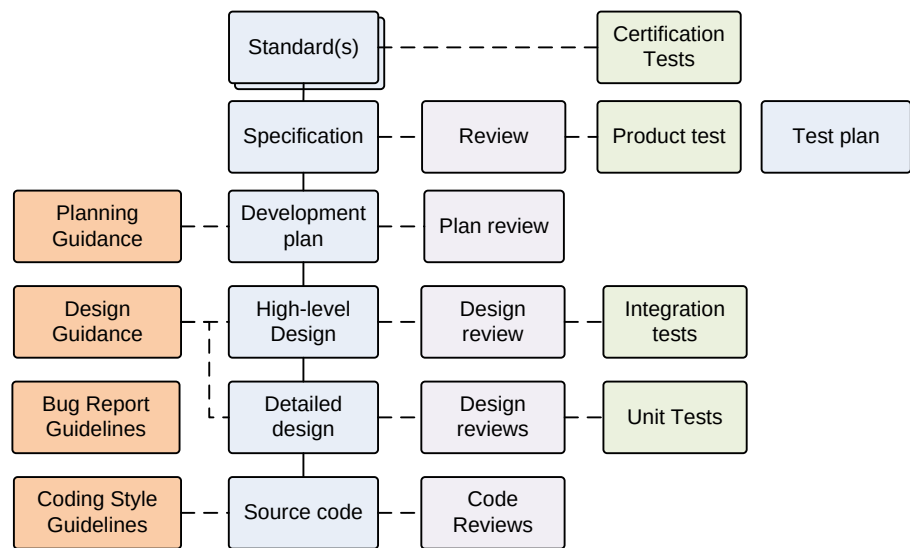


Figure 3: Levels of abstraction in development process

The motivation to use rigorous processes is to more directly check that the quality – especially the *safety related* quality – is done right. The purpose of a requirements-driven process is to promote a cultural discipline that focuses attention to build the quality products, with a particular focus on safety aspects.²

A design should be thoughtfully worked out, drilling down from the high-level specifications to the more specialized specifications, and designs.³ The documentation should be cohesive and cogent: it's to instill a clear understanding. Ideally – and depending on the rigor – each should be assessed for appropriateness, matching the products intent and requirements. Once a module's design has been approved, its source code may be created in earnest.⁴

The process should call out (and provide) workmanship guidelines, style guides, standards, and evaluation rubrics used to craft the source code; this is often done in the development plan. One goal of the guides is to provide direction to producing clear code, with a low barrier to understanding and evaluation. The following chapters provide reference guides.

The source code should be reviewed (and otherwise inspected) against those guides, and designs. The purpose of reviewing the work is to examine quality of construction – it is not an evaluation of the engineers, and it is more identifying defects.

² Although it would seem to be focused on building the product the parent organization wants, the process was only (historically) shaped to meet safety goals. There were (and are) other processes, but they did less well with safety.

³ Designing of a “lower” layer can begin (and often does) based on the anticipated top-level design, and norms for the lower layer. Its completion is dependent the top-level design being settled.

⁴ Not all reviews or designs must be complete before implementations begin, except in the most stringent of processes. Modules built in an investigatory (or as a short-term shim) fashion are useful but should be considered in an “as-is” or draft state, until they have been revised to match the design, workmanship rules, and so in.

6.1. THE DIFFERENT TYPES OF SPECIFICATION DOCUMENTS

The documents – or portions of documents – discussed here include:

A *high-level specification* is a finite set of requirements specification, e.g. system specification, customer inputs, marketing inputs, etc. *high-level specification*

A *requirements specification* that lays out what the product must do, under what conditions. Requirements are part of a chain of evidence that all the top level (user/system) & risk control requirements are fulfilled, with objective evidence. *Requirements are the means of assuring correct operation in the process of design and construction.* *requirements specification*

A *design document* explains the design of a product, with a justification how it addresses safety and other concerns. *design document*

Test specifications describe a set of tests intended to check that the product meets its requirements. The test specifications define: *test specification*

- A set of *test requirements* that define what tests a product must pass. *test requirements*
- A set of *test procedures* that carry out the test requirement and test the product *test procedure*
- A mapping of a test requirement to a set of requirements that it tests. {note: this may be covered in the trace below.}

A *test report* is a set of outcomes: *<test id, product id, result>* describing how a product performed under test. (The performance may vary with versions of the product) *test report*

A *requirements trace matrix* is used to identify requirements that are not carried thru to lower requirements specifications and designs; and (in stringent cases) identify features of the design without requirements, and requirements in lower documents that are not driven by these at a higher level. *trace matrix*

- It maps a requirement to the set of requirements that it directly descends from
- It maps a requirement to a set of requirements that directly or indirectly descends from it.

6.2. CRITICAL THINKING

Quality oriented – and especially safety oriented – processes apply analysis and reasoning to further improve the product being developed. All processes try to address what/why/where/when/how questions, by identifying where the information is or comes from:

What are we making?

1. The architecture (or high-level) specification

How do we know that we have the right (product) specification(s)?

1. Standards
2. Stakeholder reviews
3. Customer feedback (e.g. voice of customer)
4. Hazard analysis
5. Usability studies
6. Field tests

How do we know that the product meets the specification(s)?

1. Verification activities of the system and subsystem
2. Validation activities of the product

Why are we confident that product is well made and safe?

1. Reviews of specifications and design
2. Analysis of the specification for key qualities, esp. safety
3. Verification & validation, testing

How do we know if a part of a higher-level specification was missed when making a lower-level (more specific) specification?

1. Tracing
2. Validation & validation, testing

How do we know what to do?

1. Specifications
2. Development plans
3. Guidelines, e.g. coding style guides, design guides
4. Development protocols & work instructions

Why the product was designed and made this way?

1. Specifications
2. Guidelines, e.g. coding style guides, design guides
3. Design documentation
4. Design reviews

and so on

7. THE ROLE OF CERTIFYING STANDARDS

Product certification – specifically the standards being certified against – may drive software quality. Standards approach software quality as necessary to achieve product quality, especially safety and security. To simplify (and over generalize), such standards have specifications that address the following areas of software quality:

- *Risk management*, including analysis, assessment and control of the risks
- The process and artifacts, and how they will be stored and updated. These include a *software development lifecycle* (SDLC) and *quality management systems* (QMS)
- *Techniques* to be applied in the software design and implementation
- *Tests* and characterizations to be applied.

Some important examples of the safety-facing standards are:

- IEC 61508: *Functional safety of electrical/electronic/programmable electronic safety-related systems* (Part 3 deals with software and Part 7 with specific techniques)
- IEC 60730: *Automatic Electrical Controls*. (Annex H deals with software)
- ANSI/IEC 62304:2006 *Medical Device Software – Software Lifecycle Processes*

- DO-178C, *Software Considerations in Airborne Systems and Equipment Certification*
- IEEC Std 7-4.3.2 2010 *IEEE Standard Criteria for Digital Computers in Safety Systems of Nuclear Power Generating Stations*

7.1. IEC 61508 AND DERIVATIVE STANDARDS (E.G. IEC 60730)

IEC 61508's has many process facing areas, over a complete safety life cycle. It mandates IEC 61508

- A specific safety management approach, parallel to the development of primary functionality. This produces a set of *software safety requirements*.
- A specific risk management approach, including a risk assessment and analysis approach that is far more strenuous than the art in many fields. (And was when it was introduced).
- A software development lifecycle⁵, with several activities to be performed, and several artifacts to be produced.
- A mandate and guidance to apply very specific & detailed software design and implementation techniques, depending on the classification of software. Most of the techniques had been documented at least two decades prior to the first version of the version (1998-2000); all were documented at least decade prior. Most, however, were not in common use outside of niche applications.

Several IEC standards adapt IEC 61508 (a *basic safety publication*) for an industry segment, a group of products (a *group safety publication*), or specific applications (*product publications*). The IEC group safety publications may normatively reference the IEC 61508 standard (that is, mandate its use), or choose to incorporate the relevant portions into the narrower standard, with some modifications. The product publications are specific standards targeting requirements of specific categories of products or applications. These specific standards often modify the group standard, reducing the stringency in some areas. see Appendix B for more

IEC 60730-1 incorporates much of IEC 61508's software requirements (but not the risk assessment system) for home appliances. This includes the production of software safety requirements. The IEC 60730-2-xyz standards specify requirements for various types of appliances. IEC 60730 divides functionality (including software function) into three categories of safety: IEC 60730

- Class A are the functions that are not relied upon for safety see Appendix B for other classifications
- Class B are the functions that directly (or indirectly) prevent unsafe operation
- Class C are the functions that directly (or indirectly) prevent special hazards (such as explosion).

IEC 60335 follows the same pattern: 60335-1 incorporates most (but not all) of IEC 60730 software requirements. The IEC 60335-2-xyz standards specify requirements for various types of appliances. IEC 60335

This guidebook has been structured in such a manner to directly support software development under these standards. This includes not just software design & implementation, but the artifacts: requirements, design, and documentation.

⁵ Modern software development lifecycle can be found in IEEE Std 12207 (ISO/IEC 12207).

7.2. ANSI/IEC 62304

ANSI/IEC 62304 is a software development lifecycle document, and it is organized in the classic “v-model” fashion. It mandates a variety of artifacts and activities in the software development. It works intimately with a separate risk management process, and quality management system.

Like IEC 60730, it divides software into three categories of safety:

- Class A are the functions that pose no risk of injury
- Class B are the functions that pose a “non-serious” risk of injury
- Class C are the functions that could result in death or serious injury

It mandates a formal development process, including checkpoints with formal reviews and signoffs by key personnel, assuring successful completion of all criteria.

This guidebook has been structured in such a manner to directly support software development under these standards. This includes not just software design & implementation, but the artifacts: requirements, design, and documentation.

Note: ANSI/IEC 62304 is meant to work with a risk management approach, but – unlike IEC 61508 – it is expected to be provided separately. It also expects to work with a separated defined quality management system.

7.3. A SIDE NOTE ON THE ECONOMIC BENEFIT TO DEVELOPMENT

Vendors have developed support for these software functions, as these functions are employed in many product markets. Their support is in the form of certified microcontroller self test libraries, and application notes giving guidance on how to meet these standards (especially using their libraries).

This standardization also provides a means of identifying the skills and experience needed, and thus able to find expert workers.

7.4. THE SAFETY ELEMENTS

The standards atomize – with respect to behaviour and element of electronics and software –the product functions & requirements into:

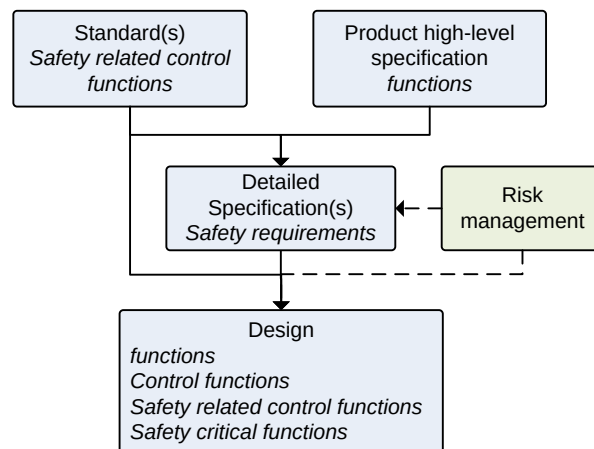


Figure 4: Where key functions & requirements are identified in the process

The high-level specification of the product defines the intended, primary *function* of the product. *function*
The function is its role or purpose, and the operations that it is intended to perform.

The standards identify *control functions*⁶ that are to be provided by the product and its design. *control function*
The standards categorize functions along three axes:

1. Whether or not it is a control function relevant to safety (rated earlier as type A, B, or C);
2. Whether or not the function is critical to the operation of a safety control function
3. Whether or not software is responsible (at least in part) for the function

This becomes:

Safety-related control functions are a type of control function that prevent unsafe conditions and/or allows the operator to use the equipment in the intended, safe manner. In IEC 60730 *Type B* control functions prevent unsafe state; *Type C* prevents special harms. The product specifications and design often expand the number of control functions and elaborate their specific operation. *safety-related control function*

The *safety critical functions* are those functions that, should they fail, present a hazardous situation. Perhaps it impairs the ability of a safety-related control function to fulfill its specification. The standards impose a variety of software functions to “self-check” that the microcontroller (or other programmable element) is functioning sufficiently to carry out the other functions. A safety-related control function is often (but may not be) a safety critical function, but not all safety critical functions are safety-related control functions. *safety-critical function*

When software is used to realize (i.e. implement) a safety-related control function, or a safety critical function, the standards impose many requirements on the design and behaviour of the software (and supporting electronics). This is a very good thing for quality, and this guidebook is intended to help address these.

The product and subsystem specifications are to provide a detailed set of *safety requirements*, which specify in detail the functional behaviour of the product, and each of those safety-related control functions and safety-critical functions. This is true for the functions implemented by software. The *software safety requirements* are to provide added requirements that address: *safety requirements*
software safety requirements

- potential faults in the software as well as the programmable element (e.g. the microcontroller) and the electronics,
- construction techniques of the software to prevent or mitigate software flaws

The motivation to use a rigorous process is to more directly check that the safety related behaviour is done right. As opposed to being satisfied by other aspects of the development and concluding that safety is acceptable.

8. SOFTWARE DEVELOPMENT PLAN

A development plan should be put into place before the software is created. The development plan typically includes:⁷

- Names
- The staffing (although this may be located in another).

⁶ Function(s) can have types (or roles) such as control, filter, protection, monitoring, test, conversion, limiting, distribution, isolation, protection, and so on.

⁷ A development plan includes much more, related project assurance, process, management, staffing, etc.

- Key processes to be followed.
- How training will be addressed. (Although I’ve seen most companies’ short shrift the training and try to skimp on this.)
- Location of artifacts and sources
- Tools and key components
- Workmanship guides and how the workmanship will be evaluated. This includes a *coding style guide*, which identifies a good, restricted subset of a programming language that is acceptable to use.
- Steps that will be done in the development process, such as reviews and risk analysis
- How changes to the software will be managed. What is the source code repository? Is commit approval required from a module owner? (e.g. the owner evaluates the appropriateness of the changes to their area of the code base.)
- How issues, bugs and so on are tracked, prioritized, and dispositioned. Example templates for bug reporting can be found Appendix D and categorization of the defect in Appendix E.

Software development plans are about being organized to succeed, and to keep succeeding for a long time. Most projects (e.g. those lasting a few months with a small number of people) do not need to spell out all the potential elements; the ones listed above are often sufficient.

9. RISK ANALYSIS

At regular steps, an analysis is performed to double check that the safety control functions, safety requirements, and design provide a acceptably safe product. The objective is “to identify and correct deficiencies and to provide information on the necessary safeguards.” UCRL-ID-122514

A *hazard analysis* is a process performed on the *product*, its specifications, functions, and design. *hazard analysis*

- It identifies a set of potential *harms* that the product (or its use) presents *harms*
- It maps a harm to *severity* or *severity class* *severity*
severity class
- It identifies a set of *hazards* or *hazard classes* that are potential sources of harm *hazards*
hazard class
- It maps a hazard or hazard class to *likelihood* or frequency that it may occur *likelihood*
- It maps the combined severity of harm and likelihood of occurrence to an *acceptability level*. This is done using an accepted rubric, most often a *risk acceptability matrix*. *risk acceptability level*
risk acceptability matrix

The acceptability level is used to prioritize changes to the specifications and design. The changes must be made until there are no unacceptable risks presented, and that the cumulative (overall) risks presented is at an acceptable level. The changes often included added functions (such as tests of the hardware or operating conditions), tighter conditions on existing requirements, added requirements, and the like.

A *risk analysis* follows the same pattern, checking that the specification, functions and design of a subsystem for the risks that the subsystem will present a hazard. A *software risk analysis* is what the software may contribute to risk or control of the product risks. *risk analysis*
software risk analysis

- Each risk analysis builds an upon earlier risk analysis
- Each type of analysis may produce a different, but related, form of output

- Each produces a summation of hazards (and risks), any identified rework, and mandates for tests for Verification & Validation activities.

9.1. INPUTS AT EACH STAGE OF SOFTWARE RISK ANALYSIS

Software is analyzed at several stages of development to assess how it will impact products risk.

The table below summarizes the inputs to each of the software risk analysis:

Requirements risk analysis	Architecture risk analysis	Detailed Design risk analysis	Source code analysis
<i>Product Preliminary Hazards list</i>	<i>Product Preliminary Hazards list</i>	<i>Product Preliminary Hazards list</i>	<i>Product Preliminary Hazards list</i>
<i>Product Risk analysis</i>	<i>Product Risk analysis</i>	<i>Product Risk analysis</i>	<i>Product Risk analysis</i>
<i>Programmable system requirements</i>	<i>Programmable system requirements</i>	<i>Programmable system requirements</i>	<i>Programmable system requirements</i>
<i>Programmable system description</i>	<i>Programmable system description</i>	<i>Programmable system description</i>	<i>Programmable system description</i>
<i>Software requirements</i>	<i>Software requirements</i>	<i>Software requirements</i>	<i>Software requirements</i>
	<i>Software requirements risk analysis</i>	<i>Software requirements risk analysis</i>	<i>Software requirements risk analysis</i>
	<i>Software architecture description</i>	<i>Software architecture description</i>	<i>Software architecture description</i>
		<i>Software architecture risk analysis</i>	<i>Software architecture risk analysis</i>
		<i>Software design description</i>	<i>Software design description</i>
			<i>Software design risk analysis</i>
			<i>Coding style guide</i>
			<i>Source code</i>

Table 3: Inputs for each kind of risk analysis

The risk analysis of the source code itself is covered by specialized code reviews.

10. TERMS RELATED TO TESTING, VERIFICATION, AND VALIDATION

A *fault* is a system or subsystem deviating from its specification, e.g. not meeting one or more of its functional requirements.

A *failure* is not providing service to the user, e.g. not meeting user requirement, often a user non-functional requirement.

*Verification*⁸ is set of activities that include:

- Testing the item against its specifications, esp. those framed as requirements.
- Inspecting and review the items standards, specifications, design, and construction

Validation includes verification of the item, *and* activities that include:

- Testing the item against the *higher-level* (such as the product's) specifications, usually the user and system requirements.

⁸ As there are many muddled definitions of verification and validation, I am using definitions that are compatible the FDA guidance, DO-178C, and DO-254

- Inspecting and reviewing the items against the *higher-level* (such as the product's) standards, specifications, design, and construction. This may include ensuring that all of the institutions process were followed.
- Testing the item against use cases
- Performing field trials, usability studies
- Evaluating customer feedback.

11. REFERENCES AND RESOURCES

DO-178C, *Software Considerations in Airborne Systems and Equipment Certification*, RTCA, Inc. 2012 Jan 5

This is a particularly stringent standard. It seeks to ensure that not only ensure that all requirements and functions (from the top on down) are carried thru and test... it also seeks proof that no element of software, function, or requirement is present unless it traces all the way back to the top.

RTCA/DO-254, *Design Assurance Guidance for Airborne Electronic Hardware*, RTCA, Inc. 2000 Apr 19

IEC 61508: *Functional safety of electrical/electronic/programmable electronic safety-related systems* 2010

Part 3 deals with software and Part 7 with specific techniques. The software and electronics techniques to check the programmable element function are well founded, providing a good explanation of their motive, approach, and many references for each.

The testing portion is a bit of a muddle.

IEC 60730: *Automatic Electrical Controls*, 2010

Annex H deals with software

UL 1998, *Standard for safety – Software in Programmable Components*

11.1. RISK MANAGEMENT

UCRL-ID-122514, Lawrence, J Dennis “*Software Safety Hazard Analysis*” Rev 2, U.S. Nuclear Regulatory Commission, 1995-October

ISO 14971:2007, *Medical devices – Application of risk management to medical devices*
EN ISO 14971:2012, *Medical devices. Application of risk management to medical devices*

This standard is for the European market; the earlier one is for the rest of the world

Speer, Jon “*The Definitive Guide to ISO 14971 Risk Management for Medical Devices*”
Greenlight Guru, October 5, 2015
<https://www.greenlight.guru/blog/iso-14971-risk-management>

A clear introduction to the concepts and steps, with some elegant diagrams.

11.2. DEVELOPMENT LIFECYCLE

ANSI/IEC 62304:2006 *Medical Device Software – Software Lifecycle Processes*

This is a well written standard on the development life cycle.

ATR-2011(8404)-11, Marvin C. Gechman, Suellen Eslinger, “*The Elements of an Effective Software Development Plan: Software Development Process Guidebook*” 2011-Nov 11, Aerospace Corporation, Prepared for: Space and Missile Systems Center, Air Force Space Command

<http://www.dtic.mil/dtic/tr/fulltext/u2/a559395.pdf>

The above guide is particularly rigorous and intended for long-lived project (e.g. two decades) with large & changing hierarchies of many people working for many different organizations (thus many organizational boundaries), across a geographic area, and wide range of organizational roles and backgrounds. The SDP is creating an institution for the development & maintenance.

ISO/IEC/IEEE 12207:2017(E) “*Systems and software engineering – Software life cycle processes*”

This standard is a successor to J-STD-016, which is a successor to MIL-STD-498, which is a successor to DOD-STD-2167A and DOD-STD7935A. (And that only dates to the 1980s!) It “does not prescribe a specific software life cycle model, development methodology, method, modelling approach, or technique.”

ISO/IEC/IEEE 15288:2015, *Systems and software engineering – System life cycle processes*

Wikipedia, *Software development process*

https://en.wikipedia.org/wiki/Software_development_process

Provides a history of the different contributions to software development processes

11.3. QUALITY MANAGEMENT, TEST

FDA, “*Design Control Guidance for Medical Device Manufacturers*,” 1997 March 11

IEEE Std 1012-2004 - *IEEE Standard for Software Verification and Validation*. 2005.
doi:10.1109/IEEESTD.2005.96278. ISBN 978-0-7381-4642-3.

ISO/IEC/IEEE 25010:2011, “*Systems and software engineering – Systems and software Quality Requirements and Evaluation (SQuaRE) - System and software quality models*” 2011

ISO/IEC/IEEE 29119, *Software Testing Standard*

ISO/IEC/IEEE 29119-1: *Concepts & Definitions*, 2013 September

ISO/IEC/IEEE 29119-2: *Test Processes*, 2013 September

ISO/IEC/IEEE 29119-3: *Test Documentation*, 2013 September

ISO/IEC/IEEE 29119-4: *Test Techniques*, 2015 December

ISO/IEC/IEEE 29119-5: *Keyword Driven Testing*, 2016 November

ISO/IEC 90003 *Software engineering – Guidelines for the application of ISO 9001:2008 to computer software*

[This page is intentionally left blank for purposes of double-sided printing]

CHAPTER 4

Specification

This chapter describes a recommend approach to writing a software requirements specification:

- An outline of a Software requirements specification
- Writing Requirements

12. STRUCTURE

Specifications outline the problem to solve, and the essential outcomes; these describe what an item must do, under what conditions. They address the natural tendency to be vague or ambiguous, often solving the wrong problems and targeting the wrong outcomes. It's important that the documentation be cohesive and cogent, it's to convey instill a clear understanding.

The following is an outline for a specification:

1. Introduction, usually providing a synopsis, the purpose and background of the product
2. Overview
 - a. Diagram, calling out key items referred to
 - b. Walk-thru of the intended use of the product
 - c. Potted product specification
 - d. Feature list
 - e. The interfaces, such as system interface, user interface, hardware interface, software interface, communication interface
 - f. Constraints such as memory, performance, energy, etc
3. Definitions
 - a. Kinds and entities
 - b. Properties and property values
 - c. Comparables, Comparisons, and Comparatives
 - d. Actions
4. Goals, Objectives
5. The states and modes of the software, their role, and state flow.
6. Safety model, Self-check / self-protect functions, watchdog, prioritization
 - a. Storage and data integrity
 - b. Communication and data integrity
7. Requirements
 - a. Functional requirements
 - b. Detailed logical controller specification
 - c. Performance requirement

8. Analysis to support the design or requirements
9. Appendices:
 - a. Glossary, acronyms
 - b. Related documents (documents that are part of the product)
 - c. References, resources, suggested reading

12.1. OVERVIEW

The document should outline the product. First, provide a big picture description, from the perspective of the final product that the customer or operator receive. Second, describe the specific to the sub-area that the software is too work in. (For embedded this is often a sub-assembly.) In some cases, additional overviews would be a good idea to bring into focus a very specific element within the big picture.

THE OVERVIEW should include:

- A diagram, calling out the items that will be discussed
- A description of what the product is, what role it serves, with some hint of its benefit
- A potted specification of the product

THE DIAGRAM. Each item in the document rest of the document should be called out thick call out lines, often red. Number the items and describe each in the text. Include the name or designator that will be used throughout the rest of the document. Below is an example.

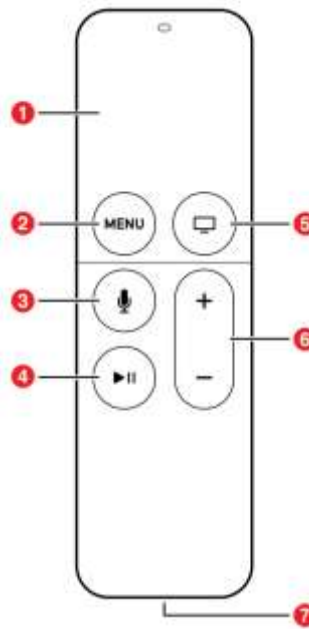


Figure 5: Example diagram with callouts

The elements external to the remote control are:

Element	Description
1 Touch surface	Swipe to navigate. Press to select. Press and hold for contextual menus
2 Menu	Press to return to the previous menu.
3 Siri/search	Press and hold to talk. Opens the onscreen search app.
4 Play/pause	Play and pause the media.
5 Play/pause	Press to return to the home screen. Press twice to view open apps. Press and hold to sleep.
6 Volume	Controls the TV value
7 lightning connector	Plug-in for charging

Table 4: Example table of diagram elements

THE DESCRIPTION of the product should capture what the product is, and its role. The descriptions should include a walkthrough of its primary operations. This overview should strive to be clear, and easy to understand. Assume that until the reader has seen this description the use is non-intuitive. Select the language carefully and define each of the terms used.

THE FEATURE LIST(s). Include a list of the key features. For example:

- Can control the TV volume
- Can control media playback with a “Jog” control
- Can search for media with a voice command

Tip: Similar (end) product-level diagrams and description may be used in many projects. Make a template document with a typical diagram and description. This can be used as a starting point in each related project.

12.2. ACTIONS

The key actions & activities of the product should be defined. Fortunately, not all actions need be spelled out in detail. Examples of actions whose detailed steps may be skipped: *actions*

- If the action is well-defined in a sourced document. If the action is defined in a sourced document, reference that document and its relevant portions explicitly.
- If the action is trivial, obvious and states the outcomes, such as the light being on.

The elements of a good action description:

- Actions should have their outcome stated or be clear from the text. For example, “turn the yellow LED on” is clear that the resulting state of the yellow LED is “on.”

Actions often take time, so have a beginning, middle, and end. Specify the minimum and maximum duration. Specify testable elements or parameter values during the time from beginning to end.

12.3. STATE

Include a description of the phases and states of operation, their roles, the activities performed in that state, and abnormal flows. One would typically use phases as a guided sequence, such as starting up, preparing the device for use, using it, then closing out. And include states for power management (e.g. off, sleep), and exceptional states such as faults.

Include a diagram of the states to show the flow. For example:

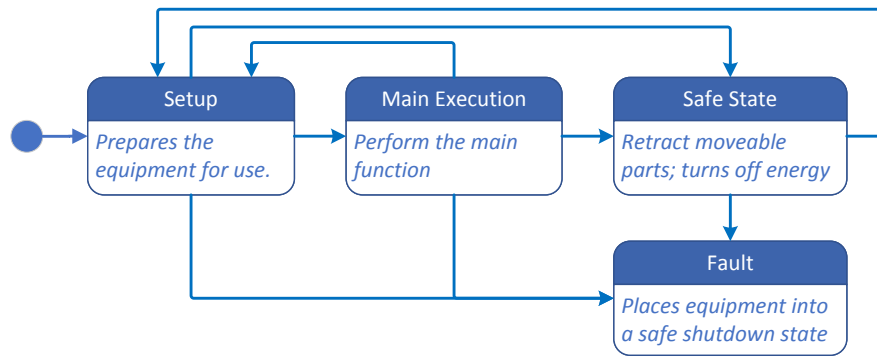


Figure 6: Example state flow diagram

Provide a structured, stylized description. Below is a template. Focus on the many readers will read the overall description, state flow and description, but not the requirements.

State name	
Role	Describe the purpose of the state, and its role. (Do not give its implementation.)
Pre-Conditions	Describe the conditions that must be true before the state can be entered and its actions can be performed. <i>Optional</i>
Expected outcome	Describe the desired result or end state that should be achieved when the state/action completes successfully. <i>Optional</i>
Normal sequence	Give a numbered, step-by-step flow of events that typically occur when the state is entered, or the action is performed under normal circumstances – its “happy path.”
Alternative, exceptional flows	Describe the different error conditions or paths that can occur when the normal sequence cannot be followed. These include error handling, edge cases, and alternative scenarios. Include if the state that the system should proceed (e.g. return) to if the conditions for this state are no longer met.

Table 5: Table template to describe a state

12.3.1 Example state description

Setup	
Role	The <i>Setup</i> state is used to wait for power on activities to complete, and the operator to setup the equipment for use.
Expected outcome	Enters the <i>Main Execution</i> state
Normal sequence	Various details here, such as 1. Performs initialization, 2. Power up self-tests, 3. Enter substate(s) to have the operator setup the equipment, and clear various interlocks 4. Proceeds to the <i>Main Execution</i> state.
Alternative,	If there is an error during initialization, it enters the <i>Safe State</i> or the <i>Fault state</i> .

exceptional flows	
-------------------	--

Main Execution	
Role	The <u>Main Execution</u> state is used to perform the intended operation of the device.
Pre-Conditions	The software and hardware successfully initialized, and equipment has been setup per the <i>Setup</i> state.
Expected outcome	The software remains in the <i>Main Execution</i> state until the device is turned off.
Normal sequence	<i>Various details here</i>
Alternative, exceptional flows	If there is an error during execution, it enters the <i>Safe State</i> or the <i>Fault state</i> .

Safe State	
Role	The <u>Safe State</u> is a context for behaviors that must occur when a <i>critical error</i> occurs. This eliminates potential sources of risk, usually by ceasing most operations, and places the equipment into a state that allows removal of removable devices.
Pre-Conditions	The software and hardware successfully initialized. <i>Not: the device may not be successfully setup for use.</i>
Expected outcome	The software remains in the <i>Safe State</i> until the device is turned off, or the error conditions have been addressed and successfully cleared.
Normal sequence	1. Turn off energy sources 2. Move moveable parts to a safe position to allow disassembly 3. Turn off remaining items 4. Wait for error condition to clear 5. Process to <i>Setup</i> state.
Alternative, exceptional flows	If there is an error during execution, it enters the <i>Safe State</i> or the <i>Fault state</i> .

Fault State	
Role	The <u>Fault State</u> is a context for behaviors that must occur when an unrecoverable error condition occurs that prevents the execution of the rest of the software. This ceases operations and disables the equipment.
Expected outcome	The software remains in the <i>Fault State</i> until the device is turned off.
Normal sequence	<i>Various details here</i>

12.3.2 Use states to organize description and requirements

Use the phases and states to grouping related requirements and to provide a context for them. Give each state a subsection, use parallel development in each. Each section should include:

- Description of what occurs during the phase/state. Define the terms. If there are sub-states – such as a sequence of activities that take time and have error flows – include a description of those.
- Describe the actions and events of this states. Describe the interlocks
- Provide the requirements for this state. All of the pre-conditions, normal actions, as well as the exceptional flows would have testable requirements.

12.4. APPENDIX

Note: the IEEE and many organizations prefer the references and glossary in the front of the document.

THE RELATED DOCUMENTS, SPECIFICATIONS. Include a designator for each document. Use this through the remainder of this specification to refer to the document.

THE REFERENCES, RESOURCES, SUGGESTED READING. Include a designator for each document. Use this through the remainder of this specification to refer to the document.

THE ACRONYM AND GLOSSARY TABLES. Define all acronyms, terms and phrases. We have all seen documents that include definitions for simple, common items (such as a LED), while not defining specialized items (such as “adaptive linear filter”) referred to heavily in a document. Don’t do this.

Tip: The acronyms and glossary are well suited for reuse in many projects. Make a stock document with the most common terms and potted definitions to be included in each project.

13. SOFTWARE REQUIREMENTS

The core of a *requirements specification* is a set of requirements. A *requirement* is an identifiable definition of what an item must do. Each must have clear text explaining or justifying it.

requirement

13.1. KINDS OF REQUIREMENTS

A *customer requirement* belongs in one of the top-level documents, but especially in the customer (or user) requirements specification.

customer requirement

Functional requirements describe:

functional requirement

- The *functions* a system should provide
- How the system should *react to particular inputs*
- How the system should *behave in particular situations*.
- What the system should *not do*.

Non-functional and domain requirements

Non-functional requirement

- Describe constraints on the services or functions offered by the system.
- These can be considerations like ease of maintenance, size, cost
- Context of Operation describes in which situation the system is to be used e.g. temperature, education standard of operators.

Safety requirements are requirements which ensure an adequate safety of a system.

safety requirement

These are separated grouped as:

- *functional safety requirements*, which identify appropriate response to potential hazards and failure conditions
- *non-functional safety requirements*,
- *context of operation*.

Note: this won't get into security requirements here.

What isn't a requirement: not all things that you wish to test actually are requirements, nor should be made into one. Use key responsibilities in the architecture or functions in the detailed design document instead.

13.2. THE REQUIREMENTS

Structure. A requirement is presented as text in a special form. A text should be separated from the text like:

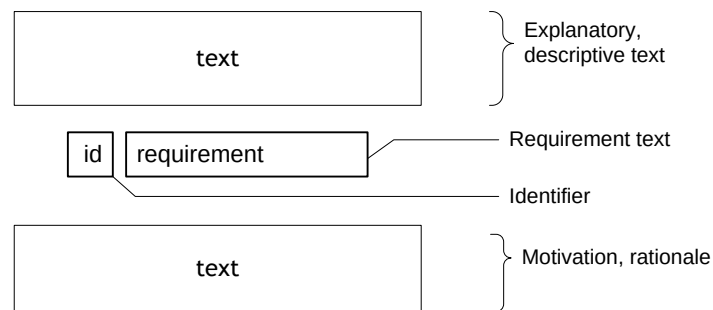


Figure 7: Typical requirement text

The presentation of a requirement in the text should include:

- *Clear demarcation* of the requirement. For instance, place the requirement on an indented line, by itself.
- A means to *uniquely identify* or refer to the requirement. It is important to be able to identify the requirement be discussed; in practice this is usually a label. The requirement will be referred to in other documents, trouble tracking, etc. *identifier*
- A brief *summary* of the requirement and its purpose or intent.
- *The actor* who carries out or meets the requirement. The actors should be defined earlier in the section or the document. *requirement*
- An identifiable definition of *what* an item must do.
- *Time bounds*: how fast, etc
- What value and bounds
- *Rationale*, usually to provide context, clarify or explain the requirement's role, purpose, motivation, and/or intent. This must be clear and readable. *rationale*
- *Source*. The origin of the values, such as a standard, parent document, lab notebook/report, and so on. The point is to reduce the arbitrariness.

- A *comment* is text, usually to provide context, clarify or explain the requirement(s). *comment*

13.3. PROPERTIES OF A GOOD REQUIREMENT

A well-written requirement has the following characteristics:

1. *Complete* – contains sufficient detail to guide the work of the developer & tester.
2. *Correct* – error free, as defined by source material, stakeholders & subject matter experts.
3. *Concise* – contains just the needed information, succinctly and easy to understand.
4. *Consistent* – does not conflict with any other requirement
5. *Unambiguous* – must have sufficient detail to distinguish from undesired behaviour. includes diagrams, tables, and other elements to enhance understanding.
6. *Verifiable* (or testable) – when it can be proved that the requirement was correctly implemented.
7. *Feasible* – there is at least one design and implementation for it.
8. *Necessary* – it is traced to a need expressed by customer, user, or stakeholder.
9. *Traceable* – can be traced to and from other designs, tests, usage models, etc. These improves impact assessment, schedule/effort estimation, coverage analysis.

13.4. THE SYNTAX OF REQUIREMENTS

The grammar used to refer to items, properties, actions, conditions, etc. should be consistent among the requirements. It should be rare that there is more than one-way use to express a clause. Below is a specific, stylized syntax for the requirements:⁹

The [actor] shall [action] <optional object> <optional trigger> <optional preconditions>

The table below synthesizes the main elements of the requirement, with an example:

<i>actor</i>	the system
<i>action</i>	create
<i>object</i>	an invoice
<i>trigger</i>	When an order is shipped
<i>precondition</i>	and the order is not prepaid

Table 6:
*Decomposition of
example requirement*

Requirements of the same “type” should employ the same format. Tip: maintain a handbook of templates or examples to represent key (or common) types of requirements.

⁹ Based on EARS. EARS has a slightly different normal order. I have kept the subject at the head of the sentence in each case.

The types of requirements and their typical phrase patterns:

Type	Pattern	Notes
Ubiquitous	The [actor] shall [action] <optional object>	Requirement is always active
Event-driven	The [actor] shall [action] <optional object> when [trigger] <optional precondition>	Required response to a triggering event
State-driven	The [actor] shall [action] <optional object> while [state]	Required response in a specified state
optional	The [actor] shall [action] <optional object> where [feature included].	Applicable only if feature is included
unwanted	The [actor] shall [action] <optional object> if [unwanted condition /event].	

Table 7: Summation of the typical requirements patterns.

13.4.1 Requirement for Ubiquitous behaviours

A ubiquitous requirement states a fundamental system property or behaviour. It defines system behaviour or features that must always be active or present. This type has the form:

The [actor] shall [action] <optional object>

For example:

FOOBAR123: The product shall provide a connector to the mains power.

Note: This form of requirement is rare. When reviewing, look for missing triggers and other conditions on the requirement.

13.4.2 Requirement for Event-driven behaviours

Event driven requirements are initiated only when a triggering event is detected. The trigger must be something that the system itself can detect. This type has the form:

The [actor] shall [action] <optional object> when [trigger] <optional preconditions>

For example:

FOOBAR124: The light driver shall turn the light off if light switch is set to the off position.

Notes on the trigger can be:

- A defined (named) event
- When a property changes to a value
- When a property changes from a value

Notes on the optional pre-conditions – may need to differentiate state and pre-condition.

13.4.3 Requirement for State-driven behaviours

State-driven requirements are active (or “continuous”), while the system is in the specified state. This type has the form:

The [actor] shall [action] <optional object> while [state]

13.4.4 Requirement for optional features

This type of requirement has the form:

The [actor] shall [action] <optional object> where [feature included]

13.4.5 Requirement for unwanted behaviours

This is a variation of event-driven requirement. The trigger or condition is one that is unwanted, error condition, failure, fault, disturbance, etc. These have the form:

The [actor] shall [action] <optional object> if [unwanted condition/event]

13.4.6 Complex requirements

These employ decision tables, and other forms.

13.5. GRAMMAR OF REQUIREMENT

Not everything is a requirement.

1. The functional requirements must clearly specify the actor and what it should do – how it should behave.
2. Use only a single “shall” and only a single action per requirement
3. Do not couch implementation (or design) as a requirement.

Do not use:

- subjective language,
- non-verifiable terms
- adverb disjunctions, or adverb adjuncts.
- uncomparable adjectives.
- ambiguous adverbs and adjectives,
- superlatives
- negative statements.
- Comparative phrases
- terms implying totality

14. REFERENCES AND RESOURCES

Arora, Chetan; Mehrdad Sabetzadeh, Lionel Briand, Frank Zimmer. “Automated Checking of Conformance to Requirements Templates Using Natural Language Processing” *IEEE Trans Software Engineering*, vol 41, n10, 2015 p944-967

Gilb, Tom. *Planguage* 1988

Gilb, Tom. *Competitive Engineering*, 2005 Butterworth-Heinemann

NASA “Appendix C: How to write a good requirement” in *NASA Systems Engineering Handbook*

Wikipedia, *Software requirements specification*

https://en.m.wikipedia.org/wiki/Software_requirements_specification

14.1. EARS: EASY APPROACH TO ENGINEERING

Mavin, Alistair. *EARS Tutorial Easy Approach to Requirements Syntax*

Rupp, Chris. *“Easy Approach to Requirements Engineering”* (EARS)

Terzakis, John. *EARS: The Easy Approach to Requirements Syntax*

“The hardest part of the software task is arriving at a complete and consistent specification, and much of the essence of building a program is in fact the debugging of the specification.”
– Fred Brooks

[This page is intentionally left blank for purposes of double-sided printing]

CHAPTER 5

Requirements Checklists

This chapter provides a requirements review checklist.

15. OVERVIEW OF WELL WRITTEN REQUIREMENTS

The presentation of a requirement in the text should include:

- *Clear demarcation* of the requirement. For instance, place the requirement on an indented line, by itself.
- A means to *uniquely identify* or refer to the requirement. It is important to be able to identify the requirement be discussed. The requirement will be referred to in other documents, trouble tracking, etc.
- A brief *summary* of the requirement and its purpose or intent.
- *The actor* who carries out or meets the requirement. The actors should be defined earlier in the section or the document.
- *What* the actor is to do
- *Source and Rationale*, the description of the requirements role, purpose, motivation, and/or intent must be clear and readable; along with sources that these come from. The quantities for time and values have sources and rationale.

15.1. PROPERTIES OF A GOOD REQUIREMENT

A well-written requirement exhibits the following characteristics:

- *Complete* – contains sufficient detail to guide the work of the developer & tester
- *Correct* – error free, as defined by source material, stakeholders & subject matter experts
- *Concise* – contains just the needed information, succinctly and easy to understand
- *Consistent* – does not conflict with any other requirement
- *Unambiguous* – must have sufficient detail to distinguish from undesired behaviour. includes diagrams, tables, and other elements to enhance understanding.
- *Time bounds*: how fast, how long, how soon it acts or when, etc.
- What *value* and *bounds*; the quantities have units

- *Verifiable* (or testable) – when it can be proved that the requirement was correctly implemented
- *Feasible* – there is at least one design and implementation for it.
- *Necessary* – it is traced to a need expressed by customer, user, stakeholder.
- *Traceable* – can be traced to and from other designs, tests, usage models, etc. These improves impact assessment, schedule/effort estimation, coverage analysis scope management/prioritization

Note: a requirement must not be the “design” in disguise; it must come from the input requirements and architecture. The statements that represent the design are called *key responsibilities* and *responsibilities*; these live in the architecture and detailed design documents.

16. REQUIREMENTS REVIEW CHECKLIST

See also

- Appendix F for the *Code Complete* Requirements Review check lists

Names:

- ☐ Are the names clear and well chosen? Do the names convey their intent? Are they relevant to their functionality?
- ☐ Is a good group / naming convention used? (e.g. related items grouped by name)
- ☐ Is the name format consistent?
- ☐ Names only employ alphanumeric characters?
- ☐ Are there typos in the names?

16.1. ARE THE PROPERTIES, STATES AND ACTIONS WELL DEFINED?

- ☐ Is a definition duplicated?
- ☐ Is a property defined multiple different times... but defined differently?
- ☐ Are the definitions complete?
 - Are all instances and kinds defined – or some missing?
 - Are there undefined (i.e., referred to, but not defined) nouns, properties, verbs?
 - Are events referred to but not defined?
- ☐ Are they consistent?
- ☐ Are the properties something that the system can measure or otherwise detect?
- ☐ Are the instances something that the system can identify or otherwise distinguish?
- ☐ Is a state not needed? Is it used by a state classification, action, event, or requirement?
- ☐ Is a property not needed? Is it used by a state classification, action, event, or requirement?
- ☐ Are the properties something that the system can detect?
- ☐ Are the events something that the system can detect?

16.2. REQUIREMENTS REVIEW

Reviewing requirements should look to identify:

- ☐ Are the requirements organized in a logical and accessible way?
- ☐ Is the requirement clearly demarcated?
- ☐ Does the requirement have a clear and fixed identifier? Is the identifier unique?
- ☐ Is the description supporting the requirement clear? Is it sufficient to support the requirement?
- ☐ Is the requirement too wordy? A requirement should be concise, containing just the needed information.
- ☐ Does the requirement use the proper modal auxiliaries?
- ☐ Does the requirement have the right conditions? The ubiquitous form of requirement is rare. Look for missing triggers and other conditions on the requirement.
- ☐ Are the time-critical features, functions and behaviours identified? Are the timing criteria specified?
- ☐ Is there requirement declarative? Or is the requirement an attempt to repackage an existing implementation with imperative statements? These are bad.
- ☐ Does the requirement conflict with any other requirement? Is its use of conditions (e.g. thresholds) consistent with the other requirements?
- ☐ Is the action to carry out clear? Is the action well defined within the rest of the specification?
- ☐ Are the actions something that can be accomplished?
- ☐ Duplicated requirements?
- ☐ Ambiguity. Can the requirement be interpreted different ways? Is there sufficient detail to distinguish from undesired behaviour?
- ☐ Is the requirement vague or ambiguous in any way? Pronouns, demonstratives, and indexicals often introduce ambiguity.
- ☐ Is the requirement specifying a single action... or many? A requirement should specify only a single action.
- ☐ Complexity. Is the requirement over specified, too complex?
- ☐ Requirements that are too expensive, burdensome, impractical or impossible
- ☐ Are the requirements ones that fit the practical use with customer wants/needs/etc?
- ☐ Is the requirement unnecessary? Does it lack a trace to a need expressed by customer, user, or stakeholder? Is each requirement traceable to a customer that requires it?
- ☐ Check for consistency and sufficient definition
- ☐ Does the requirement have errors, such as misstating bounds, or conditions in the source material, or from other stakeholders or subject matter experts?
- ☐ Are there missing requirements? Is there a lack of sufficient detail to guide the work?

16.3. ARE THE REQUIREMENTS TESTABLE?

- ☐ Are the triggers something that the system can detect?
- ☐ Is the action or result of the requirement observable? Can it be measured?
- ☐ Are the quality requirements measurable?
- ☐ Is the requirement time bound? Is there a clear time bounds between the condition or trigger, and the action?

- ☐ Is the requirement untestable? Is there a direct means of stating how to test that the requirement was correctly implemented?
- ☐ Is the actor to carry out or meet the requirement clear? Is the actor well-defined within the rest of the specification?
- ☐ Are the actions testable? Is their outcome testable?
- ☐ Is the requirement bounded? Or is the actor allowed to do the requirement at the end of the universe?

16.4. THE LEADS REVIEW REQUIREMENTS

- ☐ Are they complete? Are requirements or definitions missing? Are there undefined nouns, properties, verbs?
- ☐ Are they consistent?
- ☐ Are they doable?

CHAPTER 6

Software Risk Analysis

This chapter provides an initial template for software risk analysis.

17. SOFTWARE REQUIREMENTS RISK ANALYSIS

The outputs of a software requirements risk analysis include:

- A table mapping risks to the requirements that address it. *This table may have been produced by another activity and is only referenced in the output*
- A list of software risks, acceptability level, and their disposition
- A criticality level for each hazard that can be affected by software
- Recommended changes to the software requirements specification, programmable system architecture, etc. For example, actions required of the software to prevent or mitigate the identified risks.
- Recommended Verification & Validation activities, especially tests

The steps of a software requirements risk analysis include:

1. Identify the requirements that address each product hazards
2. Examine the risks of errors with values
3. Examine the risks of message capacity
4. Examine the risks of timing issues
5. Examine the risks of software functionality
6. Examine the risks of software robustness
7. Examine the risks of software critical sections
8. Examine the risks of unauthorized use
9. Recommendations for rework

17.1. STEP 1: IDENTIFY THE REQUIREMENTS THAT ADDRESS PRODUCT HAZARDS

Go thru each identified product hazard and list the software requirements that address it.

17.2. STEP 2: EXAMINE ALL VALUES FOR ACCURACY

Identify all the elements of the system – sensor 1, sensor 2, actuator, motor, calculations, operator inputs, operator outputs, each parameter received, etc. For each of these elements, create a copy of the table below and populate it with an analysis with respect to the requirements. Strike inapplicable conditions and add other identified conditions.

Condition	Hazard, likelihood & severity
the value is off by 5% of the actual value	
the value is stuck at all zeroes	
the value is stuck at all ones	
the value is stuck at some other value	
the value is too low; the value/ result is below minimum range	
the value is within range, but wrong; with calculation, e.g. the formula or equation is wrong	
the physical units are incorrect	
the value is incorrect (<i>for non-numerical values</i>)	
the value type, or format size is wrong	

Table 8: Value accuracy risks

In reviewing each condition, identify the least acceptable risk for each applicable condition. Other documents (i.e. the product safety risk analysis) are responsible for the identifying the set of possible hazards and their severity. The table below provides example likelihood levels; **Table 10** provides an example mapping of severity & likelihood pair to risk acceptability.

Likelihood	Estimate of Probability
Frequent	Likely to occur on in the life of an item, with a probability of occurrence greater than 10^{-1} in that life.
Probable	Will occur several times in the life of an item, with a probability of occurrence less than 10^{-1} by greater than 10^{-2} in that life
Occasional	Likely to occur sometime in the life of an item, with a probability of occurrence less than 10^{-2} but greater than 10^{-3} in that life.
Remote	Unlikely, but possible to occur in the life of an item, with a probability of occurrence less than 10^{-3} but greater than 10^{-4} in that life.
Improbable	So unlikely, it can be assumed occurrence may not be experienced, with a probability of occurrence of less than 10^{-4} in that life.

Table 9: Hazard probability levels based on Mil-Std 882

	Catastrophic	Critical	Marginal	Negligible
Frequent	high	high	high	medium
Probable	high	high	medium	low
Occasional	high	high	medium	low
Remote	high	medium	low	low
Improbable	medium	low	low	low

Table 10: An example risk acceptability matrix determining risk acceptability

17.3. STEP 3: EXAMINE THE MESSAGES CAPACITY

This step examines the ability of the software to achieve its objectives within the hardware constraints.

Identify all the messaging elements of the system – I²C sensor, task 1, user input, etc. For each of these elements, create a copy of the table below and populate it with respect to the requirements. Strike inapplicable conditions and add other identified conditions.

Condition	Hazard, likelihood & severity
message is smaller than state minimum	
message is larger than stated maximum	
message size is erratic	
messages arrive faster than stated maximum (e.g. response time)	
messages arrive slower than stated minimum (e.g. response time)	
message contents are incorrect, but plausible	
message contents are obviously scrambled	

Table 11: Message capacity risks

In reviewing each condition, identify the least acceptable risk for each applicable condition.

17.4. STEP 4: EXAMINE THE CONSEQUENCES OF TIMING ISSUES

This step examines the ability of the software to achieve its objectives within the hardware constraints.

Identify all the input elements of the system – button #1, frequency input, I²C sensor, task 1, user input, etc. This list should include elements those receive messages and send messages. For each of these elements, create a copy of the table below and populate it with respect to the requirements. Strike inapplicable conditions and add other identified conditions.

Condition	Hazard, likelihood & severity
input signal fails to arrive	
input signal occurs too soon	
input signal occurs too late	
input signal occurs unexpectedly	
input signal occurs at a higher rate than stated maximum	
input signal occurs at a slower rate than stated minimum	
system behavior is not deterministic	
output signal fails to arrive at actuator	
output signal arrives too soon	
output signal arrives too late	
output signal arrives unexpectedly	
insufficient time allowed for operator action	

Table 12: Timing capacity risks

In reviewing each condition, identify the least acceptable risk for each applicable condition.

17.5. STEP 5: EXAMINE SOFTWARE FUNCTION

This step examines the ability of the software to carry out its functions.

Identify all the functions of the system; the operations which must be carried out by the software. For each of these elements, create a copy of **Table 13** (below) and populate it with respect to the requirements. Strike inapplicable conditions and add other identified conditions.

Condition	Hazard, likelihood & severity
Function is not carried out as specified (for each mode of operation)	
Function preconditions or initialization are not performed properly before being performed	
Function executes when trigger conditions are not satisfied	
Trigger conditions are satisfied but function fails to execute	
Function continues to execute after termination conditions are satisfied	
Termination conditions are not satisfied but function terminates	
Function terminates before necessary actions, calculations, events, etc. are completed	
Function is executed in incorrect operating mode	
Function uses incorrect inputs	
Function produces incorrect outputs	

Table 13: Software function risks

In reviewing each condition, identify the least acceptable risk for each applicable condition.

17.6. STEP 6: EXAMINE SOFTWARE ROBUSTNESS RISKS

This step examines the ability of the software to function correctly in the presence of invalid inputs, stress conditions, or some violations of assumptions in its specification.

Create a copy of **Table 14** (below) and populate it with respect to the requirements. Strike inapplicable conditions and add other identified conditions.

Condition	Hazard, likelihood & severity
Software fails in the presence of unexpected input signal/data	
Software fails in the presence of incorrect input signal/data	
Software fails when anomalous conditions occur	
Software fails to recover itself when required	
Software fails during message, timing or event overload	
Software fails when messages are missed	
Software does not degrade gracefully when required (e.g. crashes instead)	

Table 14: Software robustness risks

In reviewing each condition, identify the least acceptable risk for each applicable condition.

17.7. STEP 7: EXAMINE SOFTWARE CRITICAL SECTIONS RISKS

This step examines the ability of the system to perform the functions that address or control risks.

Create a copy of **Table 15** (below) and populate it with respect to the requirements. Strike inapplicable conditions and add other identified conditions.

Condition	Hazard, likelihood & severity
Software causes system to move to a hazardous state	
Software fails to move system from hazardous to risk-addressed state	
Software fails to initiate moving to a risk-addressed when required to do so	
Software fails to recognize hazardous state	

Table 15: *Software critical sections risks*

In reviewing each condition, identify the least acceptable risk for each applicable condition.

17.8. STEP 8: UNAUTHORIZED USE RISKS

Create a copy of **Table 16** (below) and populate it with respect to the requirements:

Condition	Hazard, likelihood & severity
Unauthorized person has access to software system	
Unauthorized changes have been made to software	
Unauthorized changes have been made to system data	

Table 16: *Unauthorized use risks*

In reviewing each condition, identify the least acceptable risk for each applicable condition.

17.9. STEP 9: RECOMMENDATIONS FOR REWORK

Summarize each of the identified conditions with an unacceptable risk level. These items mandate rework, further analysis, and/or Verification & Validation activities.

18. REFERENCE DOCUMENTS

MIL-STD-882E “*Standard Practice System Safety*” 2012 May 11

NASA-GB-8719.13, *NASA Software Safety Guidebook*, NASA 2004-3-31

NASA-STD-8719.12, *NASA Software Safety Standard*, Rev C 2013-5-7

UCRL-ID-122514, J Dennis Lawrence, *Software Safety Hazard Analysis* Rev 2, U.S. Nuclear Regulatory Commission, 1995-October

[This page is intentionally left blank for purposes of double-sided printing]

“A good engineer tries to get something not to work – that is, after getting it working, the good engineer tries to find its limits and make sure they are well-understood and acceptable.”

– Michael Covington

PART II

Software Design & Documentation

This part provides guides for software design and its documentation

- OVERVIEW & WRITING TIPS.
- OVERVIEW OF SOFTWARE DESIGN.
- GUIDELINES FOR ARCHITECTURE. Provides guidelines for high-level designs.
- SOFTWARE ARCHITECTURE RISK ANALYSIS.
- GUIDELINES FOR DETAILED DESIGNS. Provides guidelines for detailed designs (e.g. major subsystems or “stacks”).
- PROTOCOL DOCUMENTATION TEMPLATE. Provides a guide for protocol documentation.
- PROGRAMMER DOCUMENTATION. The software documentation primarily for software developers, often created with *doxygen*, *docfx*, and *markdown*.
- SOFTWARE MODULE DOCUMENTATION TEMPLATE. Provides a guide for detailed design documentation of a module.
- DESIGN REVIEWS CHECKLISTS. Provides checklists for reviewing a design.
- SOFTWARE DETAILED DESIGN RISK ANALYSIS REVIEWS. Describes reviewing software for risk analysis.

[This page is intentionally left blank for purposes of double-sided printing]

“Design is the higher-level understanding of how we decompose the system into interacting parts.”
– Eric Normand

CHAPTER 7

Design Overview & Writing Tips

This chapter gives some the recommendations for design documentation

- The role and characteristic of design documentation
- Organization of the documentation

19. THE ROLE AND CHARACTERISTICS OF DESIGN DOCUMENTATION

This chapter describes my recommendations for writing design documentation. The role of documentation is to

- Provide assurance to an outside reviewer – one without the tacit knowledge that the developing team and organization will share – that the product is well-crafted and suitable for its intended purpose and will achieve the safety & quality requirements.
- Communicate with future software development, and test teammates; and to reduce the puzzles and mysteries when handed a completed software implementation with the expectation to make it work/modify it/test it.
- Drive clarity of thought on the part of the designers; experience has repeatedly shown that if it can't be explained clearly, it isn't understood. A lack of understanding impairs product quality and creates project risk (thrashing).

The design documentation

- Establishes the shape of the software modules
- Shows how the design addresses the software requirements and other specifications
- Provides a mental map of the design, making the design understandable.

Characteristics of a good design description include:

- A straightforward mapping to the implementation
- Mixing visuals and text to explain the concepts in alternate ways
- Scoping diagrams so that the amount to hold in the readers head is small

The requirements at the design stage should provide a clear enough view to allow the high-level design to be crafted.

19.1. TIPS ON THE WRITING PROCESS

This section focuses on presenting the design as much as it does on tips for crafting a design. I've found that most engineers dislike documentation and defending in detail their designs.¹⁰ That's a pity, since the documentation is a necessary skill in quality software domains (such as safety critical products), and an important one to project success. Some tips:

1. Read & study good examples of design and design documentation. These can be found in books like McKusick (2004), and Kehan (1987). Other examples might be found in application notes, and past projects *McKusick, Marshall Kirk and George V. Neville-Niel. The Design and Implementation of the FreeBSD Operating System, 1st Edition Addison-Wesley Professional; 1st edition, 2004*
2. Use templates and writing guides. They provide the scope and main outline, reducing the burden of how to organize the documentation.
3. Plan on the design in stages of completeness: a preliminary version of the design before the development begins in earnest, revisions during design discussions, and a finished design at the end of the project.
4. Take the writing in small, doable pieces. Write the document in a series of drafts, targeting only a few pages a day. Revise the draft and repeat.
5. Start with the areas that you know what to write; don't necessarily worry about starting with lower-level design documents if that is what you know. They won't be committed to yet, but the information and experience will help write the upper layers. *Kenah, Lawrence; Ruth Goldenberg. VAX/VMS Internals and Data Structures: Version 5.2, Digital Press, 1987*
6. Then work down from the top – or up from the bottom – in a vertical slice relevant to what you do know. Add in organizational material (such as outlines for the section) or an expository explanation and keep moving. Use stubs – such as “TBD” – for specific values, names or other references that you do not know yet.

19.2. AUDIENCE

The audience for the design documentation includes:

- The certifying body
- The development team
- Management, such as project manager
- The regulatory affairs department (i.e. the design history file)
- Release engineering
- The software and system test group
- Technical publications

20. DOCUMENTATION ORGANIZATION

One of the first steps in the developing the documentation is to pick a style or organization for the documentation. This will help layout (block) the overall documentation, and provide not just the structure, but the start of an outline and size of the work to accomplish the scope.

¹⁰ I've also found that the best designs, the highest quality ones, were produced quickly by designers who will love talking with others about their designs, and crave to find others who will appreciate it.

Use a *concentric approach* throughout:

concentric approach

“The concentric approach, often called spiral, is a way of organizing ... by laying out basic concepts, covering other related material, and then circling back around to the basic concept and filling in more complexity and depth.”

An expansive, large system is best served with documentation that subdivides the design into several large portions with a mid-level design and then breaks into detailed design of the individual components.

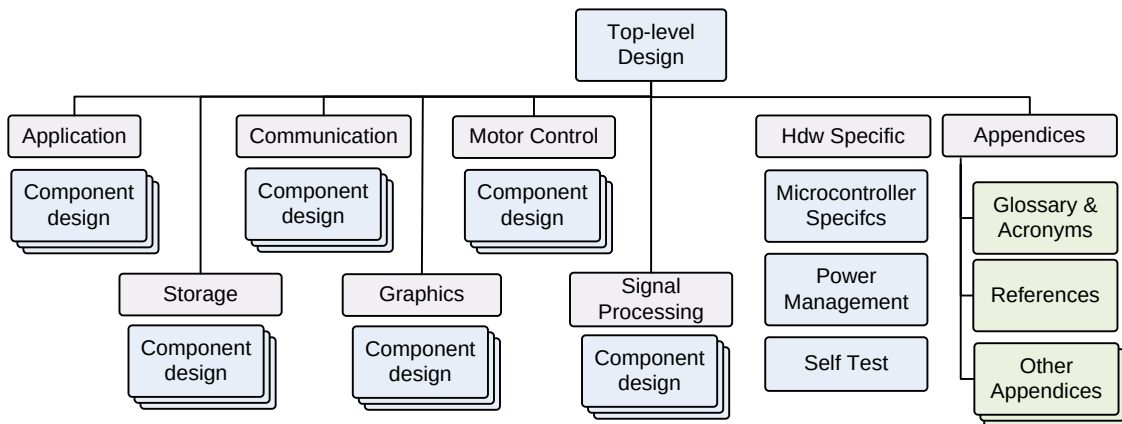


Figure 8: Structure of a broad design with moderate fan out

A moderate, small system might be best served with documentation that introduces the high-level design and then breaks into detailed design of the (many) individual components.

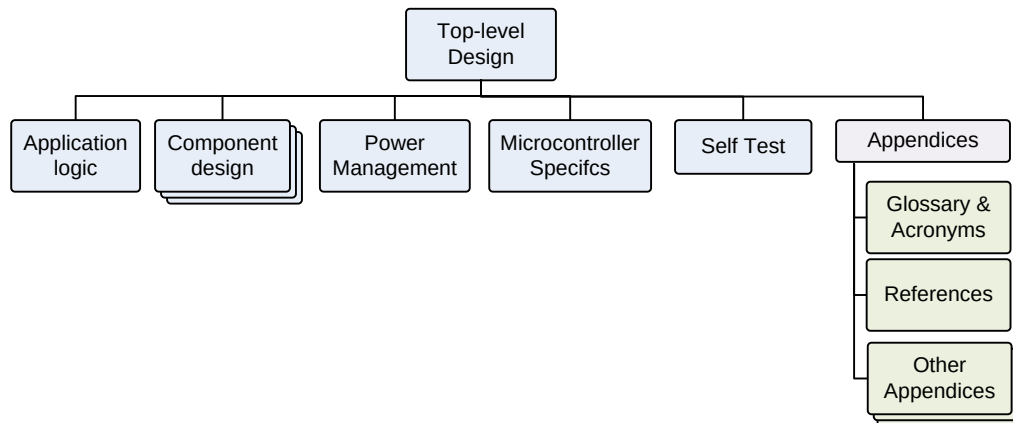


Figure 9: Structure of a mid-size design, with high fan out

Note: Major portions of the structure may be mandated by the standards the product is being certified against; and by the developing institutions processes. Many, for example, mandate the presence of the references and glossary, but that they be in the front of the document.

The *detailed design* is actually an ensemble of documents that work together to provide the description of how the software is intended to work, rationale for why decision choices were made, external interfaces to operate the software and programmer details of the data structures, procedures, and so on.

detailed design

- The *main detailed design* document – most often what is meant when referring to a detailed design. This provides the description of how the software achieve its functions, and rationale for the design choices. The architecture would typically trace the design to this

document, and the requirements would also trace to this document. This and the architecture can be a single document. Typically should be less than 150 pages.

- The documentation of *communication protocols* and *storage formats* – often called *interface control documents*. Especially those external to the software. *interface control document*
- The *programmer documentation* is often made with tools such as *doxygen* or *docfx* to format source code comments, augmented with structuring markdown files. This type of document is a supplement and must not be confused with the main detailed design above – *the code is not the documentation!* The unit tests would trace to this document, usually the file listing portion. This type of generated document is often large and hard to read. But it is possible to make it readable. *programmer documentation*

20.1. TERMS AND PHRASES TO EMPLOY

Once the broad structure is selected, begin thinking about the terms and phrases that are and will be used in the project, and your approach in the documentation:

- What terms and phrases will be used?
- Which will not be used in the documentation?
- What additional terms and phrases should be given a translation to the project – a mapping to the terms and phrases used in the design documentation?

The standards (to which the product may be certified against), and the specifications will already be employing a stock of terms and phrases. The design doesn't necessarily need to use them (and it isn't always warranted). In that case, the design document should provide a definition of what those terms and phrases are in within the project.

The terms and phrases used should make sense for the project and design. They could come from

1. The requirements and other specifications for the product
2. The jargon used within the rest of the organization, or team
3. Other conventions, such as the industry jargon.

The design should address the terms and phrases of the standards and specifications that are not otherwise used in the design documentation. This can be achieved by providing a mapping of these terms and phrases to those in the design.

20.1.1 Tips for getting the definitions for standards terms

Most standards provide a glossary of the terms and phrases that they use. However, the definitions within a standard can sometimes be unclear, confusing or otherwise not helpful. Fortunately, there are resources that can be used to gather variations of the definition to help clarify the term or phrase.

The IEEE provides a glossary of terms in IEEE Std 610.12-1990.

There are two search-tool resources than can be used to look up the definitions across IEC standards:

- <http://www.electropedia.org/>
- <http://std.iec.ch/glossary>

And for ISO standards:

- <https://www.iso.org/obp/ui>

20.2. APPENDICES

The design documentation often includes several appendices. The ones described here

- Acronyms and Glossary
- References, Resources and Suggested Reading

The following can be placed into the appendices of the programmer's guide as they are generally out of focus for the main detailed design. Second, several of these can use the same tools to aid their generation (esp. on large projects) as the rest of the programmer's guide.

- Configuration of the compiler, linker and similar tools
- Configuration or settings of the analysis tools and similar
- The files used in the project
- The configuration of the software. This is often divided into application and board-specific configurations.

20.2.1 The Acronyms and Glossary

I recommend that the list of acronyms and glossary be in the rear of the documentation. In some development protocols it is preferred that they be in the front of the documentation.

In this appendix, define all acronyms, terms and phrases. We have all seen documents that include definitions for simple, common items (such as a LED), while not defining specialized items (such as "adaptive linear filter," or "hybrid turboencabulator") referred to heavily in a document. Don't do this.

I recommend that the expansion of acronyms to its words be presented in a separate list from the definition of terms and phrases.

This appendix is "living," the acronyms, terms and phrases will continue to expand and be added to throughout the development. In good documentation the glossary can be extensive.

Tip: The acronyms and glossary are well suited for reuse in many projects. Make a stock document with the most common terms and potted definitions to be included in each project.

20.2.2 The References, Resources and Suggested Reading

I recommend that the references be in the rear of the documentation, excluding (perhaps) the list of standards that are inputs to the design. In some development protocols it is preferred that they be in the front of the documentation.

In this references appendix the list should include data sheets, industry and legal standards, communication protocols, etc. Include a designator for each document. Use this through the remainder of this specification to refer to the document.

20.2.3 Files

The detailed design often includes an appendix or attachment listing the files. How much to list depends on the stringency of development. Exhaustively listing the files is simply no joy. The list should include

- Files with strange names
- Files with particular importance

The list should *not* include temporary or generated files (the object files, assembly listings, temporary files, etc.). Describe each file and its role.

Can use groupings, folders and names to help organize the names.

20.3. REUSING DESIGNS AND DOCUMENTATION

Some observations on design and documentation reuse:

- Software libraries are one way to reuse designs. However, their design, creation, and support are a development effort in and of itself, with many factors that impair success.
- The high-level design ideas are readily reused; so are specific low-level modules (e.g. digital input, output, analog conversion, etc.).
- Good design practices facilitate easier reuse.

Some approaches and techniques to help promote reuse of designs:

- Divide the documentation in pieces that can be reused
- Provide a segment of time (e.g. at the end of the project) for reviewing and identifying reusable sections.
- Identify, during design reviews, areas where prior design *should*¹¹ have been reused.

Existing designs are (mostly) worked out; their reuse can accelerate a project schedule – *if* the design is appropriate to the project. Such design must be stored in manner so that it is accessible, easily found, and readily reusable. Each successive project may contribute to the collection of reusable design pieces. I recommend:

1. Break out each chapter into its own file.
2. Create an overall structuring document for the design documentation.

To merge these into a single design document for release with a project:

1. Make a copy of that overall structuring document
2. Insert each of the files for the chapter

When it comes time to do another project, the chapters of the past projects serve as a starting point for documentation reuse. The levels of documentation & design reuse:

1. Verbatim: The chapter is picked up and used without changes.
2. The chapter is copied and modified it to adapt it to the project.
3. A template document is reused where the structure is used largely unchanged, but the contents are customized for the new project. This avoids re-inventing the structure, and while using a fill-in-the-blank approach.

21. REFERENCES AND RESOURCES

Alred, Gerald, Charles Brusaw, Walter Oliu; *Handbook of Technical Writing*, 10th Ed, St Martin's Press, 2011

IEEE Std 610.12-1990, *IEEE Standard Glossary of Software engineering Terminology*, 1990.

¹¹ Note the emphasis is on *should*, not *could*. This step may be fraught with politics, which will undermine quality

CHAPTER 8

Architecture Template

This chapter is a high-level design description (or *architecture*) document template.

22. BASIC OUTLINE

The structure of top-level design changes the most between projects. Often these are treated as “living” – the document regularly evolves, starting from an initially skeletal design to one that is fleshed out over time. The following is an outline for a design description:

1. Synopsis
2. Other front matter
 - a. Related documents and specifications (those that are part of the product)
3. Design overview. Detailed block diagram of the software organization. This should include the IO, communication, power management, sensors, drivers, control loops and other subsections that will be described in detail in the rest of the document.
4. The states and modes of the software, their role, and state flow.
5. The software items (e.g. subsystems or major modules), and their key responsibilities.
6. Safety model, Self-check / self-protect functions, watchdog, prioritization
 - a. Storage and data integrity
 - b. Communication and data integrity
7. The interfaces to the software, including the data formats and sequences. This includes storage, external and internal communication protocols.
8. Time keeping
9. Sensors, the signal chain and other engagement with hardware
10. Power management
11. Appendices:
 - a. Glossary, acronyms
 - b. References, resources, suggested reading
 - c. Identify 3rd party software used, and the responsibilities of each

22.1. SYNOPSIS AND FRONT MATTER

THE SYNOPSIS. A one or two paragraph synopsis of what the software’s role in the product is.

THE RELATED DOCUMENTS & SPECIFICATIONS. This is a list of internal organization & project standards, and design specifications, with a designator for each document. The designator is to be used through the remainder of this specification to refer to the document.

22.2. THE GLOSSARY, REFERENCES

Next, prepare place holders for the common elements, such as the acronyms, glossary of terms, and references. These are “living,” they will continue to expand and be added to throughout the development. In good documentation the glossary can be extensive.

Note: I recommend the following be in the rear of the documentation, along with all the other supplemental information.

THE ACRONYM AND GLOSSARY TABLES. Define all acronyms, terms and phrases. We have all seen documents that include definitions for simple, common items (such as a LED), while not defining specialized items (such as “adaptive linear filter,” or “hybrid turboencabulator”) referred to heavily in a document. Don’t do this.

I recommend that the expansion of acronyms to its words be presented in a separate list from the glossary definition of terms and phrases.

Tip: The acronyms and glossary are well suited for reuse in many projects. Make a stock document with the most common terms and potted definitions to be included in each project.

THE REFERENCES, RESOURCES, SUGGESTED READING. The documents to list include data sheets, industry and legal standards, communication protocols, etc. Include a designator for each document. Use this through the remainder of this specification to refer to the document.

22.3. DESIGN OVERVIEW

Describe the role and responsibility of the software. Include the functions and features that the software is responsible for providing.

Include a diagram summarizing the software design, with the major sections and their interconnections. This may include a reference to external elements that it controls or depends on. It should show the basic structure of the signal flow (both control signals and information signals) and include a description introducing to how inputs are turned into outputs. It should provide context and show key external elements.

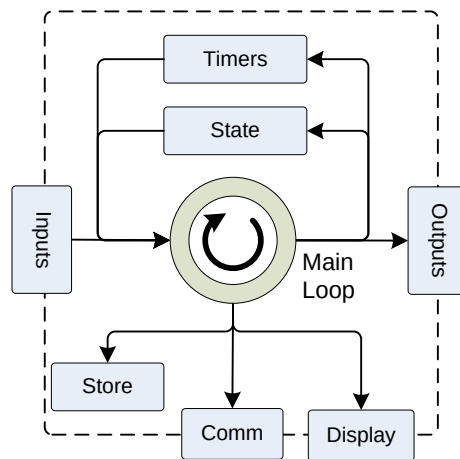


Figure 10: Basic flow structure of the software

Provide a description of the main elements of the software design:

Element	Description
element 1	Description of the element
...	
element n	Description of the element

Table 17: The software design elements

The external elements are:

External Element	Description
element 1	Description of the element
...	
element n	Description of the element

Table 18: The external elements

Note: this diagram (architecture) is often stylized and reused across products as a platform or design style. Requirements may be written against an abstract model based on it.

22.3.1 Partitioning into a Two processor model

Consider separating the more stringent functions (such as safety critical functions) from the main – but less stringent functions – by placing them into a separate processor.

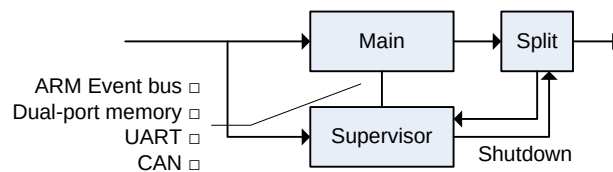


Figure 11: Processor with a supervisor processor

The second processor monitors condition and places the system into a safe state if the main processor or system conditions leave a well-defined safe operating state.

23. STATE AND BEHAVIORS

The interfaces and behaviors of the software should be documented. These are its inputs, outputs, how it interacts with the rest of the system, and responds to events. This would include any states of the software or interaction with it, along with the state flow, and how it reacts to specific events.

Include a diagram of the states to show the flow. For example:

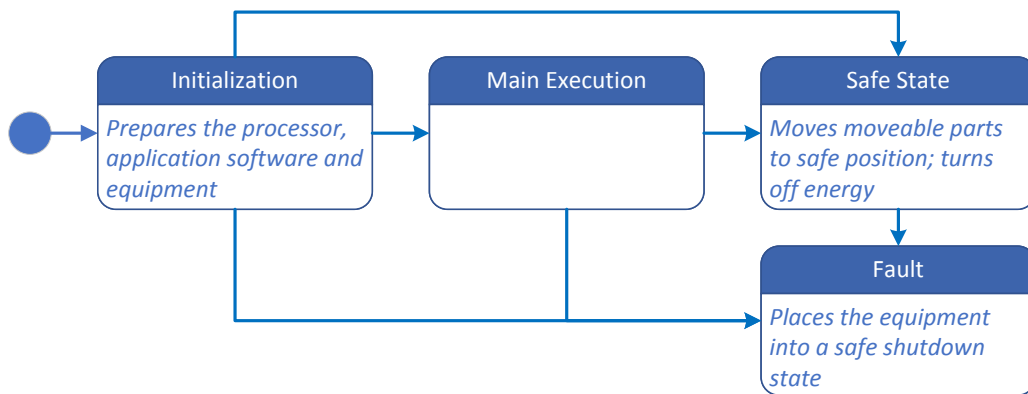


Figure 12: Example state flow diagram

Each state should be documented. Use the same pattern used at the requirements level. See section 12.3 *State* for a templates and description of the elements.

23.1. SUMMARIZE THE ERRORS, THEIR CLASSIFICATION AND RESPONSE

The description of states and behaviors should include a description of the error states of the software and the hardware it controls, and how it behaves in those states. Instead of trying to enumerate each error, uses parallel classifications or categories of errors:

- *Critical errors* being errors (or faults) that cause the controller to enter a *safe state*. *Non-critical errors* do not necessarily enter the safe state.
- *Recoverable* are ones that be cleared, perhaps by user interaction or other event; *auto-recoverable errors* may be cleared automatically when the underlying recondition is removed; and *unrecoverable errors* are not allowed to be cleared. (These are independent of whether or not the error is critical)

Faults are conditions of the MCU or other hardware component, when it is not able to perform its functions to specification.

Next include a table with the following elements

- Error category or classification, or error identifier. (Use the error identifier in cases that are more specific than a categorization, *and* the error has been documented in elsewhere in this high level design document.)
- Whether it is critical or not,
- The response,
- How to recover from the error condition, and
- A description of the error.

Include a description of the safe state.

24. DIVISION INTO ITEMS

How the software is structured and implemented should have a solid discussion. This design approach breaks the development down into manageable chunks. This also supports *integration testing* of the software.

For example, let's say that the software system has the following software items, based on the kind of work they do, or information they organize:

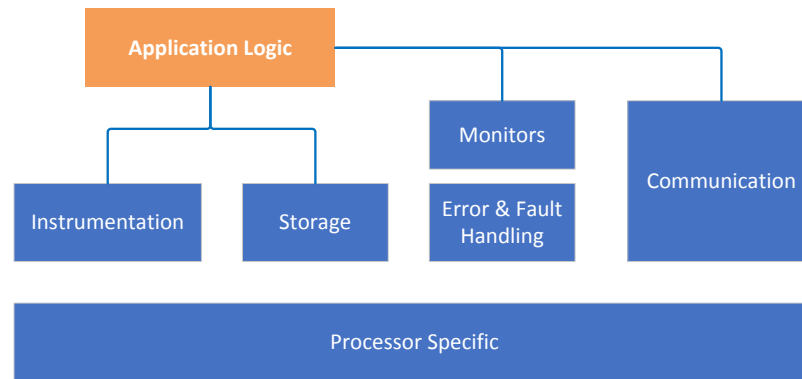


Figure 13: Software Items, major functionality groups

24.1. DESCRIPTION & KEY RESPONSIBILITIES

The software items should be allotted functionality, and their key responsibilities should be listed. These provide a statement of design and may be used to trace to the detail design, and integration tests.

A *key responsibility* is an analogue to a capability requirement, specifying functionality that an item is responsible for. A responsibility is not necessarily traced to any requirement or other input. An item should have at least one, but (at this level) less than 5.

key responsibility

An example description and responsibilities for the items might be:

Application Logic	
Description	The application logic, and how it fulfills its requirements.
Key Responsibilities	
Communication	
Description	Software to support communication with an external host and testing of the application functionality.
Key Responsibilities	[KR.Communication] Exchanging messages (such as status and error information) with a host.
Error and Fault Handling	
Description	Respond to the operating conditions (temperature, voltage, etc) outside of the specified range; and to incorrect function of the processor, software, or equipment
Key Responsibilities	[KR.Fault] Handles MCU and software faults by placing system into safe state.
Instrumentation	

Table 19: Example software items description and key responsibilities

Description	This includes functions for gathering signals and applying the control logic.
Key Responsibilities	[KR.Measurement] Measure voltage, temperature; [KR.Connectors] Provide the status of buttons, connectors, and interlocks.
Monitors	
Description	Monitor the operating conditions (temperature, voltage), and correct function of the processor, software, and equipment; and respond to operating outside of specified conditions.
Key Responsibilities	[KR. Monitoring] Monitor conditions of the voltage, temperature, and fault condition of equipment.
Processor specific	
Description	A grouping of software functionality that provides a common interface (aka abstraction) to microcontroller features and peripherals: ▪ Time Keeping ▪ Analog inputs and outputs ▪ Peripheral communication support ▪ Communication with external items ▪ Discrete IO The hardware abstraction is described in the Microcontroller section.
Key Responsibilities	[KR.Init] Initialization of the microcontroller and control board hardware. [KR.SelfTest] Provide an interface to the processor and its peripherals
Storage Services	
Description	This logs relevant information, and configuration information. Critical data is stored in a manner to prevent data loss if there is a loss of power.
Key Responsibilities	[KR.Store] Store log and configuration information in non-volatile storage.

24.2. PROPERTIES AND BEHAVIORS

The visible properties and behaviors of each item (module) should be documented. These are its *behaviours* inputs, outputs, internal logic, how it interacts with the rest of the system, and responds to events. This would include any states of the items or interaction with it, along with the state flow, and how it reacts to specific events.

24.3. CRITICAL ELEMENTS

Highlight or emphasize elements that are “critical” and need special precaution. Some elements that may be critical include:

- Elements that are necessary to achieve the safety functions of the product
- Elements mandated by the standards (being certified against) as critical

- Elements that are depended upon by those elements or are necessary to prevent systematic faults of any critical element.

The critical elements should get extra review steps and have more documentation. While not all critical elements may be known in the first pass of the design, experienced designers will be able to anticipate many that will be.

Separate the software into different categories:

- The stringently defined area that is focused on addressing the functions critical to the safety requirements in those standards. The risk management, process, techniques and testing are most focused on this category of software
- The other elements of software that whose functionality does not present a safety risk (since the above category is responsible for that function).

This separation allows the other elements to be construction in a less stringent manner. For instance, a high degree of assurance may not be tractable or even meaningfully definable (in the present state of the art).

25. REFERENCES AND RESOURCES

DI-IPSC- 81432A, *Data Item Description: System/Subsystem Design Description (SSDD)*, 1999 Aug 10

http://everyspec.com/DATA-ITEM-DESC-DIDs/DI-IPSC/DI-IPSC-81432A_3766/

DI-IPSC-81435A, *Data Item Description: Software Design Description (SDD)*, 1999 Dec 15

http://everyspec.com/DATA-ITEM-DESC-DIDs/DI-IPSC/DI-IPSC-81435A_3747/

ISO/IEC/IEEE 42010:2011, *Systems and software engineering — Architecture description*.

Note: this supersedes IEEE Std 1471-2000

[This page is intentionally left blank for purposes of double-sided printing]

“Computer architecture, like other architecture, is the art of determining the needs of the user of a structure and then designing to meet those needs as effectively as possible within economic and technological constraints.”

– Fred Brooks, Planning a Computer System: Project Stretch, Chapter 2

CHAPTER 9

Software Architecture Risk Analysis

This chapter provides an initial template for software architecture risk analysis.

26. SOFTWARE ARCHITECTURE RISK ANALYSIS

The outputs of a software architecture risk analysis include:

- A table mapping the software requirements to the architecture element that addresses it. *This table may have been produced by another activity and is only referenced.*
- A list of software risks, acceptability level, and their disposition
- A criticality level for each hazard that can be affected by software
- Recommended changes to the software architecture, software requirements specification, programmable system architecture, etc. For example, actions required of the software to prevent or mitigate the identified risks.
- Recommended Verification & Validation activities, especially tests

The steps of a software architecture risk analysis include:

1. Identify the architecture element that addresses each requirement. *This may have been produced by another activity and is only referenced in the output.*
2. Examine the risks of errors with values
3. Examine the risks of message capacity
4. Examine the risks of timing issues
5. Examine the risks of software functionality
6. Examine the risks of software robustness
7. Examine the risks of software critical sections
8. Examine the risks of unauthorized use
9. Recommendations for rework

26.1. STEP 1: IDENTIFY THE ARCHITECTURE ELEMENTS THAT ADDRESS EACH REQUIREMENT

Go thru each of the software requirements and list the architecture elements that address it.

26.2. STEP 2: EXAMINE ALL VALUES FOR ACCURACY

Identify all the elements of the architecture – sensor 1, sensor 2, actuator, motor, calculations, operator inputs, operator outputs, each parameter received, etc. For each of these, create a copy of table “*Value accuracy risks*” and populate it with respect to the architecture. Identify the least acceptable risk for each applicable condition.

26.3. STEP 3: EXAMINE THE MESSAGES CAPACITY

This step examines the ability of the software to achieve its objectives within the hardware constraints.

Identify all the messaging elements of the system – I²C sensor, task 1, etc. For each of these, create a copy of the table “*Message capacity risks*” and populate it with respect to the architecture. In reviewing each condition, identify the least acceptable risk for each applicable condition. Strike inapplicable conditions.

26.4. STEP 4: EXAMINE THE CONSEQUENCES OF TIMING ISSUES

This step examines the ability of the software to achieve its objectives within the constraints.

Identify all the input elements of the system – button #1, frequency input, I²C sensor, task 1, user input, etc. This list should include elements those receive messages and send messages. For each of these elements, create a copy of **Table 20** (below) and populate it with respect to the architecture. Strike inapplicable conditions and add other identified conditions.

Condition	Hazard, likelihood & severity	Table 20: Timing capacity risks
input signal fails to arrive		
input signal occurs too soon		
input signal occurs too late		
input signal occurs unexpectedly		
input signal occurs at a higher rate than stated maximum		
input signal occurs at a slower rate than stated minimum		
system behavior is not deterministic		
output signal fails to arrive at actuator		
output signal arrives too soon		
output signal arrives too late		
output signal arrives unexpectedly		
processing occurs in an incorrect sequence		
code enters non-terminating loop		
deadlock occurs		
interrupt loses data		
interrupt loses control information		

In reviewing each condition, identify the least acceptable risk for each applicable condition.

26.5. STEP 5: EXAMINE SOFTWARE FUNCTION

This step examines the ability of the software to carry out its functions.

Identify all the functions of the system – the operations which must be carried out by the software. For each of these elements, create a copy of the table below and populate it with respect to the architecture. Strike inapplicable conditions and add other identified conditions.

Condition	Hazard, likelihood & severity	Table 21: Software function risks
Function is not carried out as specified (for each mode of operation)		
Function preconditions or initialization are not performed properly before being performed		
Function executes when trigger conditions are not satisfied		
Trigger conditions are satisfied but function fails to execute		
Function continues to execute after termination conditions are satisfied		
Termination conditions are not satisfied but function terminates		
Function terminates before necessary actions, calculations, events, etc. are completed		
Hardware or software failure is not reported to operator		
Software fails to detect inappropriate operation action		

In reviewing each condition, identify the least acceptable risk for each applicable condition.

26.6. STEP 6: EXAMINE SOFTWARE ROBUSTNESS RISKS

This step examines the ability of the software to function correctly in the presence of invalid inputs, stress conditions, or some violations of assumptions in its specification.

Create a copy of **Table 14** (“*Software robustness risks*”) and populate it with respect to the architecture. In reviewing each condition, identify the least acceptable risk for each applicable condition. Strike inapplicable conditions and add other identified conditions.

26.7. STEP 7: EXAMINE SOFTWARE CRITICAL SECTIONS RISKS

This step examines the ability of the system to perform the functions that reduce risks.

Create a copy of **Table 15** (“*Software critical section risks*”) and populate it with respect to the architecture. In reviewing each condition, identify the least acceptable risk for each applicable condition. Strike inapplicable conditions.

26.8. STEP 8: UNAUTHORIZED USE RISKS

Create a copy of **Table 16** (“*Unauthorized use risks*”) and populate it with respect to the architecture. In reviewing each condition, identify the least acceptable risk for each applicable condition. Strike inapplicable conditions.

26.9. STEP 10: RECOMMENDATIONS FOR REWORK

Summarize each of the identified conditions with unacceptable risk levels (e.g. of “medium” or “high”). These mandate rework, further analysis, and/or Verification & Validation activities.

CHAPTER 10

Detailed Design

This chapter is my tips to make a detailed design that is clear, and eases reviews.

- Detail design is a family of documents
- Diagrams and design decomposition into modules
- Organization of the modules
- Examples of common subsystem designs
- Firmware and subsystem test support

Recall that the *detailed design* is actually an ensemble of documents that work together. This chapter will focus on the main detailed design. Later chapters focus on the others.

27. DIAGRAMS AND DESIGN DECOMPOSITION INTO MODULES

Detailed design begins with the elements of the architecture and expands upon them. This introduces the architecture, perhaps with diagrams, structural and connective elements. The detailed design breaks out the design into major areas and then into modules (with specific function) for that area. This is classic structured decomposition.

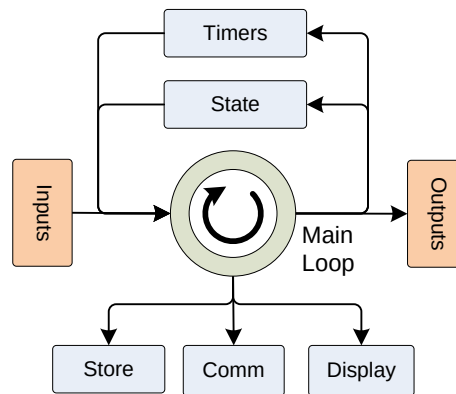
The detailed design description should include several diagrams to explain the design:

- Structural network diagram showing information, and control flow between functional units
- Stratified diagram of modules, showing the level of abstraction and logical dependency

27.1. STRUCTURAL NETWORK DIAGRAM

Include a structural diagram summarizing the software design, with the major structural elements and their interconnections. This may include references to external elements that it controls or depends on. The diagram should include a description introducing how inputs are turned into outputs. It should show the basic structure of the signal flow (both control signals and primary signals). The diagrams also provide context, showing key external elements.

Figure 14: Basic structure diagram of the software



This type of diagram should provide a mental map of the design

- Make the design understandable
- The modules are functionality (or actors) and are represented as boxes
- The links show the connectivity and flow of signal/info/data

Provide a description of the main elements of the structured network diagram:

Element	Description
element 1	Description of the element
...	
element n	Description of the element

Table 22: The structural diagram elements

Example of the structural elements may include:

- Libraries
- Functionality built into libraries
- Code layers
- Threads / processes / tasks

The external elements are:

External Element	Description
element 1	Description of the element
...	
element n	Description of the element

Table 23: The external elements

27.2. DESIGN CRITERIA FOR UNITS

Each unit should perform a distinct task or provide a distinct function:

- It should be easy to define its input-output behaviour; it should perform specific, limited functions
- Each unit should be isolated, manage their state/memory and have low complexity

27.3. THE SIGNAL/DATA

The connective links should be described (and annotated) to provide information about:

- The signal/info/data and the mechanism of representing the signal/info/data (format/encoding/structures/another object)
- The mechanism of the link: is the link shared variables? A message queue? Semaphore(s)?
- The mechanism of transporting the represented data over the link

Example connective elements include:

- Variables
- Buffers
- Queues
- Mailboxes
- Semaphores

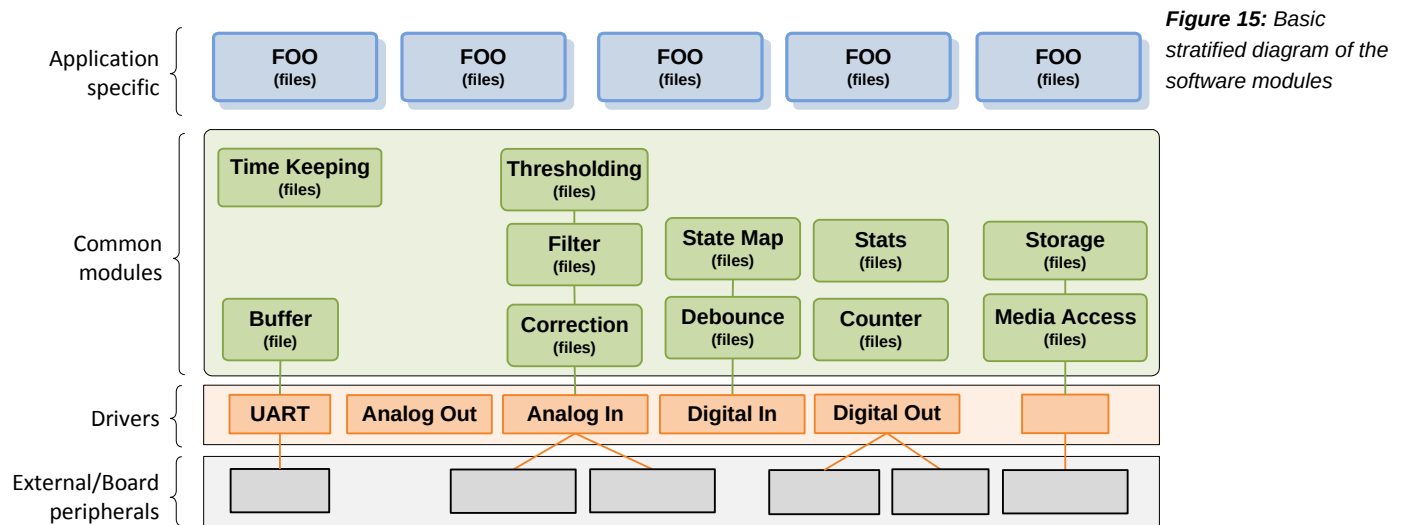
The later when used in a multitasking environment

Family of related modules, usually connected by signal flow

Divide up into sufficient detail. Modules, names of each element, names of actors, other

27.4. STRATIFIED DIAGRAM OF MODULES

The stratified diagram is organized into layers with the lowest closest to the mechanics; successively higher layers present more abstraction. This form of diagram shows much less of the structure, and little of the connectivity. The signal & control flows are up and down; both the inputs and outputs are at the bottom, making the flow is not as clear as with a structural diagram. This form of diagram usually shows the dependency – what module depends on another's function to deliver its own.



Most of the lower layers are specific to the hardware... and thus limit portability of the application to other hardware. However, they may be used in other projects. The upper layers are often specific to the application and/or product... and so they may be less likely to be reused as well.

28. REFERENCE SUBSYSTEM DETAILED DESIGNS

Standard Functionality of Common Modules:

- RTOS threading
- Instrumentation control loops
- Storage IO stacks
- Communication IO stacks
- Storage system type designs
- Motor control type designs
- Platform
- Application
- Board
- Chip specific
- Core specific

Specific fields often settle on a structure/schema which may be reused for a domain-specific class of problems. This simplifies the process and can offer advantages:

- Solid underlying theory
- Well-design test cases or easy to define in/out test cases
- Solid reference model

Use specific computing strategies; design architecture that does these kinds of functions well.

In an embedded system the design must divide the work the work between:

- Interrupts
- DMA and peripherals
- Threads (aka tasks), in the use of an RTOS or other OS.

28.1. RTOS AS AN ORGANIZING TOOL AND TO PROTECT TIME

Is an RTOS used? An RTOS provides a specific kind of structure and breakdown into threads that communicate. Major areas and COTS/SOUP are given their own thread.

RMA prioritization: Scheduling, frequency

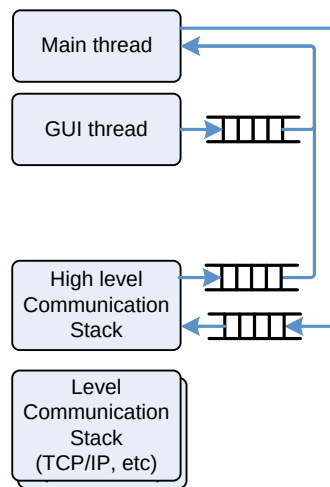


Figure 16: Basic Separation into threads.

The GUI and communication stacks are separated out into their own threads.

- Not safety critical and may be unreliable or inconsistent.
- Must be isolated in time and space from the rest of the system. By placing these into their own thread – prioritized appropriately – the other functions (especially critical functions) can proceed even if the GUI & communication is “slow.”

This is true of many other COTS/SOUP subsystems.

The device drivers may be separated out into one or more threads as well. This is common if there is a communication interface (including a communication interface to storage peripherals)

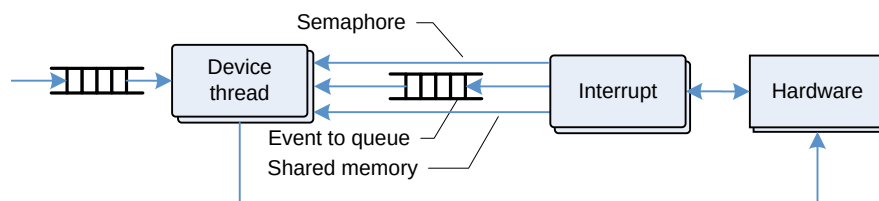


Figure 17: Separation into threads & interrupts to drive hardware

The interrupt service routine communicates with its thread by posting a semaphore (usually) or other similar event object.

A thread has the following basic structure

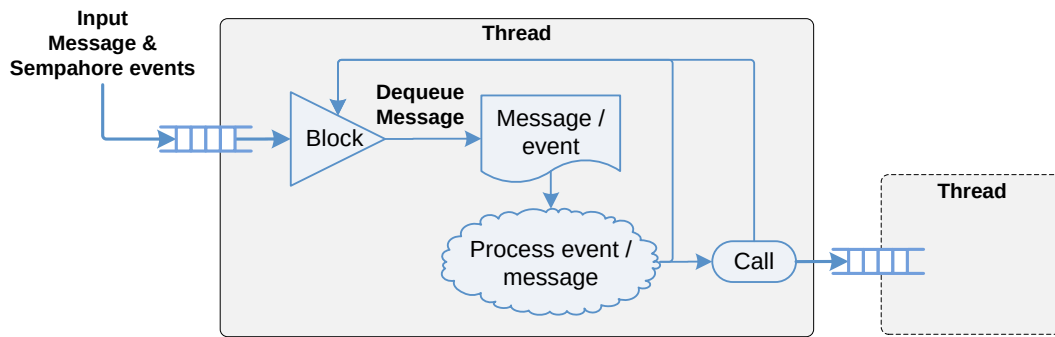


Figure 18: Basic thread structure

Threads typically have an input message queue. The thread blocks on semaphore and message queue events. When it wakes, it dequeues the event, takes action and goes back to sleep. It may post semaphores and messages to other threads, possibly indirectly as a result of framework/library/system calls.

Note: queues – message queues, IO queues, and so on – must be bounded in size.

Has concurrency protections (i.e., mutexes and interrupt enable/disable) for resources.

Signal main loop or threads (tasks) from ISR or other threads

- The number of different threads. (Note: a thread doesn't have a regular clock, as it progresses based on the OS choices of availability.)

28.2. INSTRUMENTATION

This section gives a template for instrumentation subsystems. A typical design looks like:

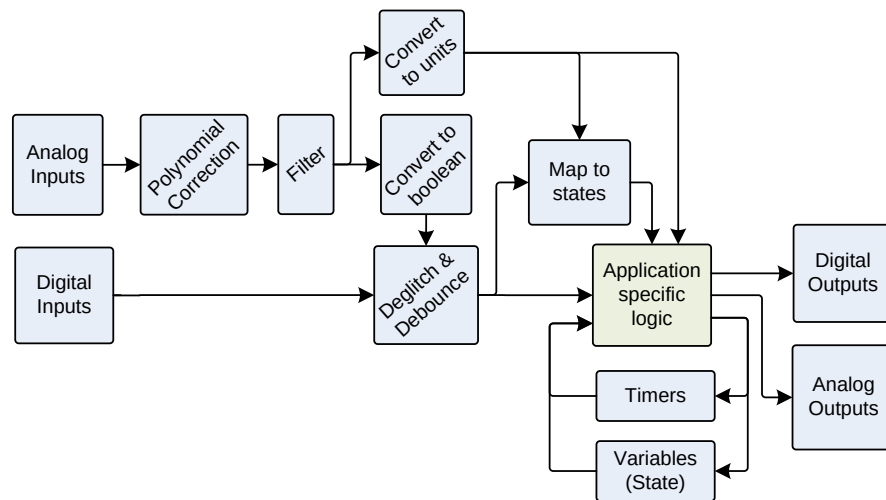
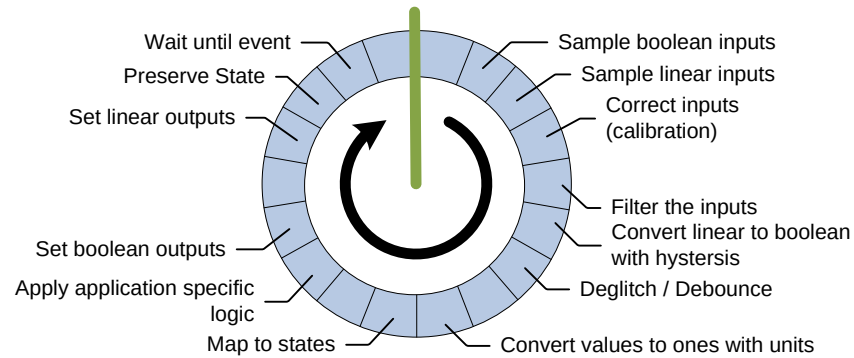


Figure 19: Typical instrumentation structural diagram

Each of the modules is “simple” input / output, so there is rarely any need to have them be in separate tasks/threads, and queues between them. The work of each can be done in a low, bounded amount of time. A task can run a loop like so to drive the inputs and outputs to each of the modules:

Figure 20: Typical instrumentation loop



The time thru the loop is more or less constant / bounded. Takes the same steps each time. The order must be a topological ordering of the directed graph.

28.2.1 Division into modules

Consider the AUTOSAR organization of its HAL to name the microcontroller relate peripheral modules and other instrumentation modules. (For example, Chibios HAL did this).

The main types of modules include:

- Digital input (DIn). This can be one or more modules. One module may gather boolean inputs from the microcontrollers GPIO inputs. Other modules may gather boolean inputs from memory mapped input; I²C, SPI and other remote peripherals (e.g. port expanders), and so on.
- Digital output (DOut). This can be one or more modules. One module may drive boolean outputs from the microcontrollers GPIO outputs. Other modules may gather boolean outputs from memory mapped output; I²C, SPI and other remote peripherals (e.g. port expanders), and so on.
- Analog input
- Analog output
- DMA

Each of these modules would get its own documentation.

28.2.2 DMA for gathering sampled linear inputs and linear outputs

The linear inputs and outputs – often called analog inputs and outputs on a microcontroller – might be performed by a DMA.

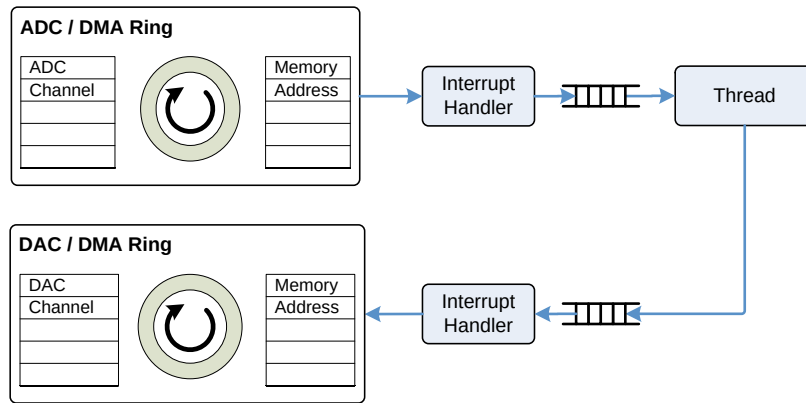


Figure 21: DMA driven linear input and output

For the linear inputs, the ADC samples a sequence of channels, and the DMA places them into a circular memory buffer. For the linear outputs, the DMA takes bytes from a circular memory buffer and applies them to a sequence of DAC output channels.

28.3. COMMUNICATION STACK

This section gives a template for communication subsystems. Such a design might look like:

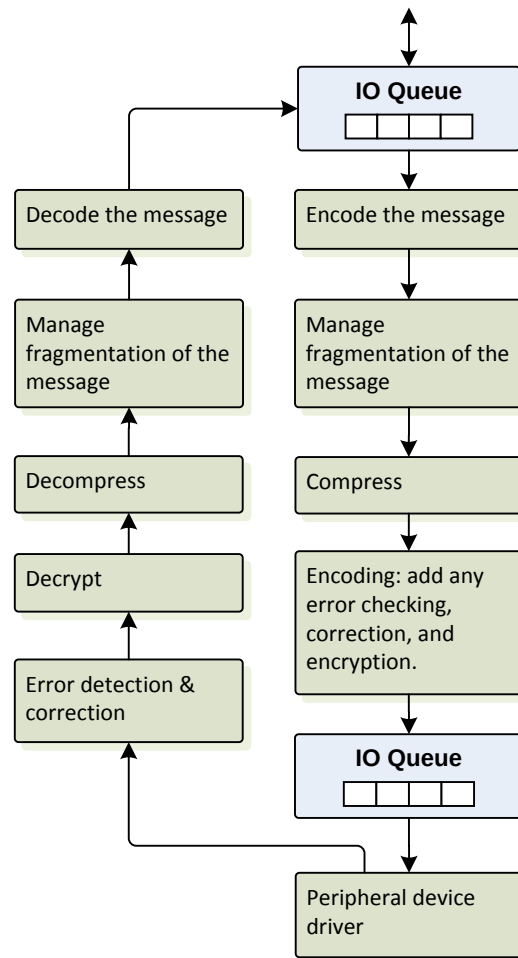


Figure 22: Typical communication stack

28.3.1 DMA for communication

The communication input & output might be performed by a DMA. (The output is well suited as the software is control of how much output is to be sent, and can delegate this to the DMA easily)

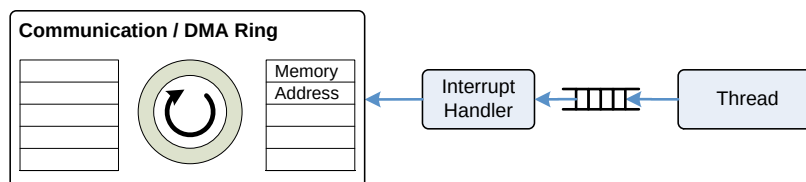


Figure 23: DMA driven communication

28.4. STORAGE

This section gives a template for storage subsystems. Such a design might look like:

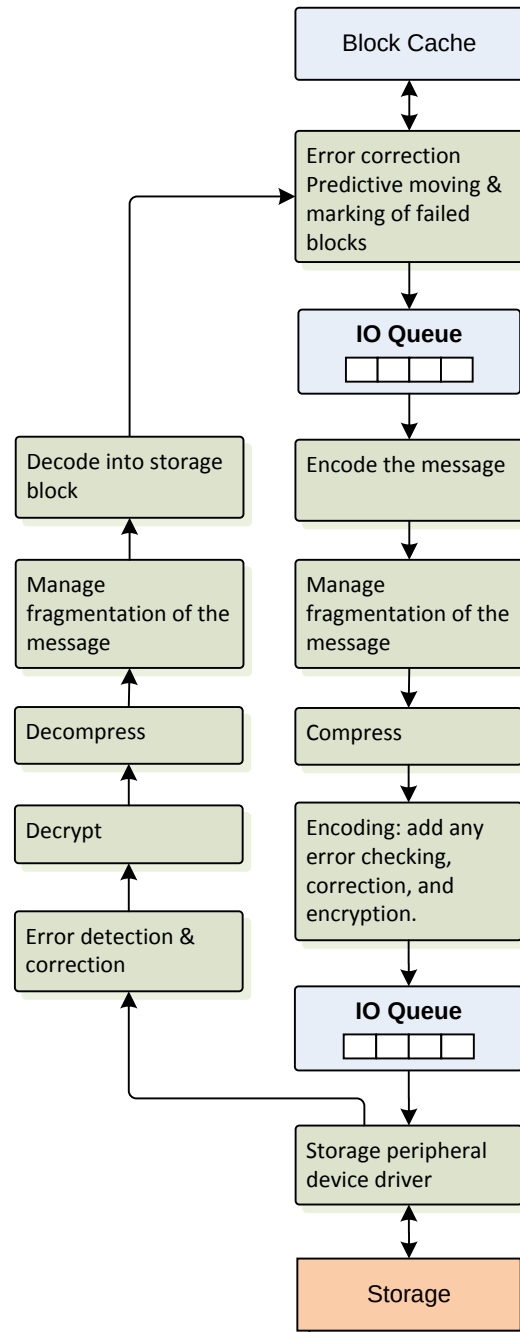


Figure 24: Typical storage stack

28.4.1 DMA for storage communication

The storage input & output might be performed by a DMA. (The output is well suited as the software is control of how much output is to be sent, and can delegate this to the DMA easily)

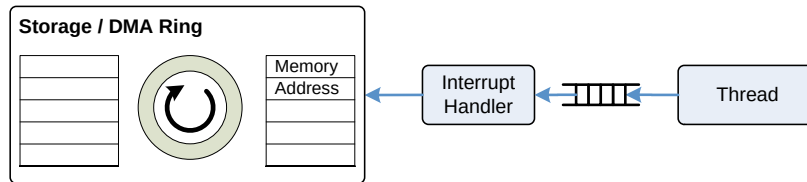


Figure 25: DMA driven storage

Flash memory structures

- Page erase
- New structure, generation count
- Strike out previous generation

28.5. MOTOR CONTROL

This section gives a template for motor driver subsystems. Such a design typically looks like:

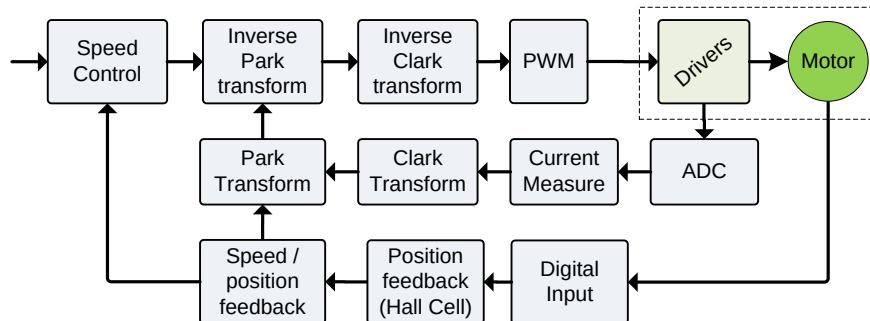


Figure 26: Typical field-oriented control of motor speed

Most are transformations to different representations and acting on that, until it is in a representation that can drive hardware.

29. ERROR AND FAULT DETECTION AND HANDLING

The design document should include a description of errors, how they are detected and the response.

- Check parameters – range and conditions
- Check the error return or flags of called procedures and work structures. Use appropriate timeout values.
- Check intermediate values / conditions, such as from calculations
- Library exceptions
- Use canary methods to find buffer overruns
- Use canary methods to find stack overruns
- Hardware exceptions and faults

Some of these may be described in further within their software item, if that makes more sense; if so, point to those subsections.

29.1. CHECKING PARAMETER VALUES

PROCEDURES check their parameters at the start and reserve any resources they will need to complete the task. If the checks do not pass, or resources cannot be reserved, the procedure should trigger the test harness, raise a software breakpoint, (optionally) log a trace-point and exit early with an error code.

AT THE COMPONENT LEVEL, messages are checked for correctness and sensible values.

The parameter value range and constraints are typically specified at the interface level.

“This application has requested the Runtime to terminate it in an unusual way.”
– An actual Microsoft error message

29.2. LIBRARY ASSERTS, EXCEPTIONS

Many of the firmware libraries perform checking, and signal errors by calling a procedure, like “assert”. By supplying this error procedure, the software can signal an error condition. The error procedure should trigger a software breakpoint (to trigger the debugger) and handle the error, perhaps by putting the machine into a safe state and halting.

30. STORAGE SUBSYSTEM

Description of the storage subsystem, how it is used, errors are detected. Below are several common ones.

30.1. FIRMWARE AND INFORMATION STORAGE

The microcontroller module includes a non-volatile storage (e.g. Flash) to retain the program, and non-volatile information. The software should include features to test it, such as the ability to read program storage, and/or perform a CRC check on it.

The firmware should include a means of setting, clearing, and reading the information storage.

30.2. STORAGE TESTS

The firmware can confirm that the RAM is functional with the conventional “marching” tests. To describe just one test, I will sketch the “walking ones” test below. The steps are:

1. In the storage, area set all bits, save one, to zero. The single bit should be set high.
2. Check the storage area contents match the expected value.
3. Repeat the above for each bit.
4. Repeat the above, but with most bits set high, and the single bit set low.

This test checks that each bit in the storage area can hold clear and set values; that a bit does not clear or set other bits in the storage area. Note this is a test that the storage area works as intended, not that the access is done on a bit level.

The storage area is non-volatile – it retains the intended values after power has been removed from the system. To check this non-volatile property:

1. Setting the values in non-volatile area to known, but non-default values.
2. Remove power. The duration should be for a time longer than it takes internal power caps to deplete.
3. Applying power
4. Checking that the storage area holds the expected values.

See Mikitjuk et al for a description of marching memory tests and what they diagnose

30.3. MEMORY SEGMENTATION

Working memory should be segmented by the criticality classification of its use or owning module. *Canaries* should be placed between segments to detect over run/under run between segments. An example partitioning of memory into segments is shown below:

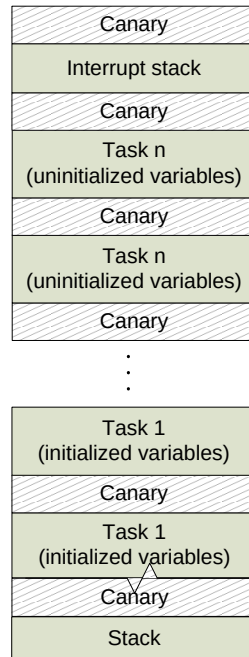


Figure 27:
*Segmentation of
memory with canaries*

30.4. BUFFER OVERFLOW CHECK

BUFFER ADDRESSING CHECKS. Buffers should have *canary* values before and after the buffer area to aid in identifying stack overflow and underflow events. Buffer over and under runs are very common form of software bug, this will help detect such out-of-bounds modification:

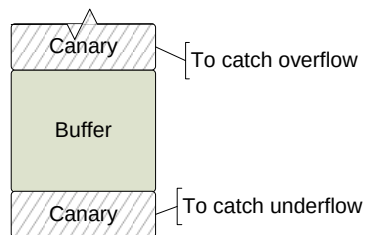


Figure 28: *Overview of
buffers with canaries*

The canary values should be checked frequently, such as during a timer tick, or every run thru a main loop. This is typically done using a struct with variables before and after an array. Use different canary values (0xdeadbeef, 0xc0fecafe, etc) to help id what's going on.

30.5. STACK OVERFLOW CHECK: CANARY METHOD (AKA RED ZONES)

Stacks should be monitored for overflow conditions by checking that the memory surrounding the stack has not been modified.

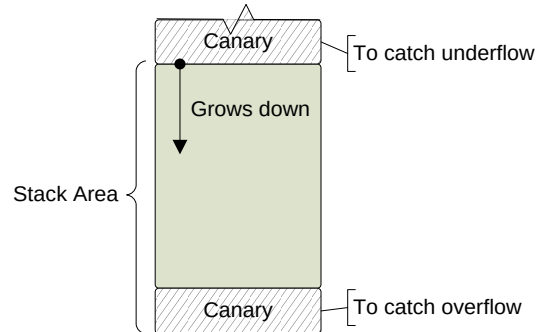


Figure 29: Overview of the stack structure with canaries

Software should place *canary* values on either end of the stack – or stacks, when an RTOS is used. Many RTOS's and compilers or linkers automate this.

FINDING UNDERFLOWS AND OVERFLOWS are a matter of checking each of the buffers to verify that that the canary's still have valid values at the start and end.

THE ADVANTAGE of this approach is that it is easy to implement, easy to understand, and has low overhead costs on the executing firmware.

THE DRAWBACK is that it still possible to miss overflows and underflows: the stack pointer can be incremented by large amounts, completely skipping over the canary area.

31. FIRMWARE UNIT AND SUBSYSTEM TEST

This section discusses design for unit tests of the software modules and networks of modules. This approach allows:

- Testing of individual software elements
- Testing of test conditions that are too tedious, hard, or speculative to replicate
- Test boundary conditions, roots, pools, and other fiducial values with precision
- Test sequences of interactions that are too tedious, hard, or speculative to replicate
- Testing that error returns and error conditions are appropriately handled
- Regressions tests before software release
- Greater examination and validation of the internal software state
- Debugging the above scenarios

31.1. DESIGN TO BE DEBUGGED

Some design tips to improve debuggability:

- Think creatively about what could go wrong and what would be needed to figure out that it has gone wrong. (Too often, designs assume that nothing will go wrong.)
- Provide readable and resettable event counters to track the occurrences of key events.

- Provide read access to view the current state of key input and output signals, such as digital input pins and analog signals.
- Provide read access to variables that shows the current state of each state machine.
- Save a copy of the timer / counter value to a separate variable each time an event occurs. These values could be placed into a trace buffer or read access to them could be provided.
- Function units – often procedures and modules – should check their parameters and internal state. (Do not use assert.) Instead clean up, increment an error count, add trace-point or breakpoint, and return a sensible / appropriate error.
- Measure performance counters for key functions and units. This can help identify inconsistent behaviour.

Read access should be interpreted to include support for a debugger watching the values but can also include providing the values in a trace system, shared memory with a monitor, and via communication stack. The communication stack can be used in implementing the software tests.

31.2. SUBSYSTEM TESTS

The starting point to test the design is the software design description. The software design description should include structural diagrams of the modules, {a stacked (or a layered) of the modules}, a detailed list of the software modules, the signals that flow thru the software, and how to refer them.

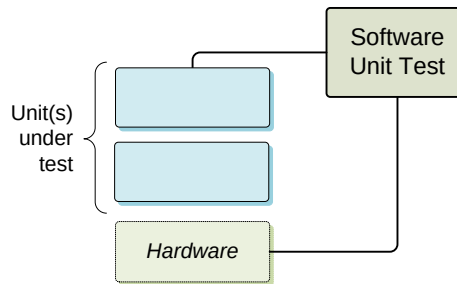


Figure 30: Unit and subsystem test configuration

The test concept is to

1. Identify the path or slice of these modules to test,
2. Develop a test to initialize and control just those software elements. This can be by injecting a stimulus into it using a DAQ. Note the projects electronic design should allow injecting into well-defined points, such as the board test points, or connectors.
3. Inject a stimulus into the slice. This can be sending a signal into the microcontrollers input or invoking the function.
4. Check that the results produced are correct. This can included checking that the results produced at each stage of the along the path are correct, or that the rest of the system operates as expected. The output may be directing the results to a GPIO output, such as a DAC or digital output.

When planning the tests, one would start with tests for a single unit under test. Later tests would expand to more modules & layers, using (where possible) only units that have already been tested. The electrical hardware tests test the lowest layer hardware interfaces.

31.3. SOFTWARE SUPPORT FOR WHITE-BOX TESTING

The design documentation for software modules (and structural group) should include a section that describes how to test the module (or stack of modules). This section should include a description of:

1. How to observe when the module is performing work, when, and for how long
2. How to confirm that the module performs its intended function
3. How to find and test the limits of the unit performing its intended function

Many of the modules will have an accessible test interface. This may let one query its state. However, some procedures within the module will not be easily accessible in this way. The next section describes support for testing them.

31.4. TESTING INTERRUPT HANDLERS, TIMERS, AND PROCEDURES

This section provides an approach to answering the following test questions:

- How does one test that an interrupt (or timer) was raised?
- How long was it from the point that the timer was started to when it was raised?
- How does one test that a procedure was called?
- How frequently?
- How long is the procedure execution duration?

One approach is to employ a GPIO signal to indicate entry and/or exit from a procedure. An interrupt or timer handling procedure would be configured to raise a digital output signal when the procedure is entered and lower the signal when the procedure exits. This signal can then be observed with an oscilloscope or DAQ on the test bench.

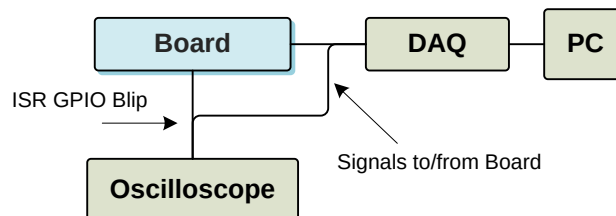


Figure 31: White box test station configuration

This can be extended to many other, non-interrupt related, procedures and key points in the software implementation.

31.5. TESTING THAT THE SOFTWARE IS NOT OVERSUBSCRIBED

To test that the software should not crash or become overburdened, no matter the input, the above techniques can be applied. Sending a high rate of switch signals – or other signals – should show negligible impact on the activity blips of the brake actuator. Blips of interest

- CPU “system tick” timer
- Run loop execution
- Interrupt handler
- Key timed events

The test procedure would configure the item to blip the GPIO signal. The blip should be regular and bounded. A long gap between is a concern and is evidence that the CPU is overloaded, interrupts are disabled too long, or the instruction engine is halted.

It is a concern if the line stays high too long as well: the item under test takes too long to execute, a higher priority interrupt or thread has occurred and impaired its execution time.

32. REFERENCES AND RESOURCES

Blahut, Richard E; *Digital Transmission of Information*, Addison-Wesley, 1990

DI-IPSC- 81432A, *Data Item Description: System/Subsystem Design Description (SSDD)*, 1999 Aug 10

http://everyspec.com/DATA-ITEM-DESC-DIDs/DI-IPSC/ DI-IPSC-81432A_3766/

IEC 61508-7: *Overview of techniques and measures* 2010

Part 7 outlines a good number of resources on how to approach the design process

Labrosse, Jean *MicroC/OS-II: The Real-Time Kernel*, 2nd Ed, CMP Books, 2002

An excellent example of the design a RTOS, with great attention given to detailed design, including algorithms.

32.1. CLEANROOM SOFTWARE ENGINEERING AND BOX STRUCTURED DESIGN

Hevner, A; Harlan Mills; *Box-structured methods for systems development with objects*, IBM Systems Journal, V32 No2, 1993

Harlan Mills write extensively on a process he called *Cleanroom Software Engineering*. His approach to structure decomposition, which he called *Box-structured design*, is a clear description on the process.

CMU/SEI-96-TR-022, Richard Linger, Carmen J. Trammell, *Cleanroom Software Engineering Reference*, Software Engineering Institute, Carnegie Mellon University, 1996 Nov

32.2. INSTRUMENTATION & SIGNAL PROCESSING

Garrett, Patrick H. *Advanced Instrumentation and Computer I/O Design: Real-Time System Computer Interface Engineering*, IEEE Press, 1994

Redmon, Nigel *Biquad Formulas* 2011-1-2 <http://www.earlevel.com/main/2011/01/02/biquad-formulas/>

Smith, Steven W “*The Scientists and Engineer’s Guide to Digital Signal Processing*,” Newnes, 1997, <http://www.dspguide.com/>

SPRU352G, *MS320 DSP Algorithm Standard, Rules and Guidelines*, Texas Instruments, 2007

An excellent example of detailed design documentation, and a good reference on microcontroller facing design of a signal processing framework.

32.3. MOTOR CONTROL

ST Microelectronics, BRSTM32MC “*Motor control with STM32 32-bit ARM-based MCU for 3-phase brushless motor vector drives*” (brochure)

ST Microelectronics, DM00195530 “*STSW-STM32100 STM32 PMSM FOC Software Development Kit Data brief*” #025811 Rev 2, 2014 Mar

Texas Instruments, BPRA073, *Field Orientated Control of 3-Phase AC-Motors*, 1998 Feb

Texas Instruments, SPRA588, Simon, Erwan; *Implementation of a Speed Field Oriented Control of 3-phase PMSM Motor using TMS320F240*, 1999 Sept

CHAPTER 11

Communication

Protocol Template

The documentation of a communication protocol is usually standalone. This documentation should include:

- The kinds of activities that can be done using the communication interface
- The interaction sequences
- An overview of the security design, what it protects and how
- An overview of the communication protocol stack
- The link message formats (data structures)

33. COMMUNICATION PROTOCOL OUTLINE

AN OVERVIEW, which includes:

- Name, designators, or unique identifiers for the protocol
- A synopsis of the functions that it is responsible for
- The roles of the communicating parties
- A description of the transport methods organized in OSI or TCP/IP-like layers.

INTERACTIONS. This section describes the typical interactions that would take place between the communicating parties.

THE PHYSICAL LAYER(S). This section describes the configurations employed with the different types of interconnections.

THE LINK / DATA LINK LAYER(S). This section describes the addresses, the detailed framing – such as the types of frames – and other differences employed with the different types of interconnections. For instance, this includes the signal rate.

THE FRAME FORMAT for each type of link/transport media. This accommodates the unique needs of the different interconnect types, while working toward a common abstraction.

THE MESSAGE FORMAT covers the information sent back and forth and how it is encoded. The messaging is usually in a command and response style. (This often corresponds to the application layer).

34. INTERACTIONS

This section describes the typical interactions that would take place between the communicating parties. This is often flow diagrams.

34.1. READING A BIG BLOB OF DATA

The XYZ data is a binary “file” stored on the slave. The intended algorithm to retrieve the XYZ data is:

1. Read the size of the XYZ data, in number of bytes. For convenience, this will be called “size.”
2. Set the current offset (which will be called “*offset*” here) to zero.
3. Send a read command with the read offset to the new offset value. The slave will send the data corresponding to that area of the XYZ data. This is synopsized in the diagram below

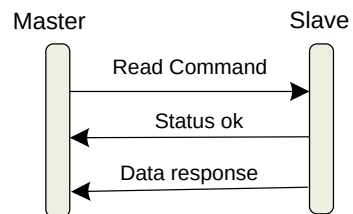
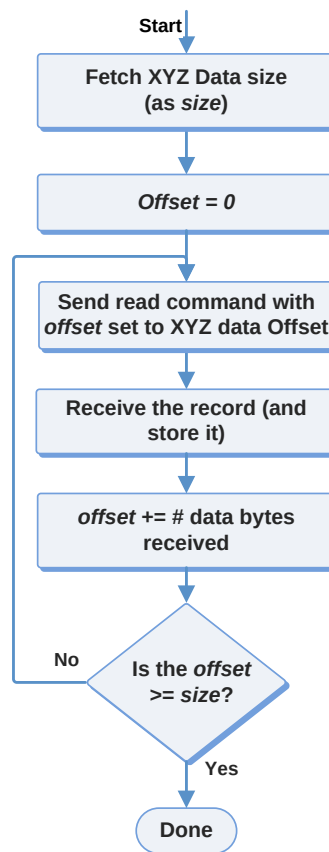


Figure 32: Sequence for reading portion of the XYZ data

4. The packet received will be an offset – this should match the one set – and a count of bytes of XYZ data. Place these bytes onto the end of the local copy of the XYZ data.
5. Increment *offset* by the number of bytes of data received.
6. If the *offset* is less than *size*, continue with step 3.

The figure below captures this process:

Figure 33: The XYZ data retrieval algorithm



35. THE DIFFERENT TRANSPORT MECHANISMS

The protocol is often possible to be conveyed over several different underlying interconnect methods. This section describes the detailed framing and other differences employed with the different types of interconnections.

For example:

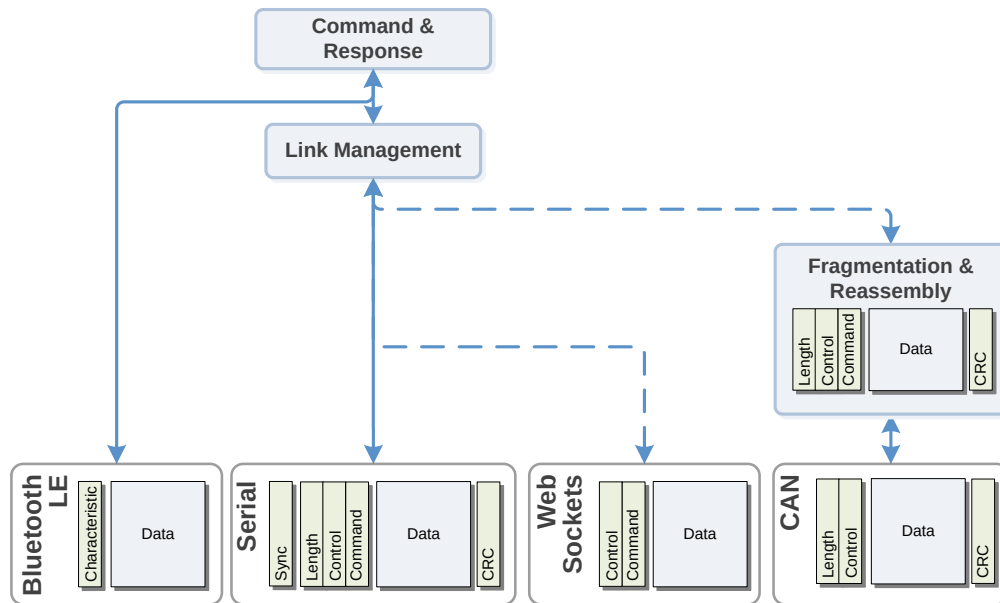


Figure 34: Logical overview of the Communication stack

- Bluetooth LE
- Serial communication, such as RS232, RS485, VCOM (over USB), or RFCOMM (over Bluetooth). This is a common protocol linking different microcontrollers together.
- WebSockets is a message-oriented protocol used on networks, such as available with Wifi, Ethernet, or Cellular data.
- CAN is a message-oriented protocol. This is a common protocol linking different microcontrollers together.

The communication links vary in the features they offer:

- Bluetooth LE handles the delivery, error detection, encryption, authentication, and much of the timeout of exchanging message frames. Bluetooth LE handles errors signaling, and the reference to the object being queried or acted on.
- Serial handles the delivery of the message. The software must provide mechanisms to detect errors, and lost messages. Serial has no encryption, authentication, or other security measures.
- WebSockets handles delivery, error detection, encryption (if a TLS module is employed), and much of the timeout of exchanging message frames. The software must provide its own error signaling and means to reference the object being queried or acted on.
- CAN handles the delivery, and error detection of exchanging message frames. The software must detect damaged or missing messages, as wells as frames received in a non-sequential order.

35.1. THE COMMON LINK FIELDS

The link structures share many of the following fields:

- The *length* field is the number of bytes (octets) that follow the length byte, including the CRC field.
- The *control* field distinguishes between fetching a value, storing such a value, confirming the receipt of one, and so on. {A detailed explanation of these should be included below}
- The *status* field indicates success, any error, an indication, or notification. {A detailed explanation of these should be included below}
- The *command* field is which element to modify and corresponds directly to a Bluetooth LE attribute / characteristic.
- The *data* fields are variable length, and optional.
- The CRC is the check value of message to help detect errors. {Of course, this is the place to describe in detail the parameters of the CRC, what is fed into it, the format of the value and so on.}

35.2. SERIAL FORMAT

RS232 serial interconnections lack the CRC checks, the start of packet, packet length and other information. This information is often added in by protocols using a serial interconnect. This section should describe that.

The commands/queries and responses have the following format

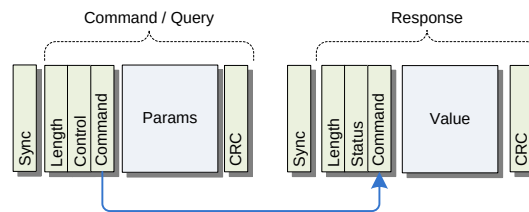


Figure 35: The format of the command/query and response messages

Other things to describe:

- What to do if the message doesn't pass CRC check? Ignore it?
- What about header field doesn't make sense?

This section should include the relevant (custom) configuration of the physical layers. Bit rate, number of bits, parity, etc.

36. TIMING CONFIGURATION AND CONNECTION PARAMETERS

This is the section to discuss the varied connection parameters and timing of recurring events on a per interconnect protocol basis.

37. MESSAGE FORMATS

This section should describe format and interpretation of the messages that go between the parties. What the fields are, how they are encoded, their units/dimension, scale, range, etc. The example here is for a command-response mode but can be adapted to other modes of communication.

It is easiest to specify the types as format, size, and sign types that match the C/C++ Coding Style (see Chapter 17), with a little-endian encoding (or big endian if you prefer). The command and responses then provide

37.1. READ DATA

This command is used to retrieve a segment of data.

Command Code	{the hex value for the command goes here}
Characteristic UUID	{the hex for a Bluetooth LE characteristic UUID goes here}
Modes	Read, Notify, Indicate {this is more useful for Bluetooth LE}
Response Code	{the hex value of the response message}
Signature	offset× nBytes → MemStore → offset × bytes ^{nBytes}
Command Size	4
Response Size	4-252
Equivalent Procedure	Foo_data_encode()

Table 24: Summary of the Read Data command

Note: not all of these fields are necessary, these are merely an example to show possibilities.

37.1.1 Command

The parameters of the command body are:

Offset	Size	Type	Parameter	Description
0	2	uint16_t	offset	The offset to retrieve the data from
2	2	uint16_t	size	The number of bytes to retrieve

Table 25: Parameters for Read Command

37.1.2 Response result

The parameters for the Read response message:

Offset	Size	Type	Parameter	Description
0	4	uint16_t	offset	The offset of data
4	varies	uint8_t[]	data	The retrieved data

Table 26: Parameters for Read Response

The intended use is to read a segment of the data buffer. The typical read sequence is below:

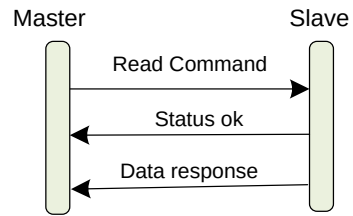


Figure 36: Read command sequence on success

The sequence for an invalid read command is show below:

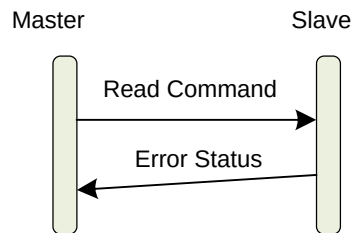


Figure 37: Read command with error response

38. A NOTE

To aid test management give identifiers to each of the commands, responses, and their fields.

39. REFERENCES AND RESOURCES

DI-IPSC-81436A, *Data Item Description: Interface Design Description (SDD)*, 1999 Dec 15
http://everyspec.com/DATA-ITEM-DESC-DIDs/DI-IPSC/DI-IPSC-81436A_3748/

[This page is intentionally left blank for purposes of double-sided printing]

“By the way, nobody likes your Doxygen docs. Okay? Nobody even uses your Doxygen docs. They are horrible. I do not care how much time you spend. Doxygen is an entire system devoted to convincing people they have written documentation..”

– Elicia & Chris White

CHAPTER 12

Programmer Documentation

This chapter discusses the programmer documentation generated by *doxygen* or *docfx*.

- Detail design is a family of documents
- Organization

40. DETAILED DESIGN DOCUMENTS

The programmer documentation is often made with tools such as *doxygen* or *docfx* to format source code comments, augmented with structuring markdown files. This type of document is a supplement to the main detailed design.

This type of generated document is often large and unread – it rarely can be read like a well-edited text. But it is possible to make it readable.

Projects can benefit from using this (partly) automated output. The trick is that it should also:

1. Make an “online” version that is easily (and usefully) searchable.
2. Generate a PDF for review and design history file.

41. ORGANIZATION

Introduce the Organization. This section outlines the organization of the modules and implementation:

- Module Prefixes
- Source configuration files
- File system layout
- File grouping for a module’s implementation
- Configuration points

The modules. Order the namespaces, classes, units/modules, structure, procedures etc. in the way that makes the most sense for reading the documentation.

Appendices. the following are typically in the appendices:

The following are typically in the appendices:

- Compiler Configuration, flags, etc.

- Analysis tool (e.g. Lint, MISRA C checks, etc.) configuration including which checks are enabled and disabled.
- Linker configuration & Linker scripts
- The software configuration settings.
- Files employed in the software.

41.1. MODULE PREFIXES [C]

{It is arguable whether this information should in the architecture, or in the appendices. I find it helpful as guidance to the development}

Each C module has a separate prefix (C++ modules would use the namespace and class name features). The table below describes the prefixes employed:

Prefix	Module
<i>AIN</i>	The analog input module, including ADC sampled values, etc.
<i>AOut</i>	The analog output procedures
<i>App</i>	Application procedures and application specific logic
<i>BSP</i>	Board specific package related procedures
<i>DIn</i>	The digital inputs are GPIO logic signals.
<i>IIR</i>	Infinite impulse response filters
<i>Poly</i>	Polynomial correction of signals
<i>Time</i>	Time-keeping related
<i>Tmr</i>	Timer related
<i>UART</i>	UART, a hardware serial interface

Table 27: Summary of C module prefixes

41.2. SOURCE CODE CONFIGURATION FILE(S)

The firmware is configurable, allowing changes in the electronics design and specific features of the application. The settings for the other three configuration files are described in appendix TBD.

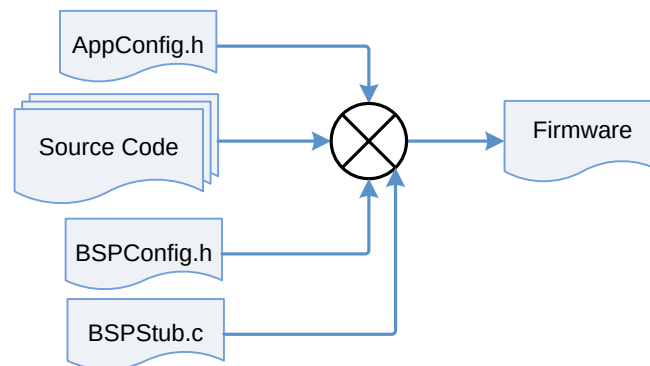


Figure 38: The configuration of the production firmware

The BSPStub.c provides the linkages the microcontroller register, such as the digital input and output data registers. (These differ between microcontroller families, and sometimes within

them; but every microcontroller has some form of these registers). This also provides resource sizing and buffers related to these inputs.

THE CONFIGURATION HEADER FILES are used to enable (or disable) features and size the remaining resources. The features that can be enabled have a control macro suffixed with `_EN`. For example, to enable feature XYZ:

```
#define XYZ_EN (1)
```

Alternatively, to disable it:

```
#define XYZ_EN (0)
```

The board specific defines are in a file called `BSPConfig.h`. The application or framework features are in a file called `AppConfig.h`.

41.3. CONFIGURATION POINTS

Build Configuration, in `.h` and `.c` files

- Application specific build configuration
- Board specific build configuration
- Chip specific build configuration
- May use configuration and structures separating these allow the source to be built as a library shared across many targets; with the configuration structures provided per target board.

41.4. FILE SYSTEM LAYOUT

Note: The software development plan typically provides the information in this section.

The directory structure is a set of nested folders. Some folders are shared between projects; some are unique to this project. The top-level folders are:

Folder	Description
doc	The documentation for this project
Components	The vendor documentation for the components used in this project or evaluated for it
src	The project source code
Release	The released application image, suitable for download to the unit

Table 28: Top-level Folders in the project file directory

Within the project source code, the folders might be:

Folder	Description
CSP	Holds core specific modules
IO	Holds support for non-communication input put, such as GPIO, PWM, etc.
Power	Holds support for measuring the battery level
Sensors	Support for sensors such as accelerometers, and temperature sensors
STM32	Holds STM32 microcontroller specifics
win	Windows specific files

Table 29: Source code folders in the project file directory

41.5. FILE GROUPING FOR A MODULE'S IMPLEMENTATION

A module may be implemented by one or more source files.

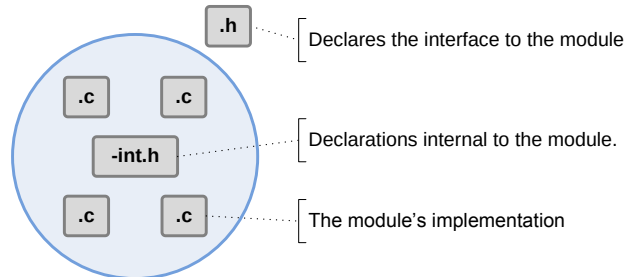


Figure 39: How `.h` and `.c` files related to a module

- One or more `.c` files that implement the module – it is better to break down a module into groups of short files rather than one large file a thousand lines or longer.
- The module may have other `.h` files (suffixed as `-int.h`) that are for use only within the module. These should not contain information intended to be used elsewhere.
- A module has one (or more) `.h` files that declare the procedures, variables, and macros that other modules may use. This file should not have 'internal' only information; that is, it should not include information that other modules *should not use*.

42. CHALLENGES

Doxygen and *Markdown*, while popular for their simplicity, has several drawbacks that frustrate projects:

- *Inconsistent & Poor Formatting.* The generated documentation can sometimes be inconsistent or poorly formatted; different platforms and tools implement the *Markdown* slightly differently, leading to inconsistencies. These create confusion and frustration, requiring many iterations to resolve.
- *Loses Information.* *Doxygen* tools frequently lose or drop key description information. It isn't clear that this is the tools fault. This too wastes time, creating confusion and frustration.
- *Not an Authoring Environment.* *Doxygen* lacks a previewer, while *Markdown* has previewers, each implements the *Markdown* slightly differently, leading to inconsistencies in how documents are rendered.
- *Excessive Complexity.* The *MkDocs* configuration, *css* files, *MkDoxy* code and *jinja* templates are complex and require a lot of fiddling to get it to work properly.
- *Hard to Structure.* *Doxygen* and *Markdown* are bottom up. It is hard and laborious to organize the documentation fragments into any cohesive structure.
- *Poor Maintainability.* The process of maintaining *Doxygen* formatting and such adds large, unnecessary overhead to the development process.

The resulting text is often neither cohesive nor cogent. It is a less-than-ideal choice for documentation, especially for complex projects that require robust formatting and features.

CHAPTER 13

Software Module Documentation Template

The documentation of the software modules goes into the “programmer documentation” documents, rather than the main detail design document. This is most often achieved thru skillful use of markdown and doxygen. This is my template for the resulting module documentation.

- Detailed design outline
- The overview of the module
- The software interface documentation
- The detailed design (internals)
- The configuration interface

43. DETAILED DESIGN OUTLINE

I use the following template for the documentation of each software module:

AN OVERVIEW, which includes:

- Name of module
- A synopsis of the functions that it is responsible for
- Diagram and description of the module's main organization. This isn't intended to be the design diagram; it is intended to show where it fits into the bigger design.

SOFTWARE INTERFACE DOCUMENTATION. This section describes how software would communicate with the module system using procedure calls.

DETAILED DESIGN. This section describes the detailed internal structures and procedures used within the module.

THE TESTING section describes how to test the module.

44. THE OVERVIEW SECTION

The overview section introduces the module, and its role. *Optionally* include a diagram that shows the where the module fits in overall, and how the other modules interface to it:

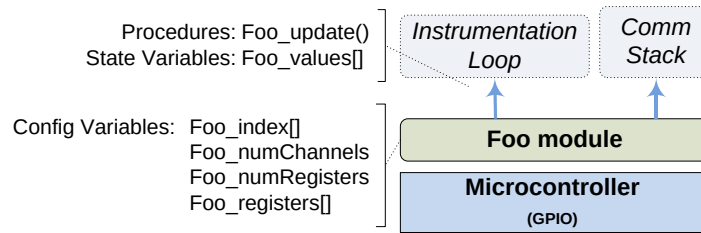


Figure 40: Overview of the Foo module

The diagram is usually vertically organized. The upper layer communicates with the rest of the system; the lower layers work with the hardware or more specific work tasks. Between the nodes for the different modules (and hardware elements) are callouts synopsizing the procedures, variables, and IPC structures that act as the links between the nodes. The typical major interfaces include:

- Interface that the system can use to configure the module.
- Interfaces that the rest of the stack or software system may interact with the module
- Interfaces from the module to the underlying layers, or the lower layers of the stack that it interacts with.

45. THE SOFTWARE INTERFACE DOCUMENTATION

The software interface section describes how software would communicate with the module system using procedure calls. This includes a description of the procedures, structures, the respective parameters of these, calling sequences, responses, timing, and error handling.

A good software interface is...

- Easy to learn / memorize
- Leads to readable code
- Hard to misuse
- Easy to extend
- Sufficient or complete for the tasks at hand

The overview should describe:

- **INITIALIZATION**, which is passed information about how the microcontroller is connected to the board, and which of the internal resources that should be used.
- **DATA ACCESS**. The procedures that get data from the module or provide data to the module
- **CONFIGURATION**. How the module is configured to use lower-level resources, and the parameters (such as data rate) in how it should use the resources. (Included where appropriate.)

After the overview there should be:

- Description of operations
- Diagram of interaction, algorithm
- The `#defines` and enumerations used in the software interface
- The data structures employed by the software interface

- The variables provided by the software interface
- The procedures (and their parameters) provided the software interface

45.1. CALLING SEQUENCES FOR THE INTERFACE

All interfaces should provide a BNF-style description of the acceptable calling sequences, or phrases, for the API. For example:

```
::= open [optional_calls] (read | write | lseek)* close;
```

or

```
::= open mmap (MemoryOp | mincore | lseek | read | write) munmap close
| open shmat (MemoryOp | mincore | lseek | read | write) shmdt close ;
```

The conventions for such BNF-like statements include:

- Parameters aren't specified in the rules
- Only specify calls related, usually in a context. That is, specify only the API related to an 'instance' (object, file channel, etc.) from its creation and manipulation through its destruction.
- Items in italic refer to other rules
- Items in parenthesis form a regex-like set of alternatives
- Items in braces are optional, the equivalent of a null option in an alternative grouping
- A sequence of calls is only valid if it is accepted by the rules outlined. Under the rules of software validation, the software is erroneous if it is possible that the software executes a calling sequence not recognized by the BNF.
- Keep the number of rules small, but reflect the real constraints on the calling sequence

45.2. DEFINES

This section describes the #defines used in the software interface.

```
#define CMD_READ (0xA000u)
```

The read command value.

```
#define CMD_WRITE (0x2000u)
```

The write command value.

45.3. ENUMERATION TYPE DOCUMENTATION

This section describes the enumerations used in the software interface.

enum ABC

This enumeration is such and such, used for so and so.

45.4. DATA STRUCTURE DOCUMENTATION

This section describes the data structures used in the software interface. The table below synthesizes the data structures:

Structure	Description
<i>Foo_t</i>	This structure is used to track info

Table 30: *Foo Structures*

Foo_t struct Reference

This structure tracks the hours of operation.

Field	Type	Description
<i>secondsElapsed</i>	uint32_t	The number seconds since the start of operation
<i>prevSeconds</i>	uint32_t	The number of seconds of operation that were logged
<i>startTime</i>	uint32_t	The time that the operation was started.

Table 31: *Foo_t structure*

45.5. VARIABLES

This section describes the variables in the software interface. The table below describes the variables provided by the module:

Variable	Description
<i>Foo_errorCount</i>	The number of errors encountered
<i>Foo_successCount</i>	The number of successes encountered

Table 32: *Foo variables*

45.6. CLASSES

The table below describes the classes employed in the module:

Class	Parent	Description
<i>Foo</i>		An abstract base class to do some interesting things.

Table 33: Module classes

Foo class Reference

This is an abstract class intended to do some interesting things.

Field	Type	Description
<i>someField</i>	ItsType	Describe the field
<i>someOtherField</i>		

Table 34: Foo class structure

Method	Description
<i>isOutOfDate ()</i>	Checks to see if the foo bar is out of date.
<i>unload()</i>	Unloads the foobar from memory.

Table 35: Foo methods

45.7. PROCEDURES: SYNOPSIS

This section introduces the procedures used in the interface. The table below describes the modules procedure interface:

Procedure	Description
<i>Foo_update()</i>	Called to update the state of the module each time step.
<i>Foo_write()</i>	Write a data block to the device

Table 36: *Foo*
interface procedures

45.8. PROCEDURE DOCUMENTATION

This section describes procedures that the module exports.

void Foo_update(void)

This updates the internal state of the module with each time step, and prepare output results.

Parameters:

none

Returns:

none

This should describe the behaviour of the procedure, its algorithm, or other steps that it may take.

Err_t Foo::write (void* *address*, uint8_t* *buffer*, uint16_t *length*)

Write a data block to the device

Parameters:

address The address within the device to store at
buffer The buffer holding the data to write; this must hold *length* bytes
length The number of bytes to write

Returns:

Err_NoError The data was successfully written
Err_Address The address is not a valid memory page
Err_Timeout The operation did not complete timed out
other Other access error

This should describe the behaviour of the procedure, its algorithm, or other steps that it may take.

46. THE DETAILED DESIGN SECTION

The detailed design section describes the detailed internal structures and procedures used within the module. This includes a description of the procedures, structures, the respective parameters of these, calling sequences, responses, timing, and error handling:

- Diagram(s) breaking down the module
- Description of operation, such as the main functions of the module, any threads and/or interrupt service routines
- Diagram of interaction, algorithm
- Detailed design info
- The #defines and enumerations used within the module
- The data structures employed by the module
- The variables internally employed in the module
- The procedures (and their parameters) within by the module
- The files employed in the module

Most of these sections follow the same format as used in the software interface.

{Optional} The diagram below synthesizes the organization of the Foo module:

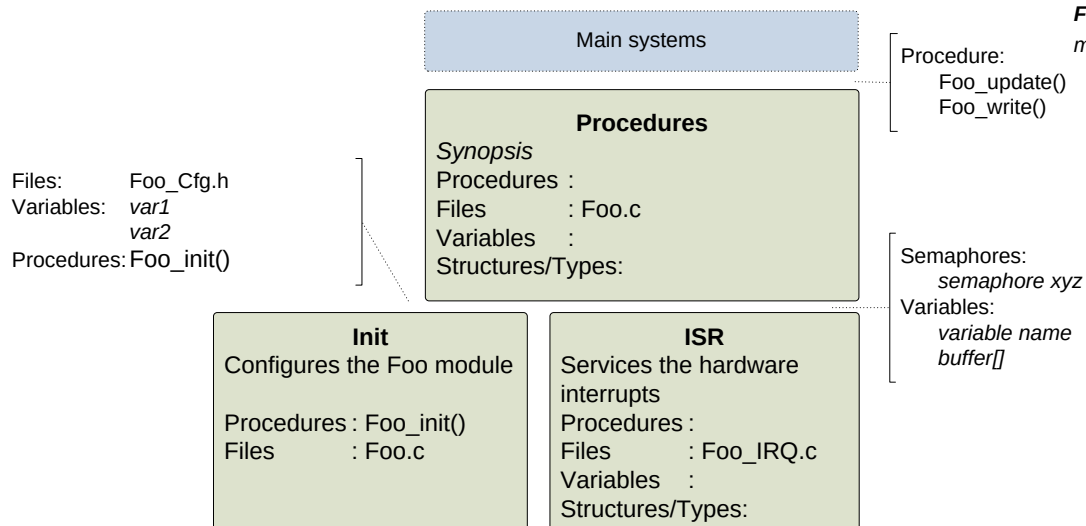


Figure 41: Detailed module organization

46.1. INTERRUPT SERVICE ROUTINES

This section should introduce and describe the interrupt service routines. This should define why they are called, what action they take, and how they interact with the rest of the system.

46.2. DEFINES

This section describes the #defines used internally.

same format as in the interface section

46.3. ENUMERATION TYPE DOCUMENTATION

This section describes the enumerations used internally.
same format as in the interface section

46.4. DATA STRUCTURE DOCUMENTATION

This section describes the data structures used internally.
same format as in the interface section

46.5. CLASSES

This section describes the classes used internally.
same format as in the interface section

46.6. VARIABLES

This section describes the variables used internally.
same format as in the interface section

46.7. PROCEDURES: SYNOPSIS

This section introduces the procedures used internally.
same format as in the interface section

46.8. PROCEDURE DOCUMENTATION

This section describes the procedures used internally.
same format as in the interface section

46.9. FILES EMPLOYED IN THE MODULE

The table below describes the files employed in the module:

File	Description
Foo.c	The foo modules API procedures
Foo.c	Public interface to the Foo module
Foo_cfg.h	Public interface to the configuration of the Foo module
Foo_IRQ.h	Header file describing the local interface to the Foo Interrupt service routine
Foo_IRQ.c	The interrupt service routines.

Table 37: Module files

47. CONFIGURATION INTERFACE

This section describes the configuration of the module.

The configuration is usually defined statically, at build time. The main application defines const variables with the values to configure the module. This allows a module to be reused in many applications, without specifying the exact size of resources used or coupling to the hardware

47.1. VARIABLES

The table below describes the BSP configuration variables provided to the module to configure it:

Variable	Description
<i>Foo_numChannels</i>	The number of channels used by the module.
<i>Foo_numRegisters</i>	The number of peripheral registers defined.
<i>Foo_register[]</i>	The set of peripheral registers

Table 38:
*Configuration of the
Foo module*

48. THE TEST SECTION

The test section describes how to test the module. It should include a description of

1. How to observe when the module is performing work, when, and for how long
2. How to confirm that the module performs its intended function
3. How to find and test the limits of the unit performing its intended function

Planning the test:

- Start with tests for a single unit under test and expand to more layers.
- Different mechanisms of tests

The rest of the test section should focus on three different mechanisms for performing the tests:

- The software-based tests are intended to catch coding and calculation bugs. These checks typically cannot catch hardware interaction bugs, but they can do *regression checks* on software and (some) hardware configuration bugs.
- Desk checks look at the actual system execution, probed by hand
- Bench checks are more automated checks, with software and hardware probes

Note: the test documentation is often placed in other documents. I find it beneficial to include an outline of tests. It helps ensure that the design is focused on the testability of the module (and module stack).

49. REFERENCES AND RESOURCES

DI-IPSC-81435A, *Data Item Description: Software Design Description (SDD)*, 1999 Dec 15
http://everyspec.com/DATA-ITEM-DESC-DIDs/DI-IPSC/DI-IPSC-81435A_3747/

SPRU360E, *TMS320 DSP Algorithm Standard, API Reference*, Texas Instruments, 2007 Feb

An excellent example of describing how the modules, algorithms, and interfaces work and are intended to be used.

“There’s no substitute for documentation written, organized, and edited by hand.

Auto-generated documentation is .. worse than useless: it lets maintainers fool themselves into thinking they have documentation, thus putting off actually writing good reference by hand. .. most of the time it's easier just to read the source than to navigate the bullshit that these autodoc tools produce. About the only thing auto-generated documentation is good for is filling printed pages when contracts dictate delivery of a certain number of pages of documentation.”

– Jacob Kaplan-Moss

CHAPTER 14

Design Review Checklists

This chapter provides checklists for use in reviewing the software designs (before implementation proceeds too far):

- Design review checklist
- Detail design review checklist

See also

- Appendix G for the *Code Complete* Design Review check lists
- Appendix H for a rubric to apply in the reviews

50. DESIGN REVIEW

A software design review is intended to answer a basic set of questions:

1. How does the design address the specifications?
2. What are the major elements that make up the system?
3. How do these elements work together to achieve the goals?

A good design is:

- Simple
- Cohesive – Each unit has a single, clear responsibility with-out mix multiple unrelated functionalities; it is a clear how they work together to serve a well-defined purpose.
- Feasible – the design can be implemented in a timely fashion
- Adaptable to other applications
- Dependable: no bugs, or unexplainable behaviour and can achieve long-lasting operation
- Efficient: applies its key resources to useful work (skillfully)

50.1. STARTING

- ☐ Are the requirements sufficiently defined to create the high-level design (architecture)?
- ☐ Is the high-level design understandable?
- ☐ Are there terms or concepts introduced / defined before they are used?

- ☐ Is the document free of slogans, hype and idiomatic terms? Remove: “KISS”, “Do Not Repeat Yourself”, “Yak Shaving”, “inversion of control”
- ☐ Are the requirements realizable?

50.2. MODULES AND FLOWS

- ☐ Are the main areas of functionality explained?
- ☐ Are the main inputs and outputs described?
- ☐ Are the main modules or components (“software items”) and their roles described?
- ☐ Does the architecture explicitly identify the key responsibilities of each software item? Are they clear? Are they located in one place?
- ☐ Is a structural diagram showing the flows given?
- ☐ Are the main signal chains and logic flows shown and described?
- ☐ Are the roles of the signals and logic explained?
- ☐ Is the application logic discussed and outlined?
- ☐ Does it describe the approach to testing and diagnostics?
- ☐ Is the approach to power management outlined?
- ☐ Is data management outlined? Is the roughly what will be stored, whether it will be non-volatile discussed?
- ☐ Is the communication outlined?
- ☐ Is the safety model discussed? Timeouts? Watchdog timers?
- ☐ Is the approach to software configuration (of features, parameters, etc) sufficiently discussed?
- ☐ Is the approach to other IO described?
- ☐ Are the module namespaces or prefixes provided and described?
- ☐ Are the main file groupings provided and described?

50.3. NAMES

- ☐ Are the module names well chosen?
- ☐ Are the signals, and other object names well chosen? Are the names clear? Do the names convey their intent? Are they relevant to their functionality?
- ☐ Is the name format consistent?
- ☐ Names only employ alphanumeric and underscore characters?
- ☐ Are there typos in the names?

51. DETAILED DESIGN REVIEW CHECKLISTS

51.1. BASIC FUNCTIONALITY

- ☐ Does the architecture sufficiently describe the software items so that a detailed design can be developed?
- ☐ Does the detailed design match the overall design and requirements?
- ☐ What is the approach used to validate that the detailed design fulfills the architecture?
- ☐ Are the requirements sufficiently defined to create the detailed design?

- ☐ Is the architecture (or other high-level design) sufficient and agreed upon to support the detailed design?
- ☐ Is all the detailed design easily understood? Is it simple, obvious, and easy to review?
- ☐ Is the detailed design sufficiently detailed to create/update a work breakdown structure?
- ☐ ...to create a schedule, down to half-day increments?
- ☐ Is the design sufficiently detailed to delegate work?

51.2. DOCUMENTATION

- ☐ Are all modules and interconnecting mechanisms documented?
- ☐ Do they properly describe the intent of the module?
- ☐ Is the module interface (procedures, data structure, sequences) documented?
- ☐ Are all parameters of the procedures documented?
- ☐ Is the use and function of third-party code/libraries documented?
- ☐ Are data structures and units of measurement explained?
- ☐ Have the submodules (e.g. subitems, units) been sufficiently identified?

51.3. DIAGRAMS

- ☐ Are block diagrams employed?
- ☐ Are the boxes labeled with their designator?
- ☐ Are the boxes connected?
- ☐ Do the diagrams show the flow of signals and external control?
- ☐ Code complexity measure is low (below set threshold)?
- ☐ Is there sufficient annotation on the connection to understand how they communicate? Is this covered in the expository text?
- ☐ Are there sequence diagrams?
- ☐ Are there flow charts?
- ☐ Does the diagram text match the terms used in the exposition?

51.4. MAINTAINABILITY AND UNDERSTANDABILITY

- ☐ Is the design unnecessarily ornate or complex?
- ☐ Is the design appropriately modular? Would it be better with more modules? Fewer?
- ☐ Can any of the modules be replaced with library or built-in functions?
- ☐ Does the design have too many dependencies?
- ☐ Any changes to improve readability, simplify structure, and utilize cleaner models?

51.5. NAMES & STYLE

- ☐ Are the module names well chosen? Are they relevant to their functionality?
- ☐ Are the signals, variables, and other object names well chosen? Are the names clear? Do the names convey their intent? Are they relevant to their functionality?
- ☐ Do the names of these objects use a good group / naming convention? e.g. related items should be grouped by name

- ☐ Is the name format consistent?
- ☐ Do the names only employ alphanumeric and underscore characters?
- ☐ Are there typos in the names?

51.6. PRIORITIZATION REVIEW CHECKLIST

- ☐ Are all threads identified? These should be in a table summarizing them.
- ☐ Are the resource protecting mutexes identified? These should be summarized in a table.
- ☐ Are all of the interrupts and their sources identified?
- ☐ Has a Rate Monotonic Analysis (RMA) and dead-line analysis been performed?
- ☐ Have the task/threads and mutexes been assigned priorities, based on this analysis?
- ☐ Have the interrupts been prioritized based on a similar analysis?
- ☐ Have the DMA channels been prioritized based on a similar analysis?
- ☐ Have the CAN message been prioritized based on a similar analysis?
- ☐ Does the ADC use prioritization? Have the ADC priorities been set based on analysis?
- ☐ Have the Bluetooth LE notification/indications been prioritized based on a similar analysis?

51.7. CONCURRENCY REVIEW CHECKLIST

- ☐ Are the protected resources (and how they are protected) listed?
- ☐ Are there resource missing mutexes to protect them?
- ☐ Is the acquisition order of locks/mutexes defined?
- ☐ Are the appropriate IPC mechanisms specified?
- ☐ Is the order of multiple accesses defined?
- ☐ How do interrupts signal threads? Which threads do they signal?
- ☐ Are there ways to reduce the blocking time?

51.8. CRITICAL FUNCTION / SUPERVISOR REVIEW CHECKLIST

Check that critical functions (e.g. Class B and C of IEC 60730) are suitably crafted:

- ☐ Does the detailed design identify the critical functions?
- ☐ Are the critical functions limited to a small number of software modules?
- ☐ Is the relation between the input and output parameters simple as possible?
- ☐ Is a power supervisor / brown-out detect employed? Should one be?
- ☐ Are self-tests and/or function tests performed before any action that depends on the critical functions?
- ☐ Are periodic self-tests or functional tests performed? How do they work? Is a vendor supplied module performing the test? Which one(s)?
- ☐ Is there a defined acceptable state for when self-check (or other functions) fail?
- ☐ Are the clock(s) functionality and rates checked?
- ☐ Is a watchdog timer is employed? Correctly? Does the design only reset the watchdog after all protected software elements are shown to be live? An example of a bad design would be to reset the watchdog every cycle thru a run loop.
- ☐ Does the design describe where the watchdog timer may be disabled? Is this acceptable?

- ☐ Is an external watchdog employed? Is the external watchdog handshake done only after all of the software has checked liveliness? A bad approach is to use a PWM for the handshake, as a PWM can continue while software has locked up or is held in reset.
- ☐ Is there a fail-safe and fail-operational procedure defined to bring the product to the defined acceptable state? There should few such procedures (e.g. 1 or 2)
- ☐ Is there acceptable handling of interrupt overload conditions?
- ☐ Is the critical program memory is protected from modification? How? Hardware? Software?
- ☐ Is the program memory checked for validity? How? CRC check? Hardware based? Software?
- ☐ Is the stack checked for overflow? How?
- ☐ Is the critical data separated, checked, and protected? How?
- ☐ Are independent checks / reciprocal comparisons to verify that data was exchanged correctly? How does it work? For example, how does it know that the correct device and correct address within the device was modified or read?
- ☐ Are there possible partition violations from data handling errors, control errors, timing errors, or other misuse of resources?

51.9. MEMORY HANDLING REVIEW CHECKLIST

Has the memory been partitioned in a manner suitable for Class B? i.e., does the software isolate and check the regions?

- ☐ Does the detailed design outline good practices to prevent buffer overflows – bound checking, avoid unsafe string operations?
- ☐ Are memory regions write protected?
- ☐ Is the memory protection unit enabled?
- ☐ What is the access control configuration?
- ☐ Is it appropriate?

Non-volatile storage:

- ☐ Does the design not overwrite or erase the non-volatile data that is in use? Or does the design overwrite the most recent/good copy of the data?
- ☐ Does the design account for loss of power, reset, timeout, etc during read/write operation? This should include checking supply voltage before erasing/writing non-volatile memory, performs read back after write, and CRC data integrity checks.
- ☐ Are data recovery methods used? Will the design work?
- ☐ Does the design ensure that the correct version of stored data will be employed (e.g. on restart)?

51.10. POWER MANAGEMENT REVIEW CHECKLIST

Power configuration for low power modes:

- ☐ Are power management goals defined?
- ☐ Are the target power performance characteristics/requirements defined?
- ☐ How will it enter the states?
- ☐ How will it exit the states?
- ☐ Are the states of clocks, IOs, and external peripherals defined for the low power states?

- ☐ Is there a race condition in entering a low-power state and not being able to sleep or wake?

51.11. STATES

- ☐ Are the states of the module(s) defined?

For each state:

- ☐ Is the role of the state described?
- ☐ Are entry conditions given?
- ☐ Is an exit condition given?
- ☐ Is the flow of the normal sequence written down? Is it understandable, and reasonably implementable?
- ☐ Are any abnormal flow / exception cases defined?

51.12. NUMERICAL PROCESSING REVIEW CHECKLIST

Check for correct specification of numerical operations, such as might be used in signal processing, kinematics, control loops, etc.:

- ☐ Is there a description of the numerical processing that will occur?
- ☐ Is the theory of operation (e.g. that forms the system of equations) sound?
- ☐ Is it numerically sound?
- ☐ Are the equations ill-conditioned?
- ☐ Is the method of calculation slow? Is the algorithm slow? Is floating point emulated on the target platform?
- ☐ Would use of fixed point be more appropriate? Would float be more appropriate?
- ☐ Is simple summation or Euler integration specified? This is most certainly lower quality than employing Simpsons rule, or Runge-Kutta.
- ☐ Floats and doubles are not used in interrupt handlers, fault handlers, or the kernel.
- ☐ The RTOS is configured to preserve the state of the floating-point unit(s) on task switch.

51.13. SIGNAL PROCESSING REVIEW CHECKLIST

[Also apply numerical processing checklist.](#)

- ☐ Is the signal chain described?
- ☐ Is the relation between the input and output of the signal chain simple? Or at least, simple as possible?
- ☐ Is the sampling approach to linear signals (aka analog inputs) described?
- ☐ Is the description of sample acquisition time defined? Does it match with the hardware design description and target signal? (e.g. input impedance, signal characteristics)
- ☐ Is the method for acquiring samples appropriate? If the processing requires low jitter, the design should support this. For instance, a design that uses a DMA ring-buffer has low variation, while run-loop or interrupt trigger can have a great deal of time variation.
- ☐ Is oversampling applied? Is the design done in a proper way?

- ☐ Are appropriate forms of filter specified? Is an unstable form used? (Would the form have ringing, feedback, self-induced oscillation or other noise? Note: IIR is unstable only when the poles are outside of the unit circle.)
- ☐ Is the signal processing unnecessarily complex?

51.14. TIMING REVIEW CHECKLIST

- ☐ Is the sequence of interactions documented?
- ☐ Is the timing of interactions documented? Are the timeouts defined and documented?
- ☐ From the time the trigger is made to the action, what worst case round-trip? Include interrupts, task switching, interrupts being disabled, etc. Is this timing acceptable?

51.15. TESTABILITY

- ☐ Is the design testable?
- ☐ Has the design been sufficiently decomposed to support unit testing?
- ☐ Is the interface clear enough to trace test cases to cover all the functionality?
- ☐ Are the interface interactions states documented sufficiently – it can be integration tested with each entry/exit condition and state traceable to tests cases?
- ☐ Are the state flows sufficiently documented to test and trace?

51.16. OTHER

- ☐ Are there regular checks of operating conditions? Should there be?

[This page is intentionally left blank for purposes of double-sided printing]

CHAPTER 15

Software Detailed Design Risk Analysis

This chapter provides an initial template for software detailed design risk analysis.

52. SOFTWARE DETAILED DESIGN RISK ANALYSIS

The outputs of a software detailed design risk analysis include:

- A table mapping the software requirements to the detailed design element (e.g. procedure) that addresses it. *This table may have been produced by another activity and is only referenced in the output.*
- List of software risks, acceptability level, and their disposition
- A criticality level for each hazard that can be affected by software
- Recommended changes to the software design, software architecture, software requirements specification, programmable system architecture, etc. For example, actions required of the software to prevent or mitigate the identified risks.
- Recommended test Verification & Validation activities, especially tests

The steps of a software detailed design risk analysis include:

1. Identify the design elements that address each requirement. *This may have been produced by another activity and is only referenced in the output.*
2. Examine the risks of errors with values
3. Examine the risks of message capacity
4. Examine the risks of timing issues
5. Examine the risks of software function
6. Recommendations for rework

52.1. STEP 1: IDENTIFY THE DESIGN ELEMENTS THAT ADDRESS EACH REQUIREMENT

Go thru each of the software requirements and list the design elements that address it.

52.2. STEP 2: EXAMINE ALL VALUES FOR ACCURACY

Identify all the elements of the design – sensor 1, sensor 2, actuator, motor, calculations, operator inputs, operator outputs, each parameter received, etc. For each of these elements, create a copy of the table “*Value accuracy risks*” and populate it with respect to the architecture. In reviewing each condition, identify the least acceptable risk for each applicable condition.

52.3. STEP 3: EXAMINE THE MESSAGES CAPACITY

Identify all the messaging elements of the system – I²C sensor, task 1, user input, etc. For each of these elements, create a copy of the table “*Message capacity risks*” and populate it with respect to the architecture. In reviewing each condition, identify the least acceptable risk for each applicable condition.

52.4. STEP 4: EXAMINE THE CONSEQUENCES OF TIMING ISSUES

Identify all the input elements of the system – button #1, frequency input, I2C sensor, task 1, user input, etc. This list should include elements those receive messages and send messages. For each of these elements, create a copy of **Table 20** (“*Timing capacity risks*”) and populate it with respect to the design. In reviewing each condition, identify the least acceptable risk for each applicable condition.

52.5. STEP 5: EXAMINE SOFTWARE FUNCTION

This step examines the ability of the software to carry out its functions.

Identify all the functions of the system; that is, the operations which must be carried out by the software. For each of these elements, create a copy of **Table 39** (below) and populate it with respect to the architecture. (Strike inapplicable conditions)

Condition	Hazard, likelihood & severity	Table 39: Software function risks
Hardware or software failure is not reported to operator		
Data is passed to incorrect process		
Non-deterministic		
Non-terminating state		
Software fails to detect inappropriate operation action		

In reviewing each condition, identify the least acceptable risk for each applicable condition.

The risk analysis shall illustrate how events, or logical combinations of events, can lead to an identified hazard

An analysis shall be conducted to identify states or transitions that can result in a risk.

52.6. STEP 6: RECOMMENDATIONS FOR REWORK

Summarize each of the identified conditions with a risk level of “medium” or “high.” These items mandate rework, further analysis, and/or Verification & Validation activities.

53. SOURCE CODE RISK ANALYSIS

Note: Reviewing code is a broad activity that may emphasize for workmanship quality (is it maintainable), can take the form of formal, prepared meetings (i.e. Fagan inspections), to directed examinations. This only is about the specific analysis related risk analysis.

Note 2: This section is not written as an analysis. Rather it is more “inspect the code for workmanship and make recommended changes.” That is not identifying risk very well.

The outputs of a software source code risk analysis include:

- Recommended changes to the source code, software design, software architecture, software requirements specification, programmable system architecture, etc. For example, actions required of the software to prevent or mitigate the identified risks.
- Recommended test V&V activities

The steps of a software source code risk analysis include:

1. Tool-based analysis
2. Examine the software for proper initialization
3. Examine the risks of timing issues
4. Examine the software critical sections
5. Recommendations for rework

53.1. STEP 1: TOOL REPORTS

Tools.

- Tools that can inspect the source code for errors or misuse of the language employed.
- Management plan: All issues should be addressed. Exceptions should be explained and have a secondary check-off.

53.2. STEP 2: IS THE SOFTWARE INITIALIZED PROPERLY?

The code is to be inspected for match with the design document, and reviewer experience.

- Is the processor set up properly – clocks / oscillators turned on properly, etc? Is the processor power tree configured properly?
- Are the watchdog timers (or similar protective timers) set up properly and detect enough unresponsiveness in the code?
- Is the processor properly configured for the designed priority levels? That is, are the interrupts, ADC, DMA channel, and other priority levels

53.3. STEP 3: PROFIT!

The source code should be examined for data passing and control flow structure:

- Check for consistency in the data and control flows across interfaces.
- Check for potential partition violations caused by such occurrences as data handling errors, control errors, timing errors, and misuse of resources

- Check that the scheduling requirements are met / meet the timing constraints specified by the safety requirements / design.
- Check that the safety-related functions meet the timing constraints specified by the safety requirements / design.

53.4. STEP 4: EXAMINE SOFTWARE CRITICAL SECTIONS

This step examines the ability of the system to perform the functions that address or control risks.

For each of the safety requirements, inspect the code that addresses the requirement:

- Does the code completely address the requirement?
- Does it do so correctly?

Other checks

- The integrity of the partitions between supervisory, critical, and non-critical sections of software.

53.5. STEP 5: RECOMMENDATIONS FOR REWORK

Summarize each of the identified issues, observations and recommendations

PART III

Source Code Craftsmanship

This part provides guides for source code workmanship

- OVERVIEW OF SOURCE CODE WORKMANSHIP.
- C/C++ CODING STYLE used for C & C++ source code.
- CODE INSPECTION & REVIEWS. Describes code reviews.
- CODE INSPECTION & REVIEWS CHECKLISTS for reviewing source code.

[This page is intentionally left blank for purposes of double-sided printing]

CHAPTER 16

Overview of Source Code Workmanship

This part promotes good source code construction:

54. SOURCE CODE WORKMANSHIP

This part seeks to reduce bugs from language mistakes and mis-implementing the detailed design. It presents coding guides and review tools. Guidelines and review are widely used to help ensure that the software is:

- *Safe* – that the software can be used without causing harm,
- *Secure* – that it is resistant to attacks,
- *Reliable* – that it functions as it should, every time,
- *Testable* – that it can be verified at the code level,
- *Maintainable* – that it enables easy adaptation and medication,
- *Portable* – that it provides consistent functionality across all platforms

Source code should follow sound practices. Some of these practices are covered in industry guides, such as MISRA C. Chapter 17 gives specific coding guidance that the industry guides do not cover. These guides provide direction to producing clear code, with a low barrier to understanding and analysis.

Chapter 18 discusses review the resulting source code against the guides and the detailed design to help ensure that the result has a good construction.

The source code should be reviewed (and otherwise inspected) against those guides, designs, and against workmanship evaluation guides. The purpose of reviews is to examine quality of construction – it is not an evaluation of the engineers, and it is looking for more than finding defects. It is to get a second opinion on the implementation.

The review checklists & rubrics are complementary to the coding style; everything in one should be in the other.

54.1. WHAT DOES GOOD CODE LOOK LIKE?

Good code is

- *Well-structured*. The code is consistent, simple, and neat, using accepted (or mandated) practices.

- *Structured simply.* It uses simple operations, with one action per line. It modularizes effectively. It limits a function to fit one screen of code.
- *Clean interfaces.* It passes minimal data, reducing memory requirements and increasing speed. It exposes only variables that are necessary. It minimizes dependencies and confines processor dependent code to specific functions.
- *Functional.* It has been tested frequently, completely, and thoroughly. It uses a layered approach to add the needed complexity.
- *Well commented.*

54.2. THE ROLE OF REVIEWS AND INSPECTIONS

The purpose of reviewing the work is to examine quality of construction (the workmanship). (It is not an evaluation of the engineers.) Code review is looking for more than finding defects.

Reviews check that:

- The construction is consistent, and coherent
- That the style is easy to understand, and clear
- That the work is maintainable over time, by many people
- That it avoids known and potential defects
- Consistent execution
- Evaluate quality of construction
- Planning goals for schedule and quality
- Improve meeting quality goals

The reviews can also be used as an education for new team members.

Tools can be used to automate some of the checks, relieving some of the reviewer labor:

- Clang-format
- MISRA C checks
- CPPCheck
- Clang-Tidy
- Compiler tool warnings

CHAPTER 17

C/C++ Coding Style

This chapter describes the subset of C/C++ that will be used, and how to format and document the source code.

- Coding style overview
- Naming
- Source code text formatting: code layout, braces, spacing,
- Documenting the code
- Conditional compilation
- Macros
- Namespaces, used to organize the code and provide some separation of 3rd party libraries.
- Preferred types
- Storage organization, and access considerations, including concurrency.
- Structure of a procedure; control flow: conditionals, goto/label/return/break/continue
- Special cases and troublesome areas

The scope of this guide is the *implementation* – the source code – of a design, rather than the design itself.

55. CODING STYLE OVERVIEW

A coding standard is used to promote understandable source code that is:

- **Reliable:** The code should consistently perform as intended, without unexpected behavior. Robust programming practices are encouraged; unreliable ones discouraged.
- **Testable:** The code should facilitate comprehensive testing.
- **Cohesive, and Readable:** Clear structure, naming and comments convey code intent, making the design understandable by others, and reducing misinterpretations.
- **Maintainable:** The code structure and organization facilitate easy updates and modifications, reducing the effort required for long-term maintenance and evolution.
- **Portable:** The code works the same across a variety of target environments.
- **Consistency:** The code should maintain a uniform look and style. Consistency ensures understanding of intent across similar code structures.

55.1. SOFTWARE LANGUAGE

The software for applications created with this guide are written in the ANSI C11 and a subset of the C++23 programming languages.

Exceptions to these rules are permitted but must be justifiable.

Compiler specific behavior (also called “compiler defined behavior”) should be used frugally. This includes extension from Clang, Keil, IAR, MDK, Microsoft, and GNU. Wrappers that present functionality consistently to the program – but are implemented in terms of the compiler defined behavior – may be used when other alternatives are not available.

Where possible, use standards that cover the language specifics. MISRA is one that goes into great depth on the types, expressions, and other nuances of the language. It is well written and reasoned. Being a standard, there are many tools that can automate checking for these.

Employ unit testing frameworks, in conjunction with tools such as address sanitizer and thread sanitizer.

56. NAMING CONVENTIONS

This section consolidates the naming rules for the many kinds of objects that can exist. The goal is a consistent naming convention to aid code readability, maintainability, and collaboration. (Later sections will delve into specialized rules for each kind of object). The principles are:

- Names should be *descriptive*, indicating the type of value, or action performed:
Examples: ``ValidateUserInput()`` ``totalPrice_dollars``
- Use *consistent terminology* throughout the project to maintain understanding.
- Avoid using *abbreviations* in variable and parameter names unless they are widely understood and add clarity.
- Do not use Hungarian notation.
- Most names should use camel case or pascal case, with lower case letters, capitalizing acronyms and the first letter of each word.

Modules. (C module or C++ classes). Each module is named. Stick to standard to acronyms and abbreviations for the module’s identifier.

Macro – Macro names are typically uppercase, with words separated by underscores. Uppercase is not required, nor a good practice, but is common.

Example: ``VALUE_MAX``

Class, Structures, Enums, Unions, and similar:

- The first letter is upper case, aka PascalCase
- Do not use a suffix (that is, no “_t” or “Enum” etc at the end).
- When typedef’d:
 - Same capitalization
 - end with ‘_t’
- Tag (field) names – no special designation
 - scalar and pointer should begin with a lowercase letter but may begin with an uppercase letter.

Method and Procedure Naming Conventions:

- **camelCase** or **PascalCase**. Method names should use PascalCase (see capitalization rules earlier), starting with an uppercase letter (unless otherwise specified).
Example: ``CalculateTotalPrice()``
- **Specific names**
 - ``init()`` for initialization of components or modules.
 - ``update()`` for methods that update the state of components or perform regular updates.

Variables

- Variables (parameter, locals, globals, fields, etc) names begin with a lower-case variable.

“Private” use Methods, Procedures, and Variables:

- Use *underscores at the end of names* for items that are not intended to be used by the rest of the application but must be external. Prefer trailing underscores (to leading ones) as the names sort in an alphabetizing listing better.

Units for Quantities

- Variables, macros, and procedures for quantities should have the unit at the end. An *underscore* precedes the unit.

56.1. CONSTANTS

Constant numbers and numbers used to arbitrarily represent state or enumeration – so called *magic numbers* – are to be given meaningful names that represent their purpose, role or use, and units.

Use `enum`, `enum`, `const variables`, or `defines`. Do not give numbers a name that is a mere anglicization of the digits or quantity. Not every number is magic.

Examples of wrong names: ZERO, ONE

56.2. C SPECIFIC

- Procedures, and exported variables are prefixed with their module identifier, followed by an underscore.
- Prefix `enum` values with an identifier, followed by an underscore. The prefix should be predictable from the `enum` name. For instance, an `enum` with type name of “`Err_t`” would have a value prefix of “`Err_`”

56.3. MICROCONTROLLER SPECIFIC NAMING

Microcontroller families may have conventions around interrupt and fault handlers:

- The microcontroller fault (or exception) handler – name ends with `_IRQHandler` (matching CMSIS guidelines)
- Interrupt service routines end in `ISR`
- “IRQ” is sometimes used for the interrupt handling routine, but most often used to design the interrupt, its internal “number”, management – enabling/disabling, prioritizing, and so on.

56.4. CONFIGURATION VARIABLES AND MACROS

The suffix for defines and variables (esp. in configuration files) are:

Suffix	Description
<code>_CNT</code>	For the number of instances.
<code>_DEBUG</code>	For gathering extra information, not normally used in production.
<code>_EN</code>	To enable or disable use of the feature, module, or hardware.
<code>_MAX</code>	The maximum allowed value for the item.
<code>_MIN</code>	The maximum allowed value for the item.

Table 40: Suffixes for Configuration Macros and Variables

57. SOURCE CODE FILE FORMATTING

57.1. CHARACTER SETS

The files shall be in ASCII and UTF8.

Tab characters shall not be used in software source code. Indents shall use spaces, not tab characters.

57.2. FILE GROUPINGS FOR A MODULES IMPLEMENTATION

A module may have many .cpp or .c files that implement the module – it is better to break down a module into groups of relatively short files rather than one large file a thousand lines or longer. The module may have other .h files (suffixed as `-int.h`) that are for use only within the module.

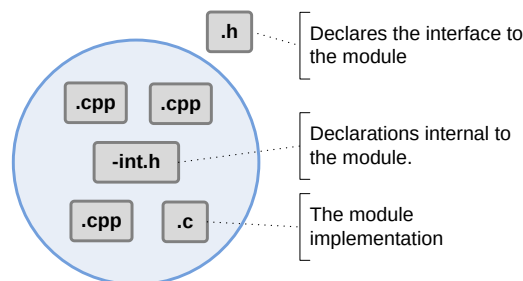


Figure 42: How .h and .c files related to a module

The documentation distinguished between external interface (procedure and variables other modules may use) and an internal one.

A module or subsystem may have many .c/.cpp and .h files that implement the module.

57.3. FILE NAMES

The file names should be prefixed with the modules prefix (if any).

57.4. HEADER FILES

The interface for a module or subsystem is provided via .h header files; multiple modules may employ the same header. It declares the types, procedures, variables, and macros for other modules to use.

- This file should not have ‘internal’ only information; that is, it should not contain information that other modules *should not use*. A practical exception is made for objects that are necessary for successful compilation, such as C++ classes/structs which must provide the private variables methods.
- This file shall not have an “-int” suffix.

Header files with a “-int” suffix (such as “Mem-int.h”) for use only by the module or subsystem.

Header files should have the same name as the .c/.cpp file that it provides an interface for.

Header files should declare classes, struct, variables and procedures. It should not define variables, class/struct objects, or non-inline procedures.

57.4.1 Guards

Header files (file ending with the extension .h) may have guard defines in the file, so the file’s declarations/definitions are not made twice.

GOOD:

```
#ifndef MYSTUFF_H
#define MYSTUFF_H

...

#endif
```

RATIONALE. Some header files create include cycles, forever including each other. This guard prevents the cycle of inclusion. Such a cycle is often a sign of poor planning; it is better to resolve the include cycle and improve division of declarations to have acyclic dependencies.

57.4.2 Extern declaration / procedure prototypes

Extern declarations and procedure prototypes are provided only in header files. (Not in C/Cpp files). There is to be only one declaration of a variable, macro, procedure or any other symbol.

Use of extern “C”. If a C++ header file needs to include a C function the ‘extern “C”’ directive is used to prevent the C++ name mangling that would otherwise occur.

57.5. FILE LENGTH

A file should be relatively short. A file shall be less than a thousand lines. A file should be less than 500 lines.

A module or subsystem may have many source files that implement the module.

57.6. TEXT FORMATTING

57.6.1 Spacing

Indentation should be 4 spaces; this is standard for most C/C++ code editors. It’s just easier to work with the *de facto* standard. Exceptions:

- Code inside of ‘extern “C”’ and a namespace block stay at the same indentation level as it would be outside of the; these do **not** increase the indentation level of code.

Spacing

- Use spaces around keywords, operators and colons.
- Use spaces after commas, and semicolons.

57.6.2 Line declaration a symbol (field, variable, enum value)

A line declaring a symbol (field, variable, enum value) should not start with a comma.

RATIONALE. This messes with doxygen.

57.6.3 Brace placement

Braces are on their own lines. Closing braces should be at the same column as the opening brace. The exceptions are namespaces and `extern "C"` where the open brace goes on the end of the line.

57.7. DOCUMENTED CODE

The source code must be documented completely. Tools will consolidate the documentation into a form containing the information developers need to understand and use the interface.

Each procedure (or function, or method), macro, variable and type declaration must have descriptive comments. Provide the documentation where the item is defined:

- Classes, structs, etc. are documented where they are declared, e.g. in the header file.
- Procedures and methods are documented in the file (c/cpp) if they are defined in (rather than where they are declared). It is acceptable to have a copy of the documentation in the header file but no longer required. The copy must match exactly.

RATIONALE. Lessons learned over the years:

- When in the documentation is in the header file, the code is more difficult to review, and more tedious to update the header documentation (and keep it consistent) with the code.
- People only read the main header file to get the overall design and only if it's a (relatively) small 3rd party module: a single file not more than few hundred lines. Huge headers, with change logs etc. are just too much text. Multiple header files are just too much muchness.
- Users of the library most likely will ignore the manual and look at an example.

57.7.1 File description (at top of file)

Files must have a brief description of the module, stating its purpose.

```
/**@file
  @brief Manages blinking functionality for display items.
```

Do not fill in the file name. The file name is automatically filled in by the doxygen.

Files that define macros, or – outside of a namespace – defined types or declare variables/procedures should have a detailed description; other source files should not. The namespace and class descriptions have proven a better organization for the documentation.

Files providing macros or variables/procedures outside of a namespace must have a description.

```
/**@file
  @brief Manages blinking functionality for display items.
```

Detailed description of the module, its purpose, and how it fits into the overall architecture.

*/

57.7.2 Procedures and Macro documentation

The comments before a procedure (or function, or method) and macro should include information about the use of the procedure. The comments should articulate the action of procedure, how it is being done; explain why (implementation) choices were made. These comments must contain:

- A capsule synopsis of the procedure's intent.
- A description of each of the input parameters, including any requirements for the parameter
- The specification of the return value, results, and output parameters.
- Changes to any global or shared data.
- A description of its function – what it the procedure does and its role. This often should include details of the implementation.

Comments should not be placed between the procedure name and the opening brace.

57.7.3 Enum, structs, classes, and variables

The comments before an enum, struct, class, or type declaration should include

- A description about the intended use of the object
- Comments should not be placed between the name and the opening brace.
- Each field, and label should be documented
- The comments should be written in complete sentences.

57.7.4 Making lists

Don't use numerical markdown lists in the doxygen comments. Doxygen will not produce correct markdown output from them.

Do not use nested lists, as doxygen loses the list structure.

57.8. LONG LINES

When you split an expression into multiple lines, split it before an operator, not after one:

```
if ( foo_this_condition && bar > win(x, y, z)
    && remaining_condition)
```

Try to avoid having two operators of different precedence at the same level of indentation. For example, don't write this:

```
mode = (inmode[j] == VOIDmode
        || GET_MODE_SIZE (outmode[j]) > GET_MODE_SIZE (inmode[j]))
      ? outmode[j] : inmode[j];
```

Instead, use extra parentheses so that the indentation shows the nesting:

```
mode = ((inmode[j] == VOIDmode
        || (GET_MODE_SIZE (outmode[j]) > GET_MODE_SIZE (inmode[j]))))
      ? outmode[j] : inmode[j];
```

58. CONDITIONAL COMPILATION

When adding conditional compilation, define a feature flag as 1 (to enable it) or 0 (to disable it); the flag name should end a ‘_EN’:

In a header configuration file:

```
#define FEATURE_EN (1)
```

and in the code

```
#if FEATURE_EN  
... some code ...  
#endif
```

RATIONALE. This style allows detecting a missing include, and misspelled feature flags. When an undefined preprocessor symbol is used, the compiler will report the issue.

Do not use a “#define FEATURE” to enable a feature or “#ifdef” to tell if a feature is enabled.

Wrong:

```
#define FEATURE  
  
#ifdef FEATURE  
... some code ...  
#endif
```

RATIONALE. Use of #ifdef would silently make a wrong choice – if a file was accidentally not included or there was a typo – with the intended option was not seen #define’d by the compiler.

58.1. SPECIAL CLAUSES FOR DOXYGEN

Sometimes the doxygen documentation is within a conditional code block. Doxygen will skip over this. To allow doxygen to analyze and include the documentation, the condition needs an exception if it is for doxygen in the clause:

```
#if FEATURE_EN || defined(DOXYGEN)
```

59. MACROS

There are three “kinds” of macros discussed in this section:

- Macros that act as expressions,
- Macros that act as statements (possibly with control flow).

Note: this section doesn't discuss #defines of values and feature flags, such as those used for configuration.

59.1. MACROS THAT ACT AS EXPRESSIONS

PRINCIPLE OF USE: #define macros that are (or act as) an expression must be wrapped in parenthesis.

RATIONALE. The macro expansion can have unintended effects.

59.1.1 Examples of effects

The following provides an example of a bad case:

```
#define MyMacro(x) 1L + x

MyValue = 3L * MyMacro(v);
```

The above will expand to:

```
3L * 1L + v
```

Rather than the intended expansion of:

```
3L * ( 1L + v)
```

59.1.2 Macro parameters

PRINCIPLE OF USE: The parameters to #define macros must be wrapped in parenthesis [within the macro body]

RATIONALE. The macro expansion can have unintended effects

EXAMPLES OF WHERE TO USE. The following example shows how the parameters are wrapped in a parenthesis:

```
#define multipleAccumulation(m,b) ((m)*3L + (b))
```

EXAMPLES OF EFFECTS. The following example shows how the parameters, when not wrapped in a parenthesis, can interact in unintended ways:

```
#define MyMacro(v) (v * 3)

local3 = MyMacro(local1 + local2);
```

This will expand to

```
local3 = local1 + local2 * 3;
```

Rather than the intended

```
local3 = (local1 + local2) * 3;
```

59.2. MACROS THAT ACT AS STATEMENTS

PRINCIPLE OF USE: #define macros that use complex expressions – those with statements, if-then, whiles, etc – or code blocks must be wrapped in do{}while(0)

These macros must have parenthesis (even if there are no parameters).

RATIONALE. The macro expansion can have unintended interactions with other control structures

59.2.1 Examples of effects

The following provides the first example a bad case:

```
#define BlipOn() if (blipPtr) *blipPtr = 1;
#define BlipOff() if (blipPtr) *blipPtr = 0;

if (myVar == 3)
    BlipOn();
else
    BlipOff();
```

Expands to the equivalent of

```
if (myVar == 3)
{
    if (blipPtr)
    {
        *blipPtr = 1;
    }
    else if (blipPtr)
    {
        *blipPtr = 0;
    }
}
```

Rather than the intended:

```
if (myVar == 3)
{
    if (blipPtr)
    {
        *blipPtr = 1;
    }
}
else
{
    if (blipPtr)
    {
        *blipPtr = 0;
    }
}
```

The following provides an example, where while's can interact inappropriately with the surrounding code:

```
#define WaitForSignalToGoLow() while (*input1)
```

Used within code:

```
// Wait for signal #1 to go low and then set led on
WaitForSignalToGoLow();
*ledPtr = 1;
```

This becomes

```
while(*input1)
{
    *ledPtr = 1;
}
```

Rather than the intended:

```
while(*input1)
{
}
*ledPtr = 1;
```

59.2.2 How to fix these problems

The body of these macros can be wrapped in do{...}while(0) statements. In the first example:

```
#define BlipOn() do{if (blipPtr) *blipPtr = 1;}while(0)
#define BlipOff() do{if (blipPtr) *blipPtr = 0;}while(0)
```

The example expands to the equivalent of

```
if (myVar == 3)
{
    do
    {
        if (blipPtr) *blipPtr = 1;
    }while(0);
}
else
{
    do
    {
        if (blipPtr) *blipPtr = 0;
    }while(0);
}
```

In the second example:

```
#define WaitForSignalToGoLow() do{while (*input1);}while(0)
```

The example expands to:

```
do
{
    while(*input1);
} while(0);
*ledPtr =1;
```

59.2.3 Other comments

There are two other mitigations for the problems in the example code:

1. The body for if, else, while, for, do, etc. should be wrapped in {}.
2. Avoid using macros with statements, conditionals, loops, etc.

60. NAMESPACES

Namespaces allow organizing the source code and prevent some potential integration issues with 3rd party libraries.

- A single, top-level project namespace is used to prevent name conflicts with 3rd party code and organize the projects modules.
- All non-3rd party modules go in this namespace hierarchy.
- The module names can be *clean* – prefixes are no longer needed to prevent clashes.

The namespace convention used here is:

```
namespace Project::ModuleName {
}
```

or

```
namespace Project::CSP::ModuleName {
}
```

Where *Project* is the name of the namespace for the overall project (family of related repos), and *ModuleName* is the particular unit of code.

Note: the opening brace goes on the same line as the namespace keyword.

Source files that use the module employ lines like the following at the top

```
using namespace Project;
```

Or

```
using namespace Project::ModuleName;
```

This seems to aid making the main code cleaner and easier to read.

`using namespace` is not allowed in header files.

60.1. A LITTLE BACKGROUND

Two of the tasks in software development are managing naming (of the software pieces) and complexity. The simplest of programs needn't worry too much about naming. But as the software gets larger – more modules, procedures, variables, larger teams, and increased use of 3rd party modules – there is more to be done to keep it sane. Projects usually want:

1. A way to sensibly group module resources together,
2. Aids to keep others from using interfaces (procedures, variables, types) that are not meant to be depended on or used by others,
3. Help to prevent procedure/variable names from colliding with other those from other (sub)teams or in 3rd party libraries.

In C (and similar languages) the tricks are:

1. Declare some variables and procedures 'static' so that they aren't exported outside of the module (thus won't collide).
2. Use prefixes to the names, based on their module. Good practice has preferred succinct (if terse) prefixes.
3. Use structs to wrap the variables; this is rarely used with "single instance modules."

In firmware, this often works well. And it works fine for small and medium size projects.

The drawback is that the name and struct tricks make the code harder to read, and complex.

A C++ namespace hierarchy – a top-level namespace, with several sub-namespaces underneath – is generally expected. The top level protects against the 3rd party collisions. The sub-namespaces are for modules.

60.2. USE NAMESPACES, ESP. INSTEAD OF STATIC METHODS, GETTER & SETTERS

Keep items local to a file using an anonymous name space, rather than being made static.

- Use a namespace rather than a class/struct; Instead of passing object pointer (or equivalent) around, even indirectly, uses the namespace to hide or control access. Doing it this way has proven to have much less noise than the conventional alternatives.
- Use procedures in a namespace rather than static methods in a class.
- Use direct access to variables where sensible; use getters and setters sparingly, if ever.

61. PREFERRED TYPES

This section is only for the selection of types (and qualifiers) to use, and the defining of types. A later section will discuss the nuances of proper use of some types, and expressions of that type.

Use proper, standardized C11 types, rather than creating new or ad hoc type names.

61.1. TYPEDEFS

Use typedefs sparingly. They should be used to reduce the amount of typing required to declare a variable.

- Do not typedef a struct or class. (C++)
- Do not typedef a pointer.
- Use a ‘_t’ suffix for a typedef name, except for classes and structs.

61.2. ENUMS

Do not use a suffix for an enum name.

Use ‘enum class’ for enumerations. (C++)

C:

- enum values shall be prefixed with an identifier relevant to the enumeration, followed by an underscore, the specific name of the value. The case of the enum value is not specified here. It is recommended that if the enum has tag name, and/or a structure name that the prefix be predictable from it. For instance, an enum with type name of “Err_t” would have a value prefix of “Err.”

C++

- Do not prefix the enum value name

61.3. UNIONS

Do not use unions.

61.4. BOOLEAN TYPES

Use stdbool and bool for boolean types. Do not create your own boolean type.

Use of a custom-defined boolean may be mandated if an unchangeable 2nd or 3rd party code employs. If so, do cast this boolean or use implicit coercion.

61.5. CHARACTERS AND STRINGS

char should only be used to represent characters, and nothing should be assumed about its sign. Characters might use char type, or a wider type, as appropriate.

Strings may use char const*.

Text strings should be zero-terminated UTF-8 strings without embedded nulls.

61.6. NUMERICAL TYPES

61.6.1 Quantities

To represent a quantity, the following types, units, and naming are recommended:

Dimension	Type	Units	Suffix	Description
angle	float	radians	_rad	
count	uint32_t		_cnt	The number of items.
distance	float	meters	_m	
duration	timediff_t	ms	_ms	The duration of something in milliseconds.
size	size_t	bytes	bytes	The size of something, in bytes. Avoid `int` and `unsigned int` types for sizes.
size	ssize_t	bytes	bytes	When negative values are needed, such as with error returns, etc.
speed	float	m/s		
temperature	float	C	_C	
timestamp	clock_t	ms		The timestamp of something using system high-resolution clock.
timestamp	time_t			The timestamp of something can be converted to local time, such as a calendar date or time of day.

Table 41: The preferred types for quantity, by dimension

Do not use signed types for timestamps, durations, counts, etc. They are always non-negative.

61.6.2 Integer numbers

Integer types shall use types as outlined in the quantities section above and the table below; these are C11 style, with all lower cases, and `\_t` suffix:

Size	Signed	Unsigned
8 bits	int8_t	uint8_t
16 bits	int16_t	uint16_t
32 bits	int32_t	uint32_t

Table 42: The preferred integer type for a given size

Number literals are to use a suffix to match type. These suffixes are uppercase.

Wrong:

```
int l;
for (l = 1; l < 32; l++)
{
  ...
}
```

Correct:

```
uint8_t l;
for (l = 1U; l < 32U; l++)
{
  ...
}
```

}

61.6.3 Floating point values

Do not use doubles.

See later section for a discussion of using floating point numbers in algorithms.

61.6.4 auto [c++]

The 'auto' type may be used with local and template variables.

61.7. POINTERS

Pointer should use the specific type, if known. A generic type should be void*.

Pointers are to be pointers to a const target, except where they explicitly will change the referent.

- Use nullptr instead of NULL. (C++ only)
- Use uintptr_t and intptr_t if a pointer needs to be converted to an integer type or stored in a variable that may be used as an integer.
 - Do not use int, long, unsigned, uint32_t, etc. for pointer types.

When taking the difference of two pointers to compute the size in bytes, cast the pointers to uint8_t const* or char const* first.

61.8. STORAGE CLASS

The register storage class shall not be used.

61.9. QUALIFIERS

Scope Qualifiers (e.g. static, extern) should always go before the thing they modify, not after.

Type qualifiers that control how a value may be modified:

- Use constexpr where possible for constants, expressions, and functions that can or must be computed at compile-time.
- Use const for constants, and items that should be modified.
- Use volatile (and std::atomic<>) when working with items that are access in concurrent environments, or by hardware.

61.10. MULTIDIMENSIONALS ARRAY

Do not use C's multidimensional arrays.

RATIONALE. Dereferencing multidimensional is frequently (near universally) misunderstood. For instance,

```
int array[9][20];
```

produces 9 arrays of 20 integer arrays. It is often misunderstood to produce 20 arrays, each holding 9 integers.

62. MEMORY

Valid References. At no instant may a pointer refer to an invalid (e.g. released) item.

No resource leaks:

- An allocated memory structure **must have** an instance, register, etc. pointing to it at all times; at no instant may such a structure not be pointed to.
- There must exist a path of reference from global structures, stack structures, and registers, to all allocated resources.
- In reference counted systems, code must balance a retain with a release; typically, an allocation is considered a retain. A release should occur at the same level as a retain.

62.1. PACKING STRUCTURES

Use the ``PACK()`` macro to pack structures.

Packing structures is a technique used to force data structures to have a predictable memory placement, while still allowing lexical access to fields. It involves arranging the data members of a structure in a way that reduces the amount of padding added by the compiler for alignment purposes. This is particularly useful in embedded systems or when dealing with low-level hardware interfaces where memory is limited.

Packing a structure in C and C++ is compiler specific. The ``PACK()`` macro is used to control the packing alignment and provide portability between compilers.

Here's an example of how to define a packed structure:

```
PACK(struct MyStruct
{
    int a;
    int b;
});
```

The macro is defined in a per compiler specific manner. For instance, this is the definition for compiler supporting the GNU attribute (e.g. GNU C, clang, IAR)

```
#define PACK( ... ) __VA_ARGS__ __attribute__((__packed__))
```

And for MSVC:

```
#define PACK( ... ) __pragma( pack(push, 1) ) __VA_ARGS__ __pragma( pack(pop))
```

When not to use packing. Many platforms have strict rules about alignment of access. Using packed structures – instead of procedure to encode/decode or serialize/deserialize them – is only allowed in special cases: where the alignment is guaranteed not to cause exception with the hardware, and the host endian matches. This must take into account, if the structure may be embedded within a received buffer, where its initial offsets can be, and how that impacts the subsequent field alignment.

62.2. DATA BUFFERS AND CROSS CHECKS

Buffer over and under runs are very common form of software bug. To help detect these bugs, is to place a canary before and after each buffer or array:

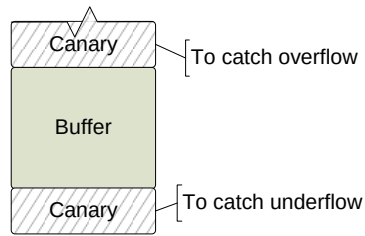


Figure 43: Overview of buffers with canaries

This is often done using a struct.

62.3. USE STDATOMIC.H AND STD::ATOMIC<> FOR CONCURRENT ACCESS

Use `std::atomic<>` for any type or variable that may be accessed in one context, and modified in another context. Contexts could be interrupt handlers, OS tasks, OS timer handlers, multi CPU (or multi core) shared memory, etc.

Examples of where to use:

- Non-locals accessed in an ISR
- Variables/structures modified in one OS task (or thread), accessed in another
- Variables/structures accessed in a OS timer handler, and in a task (or thread)

Note: `atomic<>` is to be used in conjunction with mutexes in concurrent environments.

62.4. THE VOLATILE QUALIFIER

PRINCIPLE OF USE. If `std::atomic<>` is not available or applicable, use `volatile` with anything that may be accessed in one context, and modified in another context. Contexts include interrupt handlers, OS tasks, OS timer handlers, peripheral register access, multi CPU (or multi core) shared memory, etc.

Examples of where to use:

- When accessing CPU registers or peripheral registers
- Non-locals accessed (read or modified) in an ISR or SysTick handler.
- Variables/structures modified in one OS task (or thread), accessed in another
- Variables/structures accessed in a OS timer handler, or task (thread)

MOTIVATION. The programmer's mental model is that modification is immediate. Without `volatile` the compiler has the option to delay, or reorder committing changes to the memory (or register); the compiler also has the option to reuse previously accessed values, rather than fetching an updated value from the underlying storage.

Note: for accessing things larger than a single atomic unit (e.g. in the ARM Cortex non-32bits aligned), further protection is needed.

EXAMPLES OF EFFECTS. The software without optimizations (or with a particular optimization setting), but with optimizations, it doesn't anymore. The following pseudo code as an example:

```

set GPIO pin high
wait 1uSec
set GPIO pin low

```

This code might not create a blip. The compiler might toss out all of the GPIO pin modifications, except the last one.

62.5. USE ATOMIC MODIFY, MUTEXES, ETC

A re-entrant approach must be used when modifying shared variables.

MOTIVATION. Read-modify-write race conditions are a very common bug. Some examples

```

Var++;
Var |= x;
Var &= ~1;

```

To perform any of those operations, the word is first loaded into a register, then the operation is performed, and then word is written back to memory. During the steps, at any time another thread or interrupt handler can access and modify the variable; but change will be lost when the first thread overwrites it – with the value it has in its register, never having read the modified one.

There are several acceptable ways to perform to modify a shared variable:

- Use atomic increment, decrement etc. These are the highest performance. On ARM processors these can be created using LDREX and STREX. Otherwise,
- Use mutexes around modification to values,
- With interrupts/exception handlers, interrupts must be disabled.

63. PROCEDURE STRUCTURE

A procedure should look like:

```

/**Synopsis of the procedure
  @param param1. Description
  @return description

  description
  */
ErrType_t MyProcedure(param1)
{
    // Check parameters for bounds.
    if (...)
    {
        // On error:
        // Set error message
        // Perform error return
        return err;
    }

    // Do work, Check results
    ...

    // Return with any errors
}

```

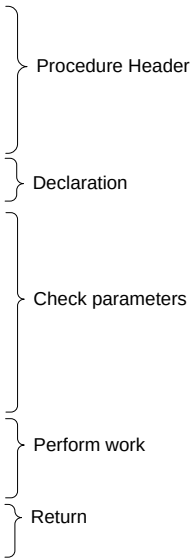


Figure 44: Typical procedure template

The elements are

1. The descriptive documentation (see earlier section on documentation for complete details)

2. Declaration, with parameter list. Procedures must be defined or declared prior to use; all extern procedures are declared in header file; static procedures are declared (if necessary) at the top of current file. The header file declaration was discussed in a previous section.
3. The procedure itself checks its parameters
4. Performs work, and checks the error return of all calls
5. Returns with error code

63.1.1 Comments

Comments are required except where the code is trivial. It is better to explain every line than to argue that the code is the documentation; source code is *never* self-documenting.

Block comments should be at the same level of indentation as the surrounding code. Within the comment, the text should align vertically.

Other comments (using the `//`) at the end of the line may be used, but this is not recommended. When used, these should align vertically.

Have at least one line of whitespace before a comment (except the first line inside of a block).

A comment should provide information needed for maintenance. The comments should complement the code to provide an understanding by describing what the goal and/or action is, how this code fits in the larger context; explain why (implementation) choices were made – the rationale behind its approach – as well as how it works.

The comment text should avoid restating the obvious.

The comments should be written in complete sentences.

Writing good comments is a skill. See [Ousterhout 2022] for a discussion of techniques to improving comments, ones that are meaningful, and not shallow restatements of the code.

63.2. PARAMETER LISTS

Procedures with no parameters shall be declared with parameter type `void`.

RATIONALE: A procedure declared without a parameter list does *not* mean no parameters are to be passed. It means that nothing was said about what the parameters may be. This is ambiguous.

63.3. DO NOT USE VARARGS

Procedures shall not use variable numbers of arguments, such as `varargs`.

RATIONALE: A variable number of arguments frequently introduces several kinds of bugs. A procedure may erroneously access more parameters than were passed. It may erroneously use a different type of access than was used to pass it. Types are coerced to ints and doubles; few programmers are aware of this. There is no type checking on passing values.

63.4. DO NOT USE A STRUCT AS A PARAMETER VALUE TYPE

Do not use a struct as a value type for parameter. Use a `const` pointer or reference instead.

RATIONALE: This copies the entire struct onto the stack to pass it.

63.5. DO NOT USE A STRUCT AS A RETURN VALUE TYPE

Avoid using a struct as a return value type.

RATIONALE: This can increase stack usage on resource constrained targets. Many compiler implementations copy the entire struct onto the stack to return it. While others the caller allocates space on the stack and passes a pointer to it to the called procedure. The later is the case for C++ (with return value optimization).

63.6. PARAMETER CHECKING

The input parameters should be checked for acceptable value ranges. This should be done prior to performing any other work.

63.7. RETURN VALUE CHECKING

Return values are to be checked. If they are to be ignored, comment must explain why, and use a construction like:

```
(void) funcCall(param1, etc); // Error doesn't matter for reason XYZ
```

RATIONALE: Return codes often include error indications or resource handles. Not checking the return values is a common source of software flaws, and incorrect error handling.

63.8. SIZE

Procedure should be small. Procedures should be small enough to fit comfortably on a screen.

RATIONALE: Big procedures are poor modularization, and undermine maintainability.

Longer procedures tend to have redundant code, something that rarely is a benefit.

63.9. INTERRUPT SERVICE ROUTINES

Interrupt handlers should have the `_IRQ_` pseudo-qualifier. Many environments (e.g. Keil) define this as nothing; the GNU C environment defines it as an interrupt attribute.

An example

```
_IRQ void fun IRQHandler()
{
    .. do stuff ...
}
```

Example 1: IRQ handler

Use the following guidelines for interrupt service routines, and procedures that are effectively interrupt service routines:

- Do very little in the interrupt service routine, only what is necessary. Push the rest of the work to the main application.
- Do not use unbounded loops in an interrupt service routine
- `atomic<>` or the `volatile` qualifier must be used to access anything modified in the ISR and another context (e.g. ISR, fault handler, main loop or a task).
- The interrupt service routine must not use mutexes or pend on IPC mechanisms.

- The interrupt service routine must not disable global interrupts.
- The interrupt service routine must not use floating point.

note: this applies to anything that the handler may call, directly or indirectly.

The ISR documentation should include:

- The function of the ISR. Common ones include:
 - GPIO rising/falling edge input
 - Compare / capture
 - ADC interrupts
- The work of the interrupt service routine, including its flow.
- The bounds of the ISR execution time.
- The work of the access procedure – the task or main procedure that receives the results of the interrupt. How does it check the values?
- When should the other contexts disable the routing?

63.10. EXCEPTION HANDLING ROUTINES

The exception handler – or microcontroller fault handler –

- `atomic<>` or the volatile qualifier must be used to access anything used in the exception handler and another context (e.g. ISR, fault handler, main loop or a task).
- The handler must not use floating point.

note: this applies to anything that the handler may call, directly or indirectly.

Excepting PendSV, and SysTick the handler should:

- Trigger a software breakpoint, to allow debugging
- Put the outputs into a safe state
- Reset the system

63.11. MISC

Don't disable interrupts for very long.

64. EXPRESSION, OPERATORS & MATH

64.1. BOOLEAN

Do not cast essentially Boolean types and expressions to any other type or use them as another (implicit casting).

64.2. THE PRECEDENCE OF C'S SHIFT OPERATORS

The C shift operators have a non-intuitive precedence. They should be used carefully:

1. Shift operations must be inside of a parenthesis – at least, if there are any operations to the left or right of it.
2. The left hand and right hand operands must be in parenthesis, if they are an expression. (That is to say, it must be "(4+2)" not "4+2".)
3. If a compiler has a flag to force precedence checking on >> as an error, it should be used;
4. If a compiler has a flag to report possible errors on >>, it should be used.

COMMENT: Lint and many compilers, like Microsoft C's compiler, do give a warning. The bad news is that the error messages are pretty hard to understand:

```
warning C4554: '>>' : check operator precedence for possible error; use parentheses to clarify precedence
```

PRINCIPLE OF USE: The results of a computation should be as expected.

EXAMPLES OF WHERE TO USE

Wrong:

```
unsigned A = 4 + 2 >> 1;
unsigned B = 2 + 1 << 1;
```

Correct:

```
unsigned A = 4 + (2 >> 1);
unsigned B = 2 + (1 << 1);
```

Correct:

```
unsigned A = (4 + 2) >> 1;
unsigned B = (2 + 1) << 1;
```

Note that this has a different result than the previous example of correct.

WHY & EXAMPLES OF EFFECTS. What are the computed values, for the C/C++ language, of A and B below?

```
unsigned A = 4 + 2 >> 1;
unsigned B = 2 + 1 << 1;
```

The answers are 3 and 6, respectively. Many programmers would expect 5 and 4. In other words, it is common to expect the shift operators to have more precedence than addition and subtraction, but less than multiplication and division.

64.3. BE CAREFUL OF UNEXPECTED INTEGER OVERFLOW

In the integer family of types in C (and C-like) language values can silently overflow, leaving you with a surprisingly small number (even a very negative one when you expect otherwise).

An example helps:

```
int X = (A-B) + (D - C);
```

is not always the same as:

```
int Y = (A + D) - (B + C);
```

The sum of A and D could be large enough to overflow the integer (or whatever) type. The same for the sum of B and C. But – and the likely reason that they were written as two subtractions before the addition – B might shrink A enough, and C might shrink D enough to not overflow.

Verification and integration tests rarely catch arithmetic overflows. Hence it often triggers only after the code deploys (‘ships’).

- Look for this in code reviews,
- Verify that there is are unit tests designed to check this
- Consider using a wider type

64.4. FLOATING POINT VALUES

Floating-point is not to be used where discrete values are needed.

Floating-point is not to be used in interrupt handlers, exception/fault handlers, or in the kernel.

RATIONALE: Many processors do not preserve the state of the floating-point unit on interrupt or exception; or the processor may have been configured to not save the state. (Saving the state can increase interrupt latency). Kernels– which are preferred to execute quickly – do not preserve the state of the floating-point unit on entry to kernel space. (They do preserve it on context switch.)

Regular review the compiler generated code to ensure that it has not employed floating points for intermediate calculation.

See the appendix C for the limits of float precision.

64.4.1 Comparison

Comparison of floating-point values should be treated with care, and reviewed for correctness.

- A float (or double) must not be used as a loop counter
- Exact comparisons (`==` and `!=`) shall not be used with float and double.
- Floating-point comparisons are not transitive: “`a <= b`” and “`b < a`” can both be false.

Math operations (especially division) of non-zero numbers can create “NaNs.” The software design should have a structured approach to checking for NANS, Infinites, and Out of range values.

64.4.2 Countable and Floating-point numbers are not associative nor distributive

Arithmetic operators in C are *not* distributive. The countable numbers (int, short, unsigned and signed, etc) preserve the *least significant digits* under arithmetic operation.

Floating point (floats and doubles) preserves the *most significant digits*, dropping the least. That is its major appeal – it prevents the problems you see with the integer family above. (Well, float point can overflow too). The following illustrates a bug from this:

```
float X = (A+B) + (D + C);
```

is not always the same as:

```
float Y = (A + D) + (B + C);
```

When A and B are small numbers, and C and D are big ones here is what happens. A plus D is D, because the digits of A are insignificant and dropped. And, similarly, the digits of B are

insignificant and are dropped. But, A plus B does some up the digits, enough so that they do add with D and C, giving a different result.

The answer, for simple cases, is to arrange the arithmetic operations from the smallest number to the biggest.

Linear algebra is very heavily used in signal processing and some control systems. In those systems, the matrix operations we learn as sophomores is very unstable. Matrices get ridiculous numbers doing, say, an eigenvector – that is, the computed results do not work very well, they can have not-a-number results ñ singularities and infinities and such. One way to prevent this is to permute the matrix before performing the operation, like that sort from smallest to largest, and the rearrange back to the proper order when done.

Numerical stability for these multiplies, divides, sums and differences, is a specialty. Check that the use of good libraries for math, has comprehensive unit tests, and use specialist reviews.

64.4.3 Standard Math functions

Many standard library math functions are slow (to provide all of the semantics specified in standards) and often are in terms of doubles.

Check to see if a fast, approximate version should be used.

65. CONTROL FLOW, AVOIDING COMPLEXITY

SUMMARY: Prefer techniques that simplify control flow structure. Complex control structures tend to be hard to maintain, evaluate for correctness, and more likely to have bugs.

A program or task may be complex, but no subroutine (or method) may be complex.

Forward Progress. The software should always include a path that moves the execution forward. Infinite loops are not allowed.

Progress to a known and controlled state. The software should always include a path that moves the execution to a known and controlled state.

65.1. BLOCK BODY

The flow control primitives if, else, while, for and do should be followed by a block, even if it is an empty block. For example:

Wrong:

```
while( /* do something */ )  
;
```

Correct:

```
while( /* do something */ )  
{  
}
```

The block following a flow control primitive should always be bounded by brackets even if the block contains only one statement. For example:

Wrong:

```
if (isOpen())  
  foobar();
```

Correct:

```
if (isOpenned())
{
    foobar();
}
```

65.2. COMMA OPERATORS

Do not use the comma operator. (Exceptions may be made for very restricted use cases, and must be reviewed.)

65.3. CONDITIONS

Do not nest if-then statements more than 2-levels.

Only use employ *essentially boolean* types and expressions conditions. Note: integer, pointer, and other value types are *not* essentially boolean.

Do not nest “switch” blocks.

65.4. LOOPS

Things to avoid with loops (as they create complete control flow)

- Do not nest “for” loops more than 2-levels.
- Too many ‘continue’ or ‘break’ statements in for loop

65.4.1 Loop conditions

Where possible, the loop conditions other than index variable should be *const* variables.

Wrong:

```
for (Idx = 0; Idx < length-2; Idx++)
{
    ...
}
```

Correct:

```
int const End = length-2;
for (Idx = 0; Idx < End; Idx++)
{
    ...
}
```

RATIONALE. This creates smaller, faster code, which uses fewer memory accesses and reduces power consumption (in lower power designs).

The compiler may reload (and recalculate) the variables used in the comparison, even though they have not changed. The compiler has to be conservative and assume that the block (somehow) may affect the value, and so it must reload the variables with each comparison. This is more likely with more complex blocks that are harder for the compiler to analyze. The exception is if it can prove (via aggressive analysis) that the block will not modify it.

65.5. EARLY RETURNS

A procedure should return errors early.

RATIONALE. It is better to have a clear procedure that returns early, rather than to muddle the procedure with nesting, convoluted control flow and temporary variables.

65.6. NO RECURSION / CALL LOOPS

Recursion – direct or indirect – is not allowed.

65.7. GOTOS

The `goto` statement is not recommended. However there may be instances where the use of a `goto` statement may actually make the source code more understandable and robust. The software engineer must document the use of the `goto` in the source code and be prepared to defend this choice in software source code reviews.

65.8. STATIC ASSERT

Use static assert to check the sizes of types, structures.

65.9. ASSERTS

`ASSERT()` (and similar) shall only be used to in conjunction with unit and integration testing, to raise an error that fundamental calling assumptions have been violated.

Do not use side effects in the arguments to these.

The code is expected to employ checks of parameters, internal state and results. If there is an error, clean up, log a trace point or raise a software breakpoint, and return an appropriate error.

RATIONALE. The default behaviour of an assert is to crash the system, rather than handling the error. In a production build, that should be done only for catastrophic errors. The asserts for test case support should disabled (“compiled out”).

COMMENT. Many 3rd party modules employ `ASSERT()` like functionality. These modules should be assessed whether `ASSERT()` is employed and provide a plan to handle when an assert is raised.

66. PROHIBITED PROCEDURES

There are several procedures, that are more trouble than they are worth. These may be hard (or impractical) to make function correctly, better alternatives exist, and so on. To help flag them, may use git’s `#define` system:

<https://github.com/git/git/blob/master/banned.h>

66.1. PROCEDURES NOT TO USE

String procedures. Do not use `scanf()`, `sprintf()`, `strcat()`, `strncat()`, `strcpy()`, `strncpy()`, `fprintf()`, `printf()`, `sprintf()`, `snprintf()`

If a formatted string procedure is needed, do not use `snprintf()` (and similar). This procedure does not return the count of bytes used in the buffer – it returns the count it would have used under ideal circumstances.

File system procedures: `fgets()`, `getcwd()`, `gets()`

66.2. ALTERNATIVE PROCEDURES

To reduce chances of buffer overflows and other characteristic bugs, Annex K of the C standard (from TR24731) provides an alternate API. They are supported in several tool chains, but many do not support it.

The procedures tend to have an ‘_s’ suffix and require the sizes of both buffers involved. For instance:

- Copying block of memory, use `memmove_s()`
- To check the length of string use `strnlen_s()`

When these procedures present (and the functionality is needed) use these procedures. For portability include shims that bridge to the more widely available ones as needed.

If the length of the string is known, do not use `strcat()`, `strcpy()` or similar procedures. Use the memory copy procedures instead.

67. MICROCONTROLLER SPECIFIC GUIDELINES

Be aware of whether the processor can support floats. On processors that support only floats, check for double promotion. Expressions, and vaargs can promote float operations to double; some procedures (such as `atof()`) only return double. This pulls in emulation, which is slow and can consume code space. Check the linker report and map file to confirm that emulation code has not been pulled in.

Do not use floats on ARM Cortex-M0 and Cortex-M3. These processors do not support floating point (floats and doubles). When used, they are emulated in software, which is slow.

67.1. USE OF RATIONAL NUMBER FORMS

Many microcontrollers lack hardware floating point. Use rational numbers on those.

67.2. USE OF MULTIPLICATION AND DIVISION

Many microcontrollers (PIC, Cortex-M0) do not include a division or (in some cases) a multiplication. The compilers are quite good, especially if only one of the terms in the multiplication is a variable. Under some circumstances, the compiler is also able to analyze the code and translate a formula of two or more variables into a small set of formulas of a single independent variable. It is best to for the programmer to do this manually.

When the above technique cannot be applied, and the variables can be large in value, it may be better to convert the value to a logarithmic form, do the operation as arithmetic, and exponentiate the value back.

67.3. HOW AND WHEN TO USE ASSEMBLY

Using assembly is inherently processor specific, so it should only be created in important blocks. The assembly must be rigorously tested against a set of known values at critical points. The blocks that use these optimizations must be similarly tested.

68. REFERENCES AND RESOURCES

- Barr, Michael, *How to use the volatile keyword*
<http://www.barrgroup.com/Embedded-Systems/How-To/C-Volatile-Keyword>
- Barr, Michael, *Coding standard rule for use of volatile*
<http://embeddedgurus.com/barr-code/2009/03/coding-standard-rule-4-use-volatile-whenever-possible/>
- Boswell, Dustin; Trevor Foucher, “*The Art of Readable Code*,” O’Reilly Media, Inc. 2012
- Ellemtel Telecommunication Systems Laboratories, “*Programming in C++ Rules and Recommendations*”, Document: M 90 0118 Uen, Rev. C, 1992-April 27
<http://www.doc.ic.ac.uk/lab/cplusplus.rules/>
- Exida Consulting, “*C/C++ Coding Standard Recommendations for IEC 61508*” V1 R2 2011 Feb 23, http://exida.com/images/uploads/exida_C_C++_Coding_Standard_-_IEC61508.pdf
- Hollub, Allen
<https://holub.com/goodies/rules.html>
- Labrosse, Jean *MicroC/OS-II: The Real-Time Kernel*, 2nd Ed, CMP Books, 2002
This includes a chapter on the coding style guide employed in the RTOS.
- Lockheed Martin Corporation, “*Joint Strike Fighter, Air Vehicle, C++ Coding Standards*”, Document: 2RDU000001 Rev. C, 2005 December
- Microsoft, *Secure Coding Guidelines*
<https://docs.microsoft.com/en-us/dotnet/standard/security/secure-coding-guidelines>
- MISRA Consortium Limited, “*MISRA C++:2023, Guidelines for the use of the C++:17 in critical systems*” 2023
- MISRA Consortium Limited, “*MISRA C:2023, Guidelines for the use of the C language in critical systems*” 2023, 3rd ed
An excellent, well-written coding standard. Strongly recommended.
- NASA; Steven Hughes, Linda Jun, Wendy Shoen, “*C++ Coding Standards and Style Guide*”, 2005
<https://ntrs.nasa.gov/search.jsp?R=20080039927>
- Oshana, Robert and Mark Kraeling Newnes, “*Software Engineering for Embedded Systems: Methods, Practical Techniques, and Applications*,” 2013
Appendix 1 contains a useful coding style guide
- Ousterhout, John A *Philosophy of Software Design*, 2nd ed, 2022
This has an excellent discussion on techniques to improve commenting, naming, and structuring the design.
- Sanfilippo, Salvatore, *Writing system software: code comments*. <https://antirez.com/news/124>
An interesting classification of different kinds of comments and their roles.
- Seebach, Peter “Everything you ever wanted to know about C types” 2006 Seebach, Peter
“*Everything you ever wanted to know about C types*” 2006
Part 1: <http://www.ibm.com/developerworks/library/pa-ctypes1/>
Part 2: <http://www.ibm.com/developerworks/power/library/pa-ctypes2/index.html>
Part 3: <http://www.ibm.com/developerworks/power/library/pa-ctypes3/index.html>
Part 4: <http://www.ibm.com/developerworks/power/library/pa-ctypes4/index.html>
- SEI CERT, *C Coding Standard*
<https://wiki.sei.cmu.edu/confluence/display/c/SEI+CERT+C+Coding+Standard>
- Turner, Jason; *C++ best practices*
https://github.com/lefticus/cppbestpractices/blob/master/00-Table_of_Contents.md

“The compiler doesn’t know whether you know what’s good for you.”
– Raymond Chen

CHAPTER 18

Code Inspections and Reviews

Source code should be checked for good workmanship, thru inspections & reviews:

- When a review should occur
- Who should review
- How to inspect and review
- What to report, outcomes

Note: There is no universally accepted and adopted approach to peer review. Each environment has its own norms for peer review. These are checklists and templates that I have constructed over years, having found little available elsewhere.

The best reviews are modified Fagen-style reviews. *Peer Reviews in Software* [Wiegers 2001] is the best resource that I have found.

69. WHEN TO REVIEW

A review might occur when

- There are proposed changes to a stable codebase,
- When closing out a bug
- When a project reaches a control gate

70. WHO SHOULD REVIEW

What kind of person should participate in a review?

- The reviewers should have experience with the class of hardware being used. In typical embedded development today, they should be experienced with 32-bit embedded software, Cortex-M and similar microcontrollers.
- In some cases, the reviews will require someone with experience in the microcontroller.
- Reviewers should have a lot experience with the way software, microcontrollers, and hardware can go wrong.
- Some reviewers should be independent; they should not be working on this artifact.
- The owner of the subsystem or other area of code

Others may participate in a review, of course. This includes:

- The author need not attend, as the code should stand on its own
- People being brought into the team
- People with little experience in this area of engineering.

The later are not expected to contribute specific technical comments, but they may learn the system, the performance of reviews, and provide feedback on the understandability & maintainability of this foreign code.

71. HOW TO INSPECT AND REVIEW CODE

How can reviews be performed? One may apply any of the well-documented review and inspection techniques that can be found in the references. Common review methods are:

- The reviewers can meet and perform a formal inspection: e.g. with presentation, roles, and sign-offs.
- Some small reviews can be reviewed at each person's desk. Code annotation tools such as CodeCollaborator¹², and Github review tools are often used.

This applies to general reviews, as well as specialized inspections.

- General reviews emphasize the workmanship of the code – maintainability (is it clear enough for others to work on in the future), basic quality-of-construction, etc.
- Specialized inspections are used to focus attention on specific areas that may be esoteric or require specialized technical skill to judge.

The reviews should be provided (in addition to the code):

- Style and other workmanship guides,
- Evaluation guides and rubrics
- The top level and detailed designs
- Supporting data sheets, application notes, vendor documentation

Note: in some institutions, the unit and integration tests and reports are also provided. It isn't practical review these; only to check them at high level, and a few key test cases.

The reviewers should be provided a summary of areas to look at. The reviewers examine these areas (and inputs), looking defects, or constructions that can be difficult to maintain.

71.1. SPECIALIZED INSPECTIONS

Specialized inspections are used to focus attention and effort. These delve into key areas and slices of code to answer narrow questions. Typical questions may be:

- Is the processor set up properly – are the clocks / oscillators turned on properly, etc?
- Are the watchdog timers (or similar protective timers) set up properly and detect enough unresponsiveness in the code?
- Is the source code specific to the microcontroller / hardware implemented correctly?

¹² <https://smartbear.com/product/collaborator/overview/>

- Do the critical and supervisory sections of software perform only their intended functions (and not other functions) and do not result in a risk?
- Is there consistency in the data and control flows across interfaces?
- Is it correct and complete with respect to the safety requirements?

See also

- Appendix I for the *Code Complete* Code Review check lists,
- Appendix J for a rubric to apply in the reviews
- Appendix E for Bug classification

72. THE OUTCOMES OF A CODE REVIEW

Reviewers comment on the aspect of the code quality:

- Detailed design
- Functionality
- Complexity
- Testing
- Naming
- Comment Quality
- Coding style
- Maintainability
- Understanding/comprehension.

The results of a review ideally should:

- Be actionable and easy to fix
- Produce few false positives
- Focus on where there can be improvements with significant impact on code quality.

The results of a review might be realized one or more of the following ways:

- Gathering the results in a document (or spreadsheet) in a tabular fashion
- Annotate the source code, e.g. using a tool such as Code Collaborator
- Fill out bug reports
- Provide written feedback

72.1. A TIP ON FEEDBACK

When you are providing feedback, consider:

- Should it be said? Is the comment necessary, kind, true and helpful?
- Does it have the right emphasis? The emphasis, especially critiques, should be proportionate. Scale using a rubric; some are included in Appendix H and Appendix J.
- How should the comment be said? Specific, actionable, measureable or distinct (that it has an effect when performed; can tell that it was done), timely (can be done immediately, or has time bounds)

- What is the person try to accomplish? {with the thing they are getting feedback on?}

72.2. REWORK CODE AFTER A REVIEW

The rework, in most cases, can be done by a second person or the primary developer.

73. REFERENCES AND RESOURCES

Cohen, Jason, *Best Kept Secrets of Peer Code Review: Modern Approach. Practical Advice*, 2006

Includes discussion of “online” reviewing of software changes and new code, appropriate for merge requests.

IEEE Std 1028-2008 - *IEEE Standard for Software Reviews and Audits*

The standard provides minimum acceptable requirements for systematic reviews

Wiegers, Karl *Peer Reviews in Software: A Practical Guide* 2001, Addison-Wesley Professional

This is the best book on software reviews.

CHAPTER 19

Code Inspection & Reviews Checklists

This chapter summarizes the code review checklists

- The types of reviews to perform checklist
- Basic review checklist
- Specialized Review checklists

74. REVIEWS

These are the kinds of reviews to perform

- ☐ Basic reviews
- ☐ Microcontroller / Hardware Initialization review
- ☐ Error returns review
- ☐ Fault handling review
- ☐ Memory/Storage handling review
- ☐ Prioritization review
- ☐ Concurrency review
- ☐ Critical function / Supervisor review
- ☐ Low power mode review
- ☐ Numerical processing review
- ☐ Signal processing review
- ☐ Timing review

75. BASIC REVIEW CHECKLIST

Before a review proceeds:

- ☐ Code has clean-result when checked with analysis tools – MISRA C rules, clang-format, clang-tidy, compiling with extensive warning checks enabled, etc.
- ☐ There is design documentation, and it has been reviewed.
- ☐ There are unit and integration tests.

See also

- Appendix I for the *Code Complete* Code Review check lists

- Appendix J for a rubric to apply in the reviews

75.1. BASIC STYLE

Layout checks:

- ☐ Commenting: there are comments at the top of the file, the start of each function, and with all the code that needs an explanation.
- ☐ Does the source code conform to the coding style guidelines & other conventions? These cover location of braces, variable and function names, line length, indentations, formatting, and comments.
- ☐ Code naming, indentation, and other style elements are applied consistently (esp in areas beyond the style guidelines)

Names:

- ☐ Are the file names well chosen?
- ☐ Are the files in the correct location in the file tree? In the repository?
- ☐ Are the names – for variables, files, procedures, and other objects – clear and well chosen? Do the names convey their intent? Are they relevant to their functionality?
- ☐ Is a good group / naming convention used? Related items should be grouped by name
- ☐ Is the name format consistent?
- ☐ The names only employ alphanumeric and underscore characters?
- ☐ Do the names provide the relevant units (and scale)?
- ☐ Are there typos in the names?
- ☐ Numbers are not given stupid names like ZERO or ONE, etc?

Values and operators:

- ☐ Parentheses used to avoid operator precedence confusion?
- ☐ Are `const` and `inline` used instead of `#define`?
- ☐ Is conditional compilation avoided? Can it be reduced?
- ☐ Avoid use of *magic* numbers (constant values embedded in code) – and basic numbers are *not* given trite names such as ZERO, or ONE?
- ☐ Use strong typing (including sized types, structs for coupled data, `const`)?
- ☐ Use the proper types for quantities and pointers?

Control flow checks:

- ☐ Are all inputs checked for the correct type, length, format, range?
- ☐ Are invalid parameter values handled?
- ☐ Are variables initialized at definition?
- ☐ Are output values checked and defined?
- ☐ Are NULL pointers, empty strings, other boundary conditions (for results) handled?

75.2. BASIC FUNCTIONALITY

- ☐ Does the code match the detailed design (correct functionality)?

- ☐ All documented functionality is implemented? All implemented functionality is documented?
- ☐ Does the code work? Does it perform its intended function? Is the logic correct? etc.
- ☐ Is the update/check of state correct? Any incorrect updates or checks?
- ☐ Is the wrong algorithm/assumption/implementation used?
- ☐ Is the work performed in the correct order?
- ☐ Check that proper types are employed

75.3. SCOPING

- ☐ Proper modularity, module size, use of .h files and #includes
- ☐ Is the code as modular as possible?
- ☐ Minimum scope for all functions and variables; e.g. few globals?
- ☐ Can any global variables be replaced?
- ☐ Are there unused or redundant variables? Macros?
- ☐ Do the variables have an appropriate storage class (and scope) – static, extern, stack?
- ☐ The register storage class is not used?

75.4. CONTROL FLOW

- ☐ There is forward progression: loops are bounded, delays are bounded, etc.
- ☐ Do loops have a set length and correct termination conditions?
- ☐ Loop entry and exit conditions correct; minimum continue/break complexity
- ☐ Conditionals should be minimally nested (generally only one or two deep)
- ☐ Conditional expressions evaluate to a boolean value
- ☐ Conditional expressions do not assignments, or side-effects
- ☐ All switch statements have a default clause, with error return
- ☐ Do the work events/messages get submitted backwards in the IO queue network? Is there a potential infinite work loop?

75.5. DOCUMENTATION

- ☐ Are all procedures/functions/variables/etc commented?
- ☐ Do they properly describe the intent of the code?
- ☐ Is any unusual behavior or edge-case handling described?
- ☐ Are all parameters of the procedure are documented?
- ☐ Are the ranges and constraints of the parameters documented?
- ☐ Are the return value(s) documented?
- ☐ Are all of the error conditions/returns documented?
- ☐ Is the use and function of third-party libraries documented?
- ☐ Are data structures and units of measurement explained?
- ☐ Is there any incomplete code? If so, should it be removed or flagged with a suitable marker like 'TODO'?

75.6. MAINTAINABILITY AND UNDERSTANDABILITY

- ☐ Is all the code easily understood? Is the code simple, obvious, and easy to review?
- ☐ Is the code unnecessarily, ornate or complex? Are there more intermediate variables than necessary? Is the control flow overly complex? (Look for variables that hold the return value far from the return)
- ☐ Code complexity measure is low (below set threshold)?
- ☐ Is there any redundant or duplicate code?
- ☐ Is there any dead or commented out code?
- ☐ Can any of the code be replaced with library or built-in functions?
- ☐ Any changes that would improve readability, simplify structure, and utilize cleaner models?
- ☐ Does the code have too many dependencies?

75.7. TESTABILITY

- ☐ Is the code testable?
- ☐ Is the implementation suitable to be unit tested?
- ☐ Are the procedures small enough?
- ☐ Is the control flow too complex?
- ☐ Does it take too many external inputs – parameters, globals (including private variable), other calls, complex state, etc.?
- ☐ Are there unit tests? Do they cover key cases for the functions?
- ☐ Are there integration tests?

75.8. PERFORMANCE

- ☐ Are there obvious optimizations that will improve performance?
- ☐ Can any of the code be replaced with library functions built for performance?

Performance changes to improve the implementations:

- ☐ Can the data access be improved? E.g. caching and work avoidance.
- ☐ Can the I/O scheduling be improved? E.g. batching of writes, opportunistic read ahead and avoiding unnecessary synchronous I/O.
- ☐ Are there better / faster data structures for in-memory and secondary storage?
- ☐ Are there other performance improve techniques that can be applied?

Synchronization-based performance improvements:

- ☐ Are the synchronization methods inefficient?
- ☐ Can a pair of unnecessary locks be removed?
- ☐ Can compare-and-swap or other “lock free” atomic procedures be employed?
- ☐ Can finer-grained locking be employed?
- ☐ Can write locks be replaced with read/write locks?

75.9. OTHER

- ☐ Can any logging or debugging code be removed?
- ☐ Check that prohibited procedures and other constructs are not used.
- ☐ Are there regular checks of operating conditions?
- ☐ Data structure ordering is efficient for access pattern? Alignment and padding will not be an issue?
- ☐ Do the variables have the appropriate qualifiers? `volatile`? `const`?

76. SPECIALIZED REVIEW CHECKLISTS

This section provides checklists for specialized, focused reviews:

- Microcontroller / Hardware Initialization review
- Error returns review
- Fault handling review
- Memory/Storage handling review
- Prioritization review
- Concurrency review
- Critical function / Supervisor review
- Low power mode review
- Numerical processing review
- Signal processing review
- Timing review

Note: these can be used in conjunction with the detailed design review checklists. If the detailed design review covered these, the review is much faster; often there is no detailed design review.

See also

- Chapter 14 Design review check lists
- Appendix I for the *Code Complete* Code Review check lists
- Appendix J for a rubric to apply in the reviews

76.1. MICROCONTROLLER / HARDWARE INITIALIZATION REVIEW CHECKLIST

Looks for bugs in the initialization and configuration of the hardware:

- ☐ Check the initialization order
- ☐ Are the clocks set correctly? i.e., no over-clocking at the voltage and/or temperature
- ☐ Does the code handle oscillator (or clock) startup failures?
- ☐ Does the code check the initial clock rate? Properly?
- ☐ Check that the source clock, prescaler, divisor, and PLL configuration are setup correctly.
- ☐ Check the peripherals are configured and enabled properly
- ☐ Is the software using the right bus for the peripheral?
- ☐ Check that the proper clock source is enabled for the peripheral.

- ☐ Check that the peripheral is not over-clocked for the power source and temperature range. (Some peripherals have tighter constraints)
- ☐ Check that the correct power source / enable is used in setting up the peripheral
- ☐ DMA channel assignments match hardware function constraints
- ☐ GPIO mode, direction (in/out), biasing (pull-ups, pull-downs) are configured correctly.
- ☐ Power supervisor / brown-out detect is configured properly.
- ☐ Lock bits are set on peripherals – GPIO, timer, etc.
- ☐ The microcontroller's errata has read and applied?

76.2. ERROR RESULTS REVIEW CHECKLIST

A lack of checking results, or incorrectly handling the results, is a frequent source of critical failures. Look for bugs in the handling (or lack thereof) of return values and error results:

- ☐ Check that NULL pointers, empty strings, other result boundary conditions are handled
- ☐ Are the return values/cases defined? Are the error returns documented?
- ☐ Error handling for function return is appropriate
- ☐ Does it check the correct (or wrong) set of error codes?
- ☐ Is there missing or incorrect error code handling?
- ☐ Where third-party utilities are used, are returning errors being caught?

76.3. FAULT HANDLING (WITHIN PROCEDURES) REVIEW CHECKLIST

Defects in fault handling are a common source of critical failures. Review the failure paths.

- ☐ Check that the semantics for the failure are handled correctly. Is metadata updated properly? Are the resources freed?
- ☐ Check that allocated resources are release
- ☐ Check that the locks/semaphores/mutexes are released correctly
- ☐ Look for null pointer dereferences, and code that incorrectly assume the pointers are still valid after failure
- ☐ Check that it returns correct error code – i.e. not the wrong error code

76.4. MEMORY HANDLING REVIEW CHECKLIST

Has the memory been partitioned in a manner suitable for Class B? i.e., does the software isolate and check the regions?

- ☐ Are there potential buffer overflows?
- ☐ Are there good practices to prevent buffer overflows – bound checking, avoid unsafe string operations?
- ☐ Dereferences of free'd memory
- ☐ Dereferences of NULL pointer
- ☐ Dereferences of undefined pointer value
- ☐ Incorrect handling of memory objects
- ☐ Didn't release memory / resource

- ☐ Free'd memory resource twice
- ☐ Parity checking is enabled
- ☐ Redundant memory is segregated and stored in a different format
- ☐ Check that the data access will be performant; that a slow approach is not employed unnecessarily
- ☐ Memory pages write protected
- ☐ Memory protection unit is enabled? Access control is configured properly?
- ☐ Are pointers cast to non-pointer types? Are non-pointer types cast to pointers? Are they the same size?

Non-volatile storage:

- ☐ Doesn't overwrite or erase the non-volatile data in use
- ☐ Doesn't use a "replacement" strategy of writing the most recent/highest good-copy of the data.
- ☐ Accounts for loss of power, reset, timeout, etc during read/write operation
- ☐ Checks supply voltage before erasing/writing non-volatile memory
- ☐ Performs read back after write
- ☐ Checks that software detects bit-flip and other loss of data integrity (e.g. employs CRC)
- ☐ Check that data recovery methods will work, if employed
- ☐ Check that the correct version of stored data will be employed (such as on restart)
- ☐ Interrupts and exceptions are disabled during program memory is modified.
- ☐ Cache/instruction pipeline is flushed (as appropriate) after program memory modification.
- ☐ Check that the data access will be performant; that an slow approach is not employed unnecessarily
- ☐ Check that the data access will not interfere with the other timing.

76.5. PRIORITIZATION REVIEW CHECKLIST

- ☐ Rate Monotonic Analysis (RMA) and dead-line analysis has been performed?
- ☐ Task/thread prioritization is based on the analysis?
- ☐ Mutex prioritization is based on the analysis?
- ☐ Events, Messages and IO queue prioritization are based on the analysis?
- ☐ Interrupt prioritization is based on the analysis
- ☐ DMA channel prioritization is based on the analysis
- ☐ CAN message priorities are based on the analysis
- ☐ ADC priorities are based on the analysis
- ☐ Bluetooth LE notification/indication priorities are based on the analysis

76.6. CONCURRENCY REVIEW CHECKLIST

- ☐ Are there any missing mutex, locks, or IPC mechanisms?
- ☐ Check acquisition order of locks/semaphores/mutexes – is the order wrong or potential for dead locks?
- ☐ Check for violations of access atomicity: not using `atomic<>`, missing `volatile` keyword, assuming read/write is atomic when it is not, missing write barriers, etc.

- ☐ Check for read-modify-write race conditions.
- ☐ Check order of multiple accesses
- ☐ Check for missing release of lock/semaphore/mutex
- ☐ Mutexes are unlocked in the same procedure that they are acquired (locked).
- ☐ Check for unlocking locks, posting semaphores or mutexes multiple times – not just at runtime, but that there is only one place that they are unlocked or posted.
- ☐ Mutexes are unconditionally unlocked – they are not in “if” or other condition.
- ☐ Look for forgotten release of locks/semaphores/mutexes
- ☐ Are there ways to reduce the blocking time?
- ☐ Are there ways to reduce disabling interrupts?
- ☐ Are non-thread-safe (non-reentrant) procedures or structures used?

76.7. CRITICAL FUNCTION / SUPERVISOR REVIEW CHECKLIST

Check that critical functions (e.g. Class B and C of 60730) are suitably crafted:

- ☐ Is the code for the critical functions limited to a small number of software modules?
- ☐ Is the code for the critical functions small?
- ☐ Is the code complexity low? Are there no branches – or only simple branches?
- ☐ Are the possible paths thru the critical function code small, and simple?
- ☐ Is the relation between the input and output parameters simple? Or at least, simple as possible?
- ☐ Are complex calculations used? They should not be. Especially as the basis of control flow, such as branches and loops.
- ☐ Power supervisor / brown-out detect is configured properly.
- ☐ Checks the clock functionality and rates
- ☐ Watchdog timer is employed (and correctly)
- ☐ Is the watchdog reset only after all protected software elements are shown to be live? Example bad design: resetting the watchdog in the idle loop, or every time thru a run loop.
- ☐ Check that the watchdog timer is not disabled anywhere in the code
- ☐ Is the external watchdog handshake done only after all the software items under protection have been checked for liveliness? A bad approach is to use a PWM for the handshake, as a PWM can continue while software has locked up or is held in reset.
- ☐ Handles interrupt overload conditions
- ☐ Critical program memory is protected from writes. How: Hardware level? Software?
- ☐ Program memory CRC check.
- ☐ Stack overflowing checking
- ☐ Critical data is separated, checked, protected.
- ☐ Cross checks values
- ☐ Performs read backs of sent values
- ☐ Independent checks / reciprocal comparisons to verify that data was exchanged correctly.
- ☐ Periodic self-tests or functional tests
- ☐ Are there possible partition violations from data handling errors, control errors, timing errors, or other misuse of resources?

- ☐ That the software can meet the scheduling requirements, and the timing constraints specified.
- ☐ Do the fail-safe and fail-operational procedures bring the product to the defined acceptable state?

76.8. LOW POWER MODE REVIEW CHECKLIST

Power configuration for low power modes:

- ☐ Does it switch to low clock source(s) and disable the others?
- ☐ Are the IOs set to a low direction, mode (e.g. analog in?) and bias (e.g. pull-down, pull-up)?
- ☐ Are peripherals disabled where they can be?
- ☐ Are peripheral clocks disabled where they can be?
- ☐ Are the proper flushes, barriers, etc. executed before going into a sleep state?
- ☐ Is the proper low-power instruction used?
- ☐ Is there a race condition in going into low-power state and not being able to sleep or wake?
- ☐ Check coming out of low power mode restores the operating state

76.9. NUMERICAL PROCESSING REVIEW CHECKLIST

Check for correct arithmetic, and other numerical operations:

- ☐ Check that division by zero, other boundary conditions are handled
- ☐ Is the FPU configured properly? For example, is lazy context save of floating-point state (LPSEN) disabled on ARM Cortex-M4s?
- ☐ Floating point is not used in interrupts, exception handlers, or the kernel
- ☐ Check that floating point equality is used properly – i.e., something other than ==. Does it handle denormals, non-zeros, NaNs, INFS and so on?
- ☐ Are the equations ill-conditioned?
- ☐ Is the method of calculation slow?
- ☐ Check that denormals, NaNs, INFs, truncation, round off that may result from calculations are properly handled.
- ☐ Are the use of rounding and truncation proper?
- ☐ Would use of fixed point be more appropriate?
- ☐ Is simple summation or Euler integration employed? This is most certainly lower quality than employing Simpsons rule, or Runge-Kutta.

76.10. SIGNAL PROCESSING REVIEW CHECKLIST

- ☐ Are the ADCs over-clocked for the signal chain? Check that the sample time and input impedance are aligned.
- ☐ Is the sample time sufficient to measure the signal?
- ☐ Is there a potential time variation (e.g. jitter) in the sampling? The code should be implemented for low jitter. For instance, a design that uses a DMA ring-buffer has low variation, while run-loop or interrupt trigger can have a great deal of time variation.
- ☐ Is oversampling applied? Is the oversampling done in a proper way?

- ☐ Is simple summation or Euler integration employed? This is most certainly lower quality than employing Simpsons rule, or Runge-Kutta.
- ☐ Is the proper form of the filter used? Is an unstable form used?
- ☐ Does it have ringing, feedback, self-induced oscillation or other noise?
- ☐ Does handle potential saturation, overflows?
- ☐ Efficient, fast implementation?
- ☐ Is there good instruction locality on the kernel(s)?
- ☐ Is there good data locality on the kernel(s)?
- ☐ Is the signal processing unnecessarily complex?
- ☐ Check the step response of the signal processing

76.11. TIMING REVIEW CHECKLIST

- ☐ Does the timing meet the documented design and requirements?
- ☐ Are there possible timing violations?
- ☐ Are there race conditions?
- ☐ Is enough time given to let a signal/action/etc propagate before the next step is taken?
- ☐ Is there a potential for hidden delays (e.g. interrupt, task switch) that would violate the timing?
- ☐ From the time the trigger is made to the action, what worst case round-trip? Include interrupts, task switching, interrupts being disabled, etc. Is this timing acceptable?
- ☐ The length of operations, in the worst case, does not cause servicing the watchdog timer to be missed?

Appendices

- ABBREVIATIONS, ACRONYMS, & GLOSSARY. This appendix provides a gloss of terms, abbreviations, and acronyms.
- PRODUCT STANDARDS. This appendix provides supplemental information on standards and how product standards are organized
- FLOATING POINT PRECISION. This appendix recaps the limits of floating-point precision.
- BUG REPORTING TEMPLATE. A template (and guidelines) for reporting bugs
- TYPES OF DEFECTS. This appendix provides a classification of different kinds of software defects that are typically encountered.
- CODE COMPLETE REQUIREMENTS REVIEW CHECKLISTS. This appendix reproduces checklists from *Code Complete, 2nd Ed* that are relevant to requirements reviews.
- CODE COMPLETE DESIGN REVIEW CHECKLISTS. This appendix reproduces checklists from *Code Complete, 2nd Ed* that are relevant to design reviews.
- DESIGN REVIEW RUBRIC. This appendix provides rubrics relevant in assessing the design and its documentation.
- CODE COMPLETE CODE REVIEW CHECKLISTS. This appendix reproduces checklists from *Code Complete, 2nd Ed* that are relevant to code reviews.
- SOFTWARE REVIEW RUBRIC. This appendix provides rubrics relevant in assessing software workmanship.
- ARM CORTEX-M SPECIFICS. Technical tips and design information too low-level for a detailed design document.
- HARDWARE-FIRMWARE INTEGRATION TESTS.

[This page is intentionally left blank for purposes of double-sided printing]

APPENDIX A

Abbreviations, Acronyms, Glossary

Abbreviation / Acronym	Phrase
ADC	analog to digital converter
ANSI	American National Standards Institute
ARM	Advanced RISC Machines
BNF	Backus-Naur Form
BSP	board support package
API	application programming interface.
CAN	controller-area network
CRC	cyclic redundancy check
DAC	digital to analog converter
DMA	direct memory access
EN	European Norms
GPIO	general purpose IO
Hz	Hertz; 1 cycle/second
I ² C	inter-IC communication; a type of serial interface
IEC	International Electrotechnical Commission
IPC	interprocess communication
IRQ	Interrupt request
ISO	International Organization for Standardization
ISR	Interrupt service routine
JTAG	Joint Test Action Group
MCU	microcontroller (unit)
MPU	memory protection unit
NMI	non-maskable interrupt
NVIC	nested vector interrupt controller
NVRAM	non-volatile RAM
PWM	pulse width modulator

Table 43: Common
acronyms and
abbreviations

QMS	quality management system
RAM	random access memory; aka data memory
RISC	reduced instruction set computer
RTOS	real time operating system
SDK	software development kit
SDLC	software development lifecycle
SPI	serial peripheral interface
SRAM	static RAM
SWD	single wire debug
TBD	to be determined
TMR	timer
UART	universal asynchronous receiver/transmitter
WDT	watchdog timer

Phrase	Description
abnormal operating condition	A condition when an operating variable has a value outside of its normal operating limits. ¹³ See also <i>fault</i> , <i>normal operating condition</i> .
allowed operating condition	A condition when each of the operating variables (flow, pressure, temperature, voltage, etc.) has a value within of its respective normal operating limits, and so the “system will satisfy a set of operational requirements” [IEC 62845 3.10]. See also <i>abnormal operating condition</i> , <i>fault</i> .
analog to digital converter	An analog to digital converter measures a voltage signal, producing a digital value.
application logic	Application logic is a set of rules (implemented in software, or hardware) that are specific to the product.
Backus-Naur form	A notation used to describe the admissible calling sequences for an interface. Traditionally this form is used to define the syntax of a language.
bitband	An ARM Cortex-M mechanism that allows a pointer to a bit.
black-box testing	Testing technique focusing on testing functional requirements (and other specifications) with no examination of the internal structure or workings of the item.
board support package	The specification to an RTOS and/or Compiler of what peripherals the MCU has internally and is directly connected to.
certification	A “procedure by which a third party gives written assurance that a product, process or service conforms to specified requirements, also known as conformity assessment” [IEC 61400-22 3.4] longer description at [IEC 61836 3.7.6]
coding style guide	“specif[ies] good programming practice, proscribe unsafe language features (for example, undefined language features, unstructured designs, etc.), promote code understandability, facilitate verification and testing, and specify procedures for source code documentation.” [IEC 61508-3 7.4.4.13] aka <i>coding standard</i>
coefficient	A measure of a property for a process or body. This number is constant under specified, fixed conditions.

Table 44: Glossary of common terms and phrases

¹³ Modified from <http://www.wartsila.com/encyclopedia/term/abnormal-condition>

comment	Text, usually to provide context, clarify or explain the requirement(s).
control function	“functions intended to regulate the behaviour of equipment or systems” [IEC 61892-2 3.9], it typically “evaluates input information or signals and produces output information or activities” [IEC 62061 H.3.2.14] see also <i>safety-related control function</i>
control function (class B)	Those “control functions intended to prevent an unsafe state of the appliance... Failure of the control function will not lead directly to a hazardous situation” [IEC 60730-1:2013 H.2.22.2]
customer requirement	A requirement in any of the top-level documents, but especially in the customer (or user) requirements specification.
cyclic redundancy check	A form of error-detecting code. A check value is computed from a block of data.
data integrity	That the stored data – such as program memory – is intact, unchanged, in the expected order and complete; that is, that the entire program memory area matches <i>exactly</i> with the data defined for a particular revision.
data retention	The ability for a storage to hold bits
debounce	Switches and contacts tend to generate multiple rising & falling edges when coming into contact; debouncing removes the extra signals.
diagnostic	A “process by which hardware malfunctions may be detected” [IEEE 2000]
defect	An “imperfection in the state of an item (or inherent weakness) which can result in one or more failures of the item itself, or of another item under the specific service or environmental or maintenance conditions, for a stated period of time” [IEC 62271-1 3.1.16]
design document	A family of technical reports outlining the implementation of a product, providing a context of the tradeoffs, with a justification how it addresses safety and other concerns.
digital to analog converter	A digital to analog converter is used create a voltage signal from an internal value.
direct memory access	A special purpose microcontroller peripheral that moves data between the microcontroller’s storage and another peripheral or storage; this is useful to reduce work done in software.
error	An error is the occurrence of an incorrect (or undesired) result.
exception	An “event that causes suspension of normal execution” [IEC 61499-1 3.36] A special condition – often an error – that changes the normal control flow. On an ARM Cortex, this can cause the processor to suspend the currently executing instruction stream and execute a specific exception handler or interrupt service routine.
failure ₁	A failure “is a permanent interruption of a system’s ability to perform a required function under specified operating conditions.” (Isermann & Ballé 1997).
failure ₂	An incident or event where the product does not perform functions (esp. critical functions) within in specified limits. e.g. the product did not meet its requirements.
fault ₁	A fault is an abnormal condition, or other unacceptable state of some subsystem (or component) that will disallow the intended operation. The part or subsystem did not meet its requirements. See also <i>abnormal condition</i> , <i>normal operating condition</i> .
fault ₂	A fault is represented an interrupt or exception on ARM processors that pass control to handler of such an abnormal condition.
fault tolerant	“The capability of software to provide continued correct execution in the presence of a defined set of microelectronic hardware and software faults.” [ANSI/UL 1998]

firmware	A program permanently recorded in ROM and therefore essentially a piece of hardware that performs software functions.
flash	A type of persistent (non-volatile) storage media.
frequency monitoring	“a fault/error control technique in which the clock frequency is compared with an independent fixed frequency” [IEC 60730-1]
function	The “specific purpose of an <i>entity</i> or its characteristic action” [IEC 61499-1] That is, what the product is intended to do, and/or what role it is to serve.
function block	A self-contained unit with specific functionality
functional hazard analysis	An “assessment of all hazards against a set of defined hazard classes” [IEC 62396-1 3.21] see also <i>hazard analysis</i>
hard fault	A type of microcontroller fault.
harm	A “physical injury or damage to health” [ISO 12100-1:2003]
hazard	A “potential source of physical injury to persons.”
hazard analysis	The “process of identifying hazards and analysing their causes, and the derivation of requirements to limit the likelihood and consequences of hazards to an acceptable level” [IEC 62280 section 3.1.24] see also <i>functional hazard analysis</i> , <i>preliminary hazard analysis</i> , <i>risk analysis</i>
hazard class	Energy (electric: voltage, current, electric & magnetic fields, radiation, thermal energy, vibration/torsion/kinetic energy/force, acoustic), biological & chemical, operational (function and use error), are informational (labeling, instructions, warnings, markings) [ISO 14971]
hazard list	A list of all identified hazards that a product may present.
high-level specification	System specification, customer inputs, marketing inputs, etc.
identifier	A label that can refer to product, specific version of the product, a document, requirement, test, external document, or comment.
initialization	Places each of the software and microcontroller elements into a known state; performed at startup.
input comparison	“a fault/error control technique by which inputs that are designed to be within specified tolerances are compared.” [IEC 60730-1]
integrity	“The degree to which a system or component prevents unauthorized access to, or modification of, computer programs or data.” [ANSI/UL 1998]
integrity check	Checks to see that a storage unit has retained its data contents properly and that the contents have not changed unintentionally.
integrity level	“An integrity level defines a required level of confidence that a system satisfies critical properties related to relevant risk criteria.” <i>Understanding Integrity Level Concepts</i> , P. Joannou, Alan Wassyng, Computer, 2014
internal fault condition	A programmable element resets for a reason other than a power-on reset; or a fault occurs with any programmable-element, or power supervisor; or a self-test did not pass.
interface	An interface is a defined method of accessing functionality. An object may support several interfaces.
key responsibility	Specifies a functionality that a module is responsible for. It is like a capability requirement.
non-maskable interrupt	A type of microcontroller fault.
non-volatile memory	A storage mechanism that will preserve information without power.

parameter	A controllable quantity for a property.
parity check	A simple form of error detection. Each byte in SRAM has an extra check bit that can catch memory errors.
peripheral lock	The microcontroller's peripheral registers can be locked, preventing modification until microcontroller reset.
power management	An "automatic control mechanism that achieves the ... input power consistent with a pre-determined level of functionality" [IEC 62542 5.10]
power on reset	A type of microcontroller reset that occurs when power is applied to the microcontroller; release from reset allows software to execute.
preliminary hazard analysis	"This evaluates each of the hazards contained in the [preliminary hazard list], and should describe the expected impact of the software on each hazard."
programmable component	"any microelectronic hardware that can be programmed in the design center, the factory, or in the field." [ANSI/UL 1998] This includes FPGAs, microcontrollers, microprocessors, and so on.
programmable system	"the programmable component, including interfaces to users, sensors, actuators, displays, microelectronic hardware architecture," and software [ANSI/UL 1998]
protective control	A control whose "operation ... is intended to prevent a hazardous situation during abnormal operation of the equipment" [IEC 60730-1]
protective electronic circuit	An "electronic circuit that prevents a hazardous situation under abnormal operating conditions" [IEC 60335]
quality management system	A "management system with which an organization will be directed with regard to product quality" [IEC 60194 10.141]
realization	An implementation, or a mathematical model or design that has the target input-output behaviour and can be directly implemented.
redundant monitoring	"the availability of two independent means such as watchdog devices and comparators to perform the same task" [IEC 60730-1]
requirement	An "expression ... conveying objectively verifiable criteria to be fulfilled and from which no deviation is permitted " [ISO/IEC Directives, Part 2, 2016, 3.3.3]
requirements specification	A set of requirements
risk	"a measure that combines the likelihood that a system hazard will occur, the likelihood that an accident will occur and an estimate of the severity of the worst plausible accident." [UCRL-ID-1222514]
risk analysis	A "systematic use of available information to identify hazards and to estimate the risk" [ISO 14971:2007 2.17]
risk management	The "systematic application of management policies, procedures and practices to the tasks of analyzing, evaluating and controlling risk" [ISO 14971:2007 2.22]
safe state	A state that the equipment may be placed into where the relevant risks have been addressed to an acceptable risk index.
safety case	"A safety case is a [clear], structured [comprehensive and defensible] argument, supported by a body of evidence, that provides a compelling, comprehensible and valid case that a system is acceptably safe for a given application in a given environment (i.e. a context)" [MOD (UK) 2007]
safety-critical function	A "function(s) required ... the loss of which would cause the tool to function in such a manner as to expose the user to a risk that is in excess of the risk that is permitted ... under abnormal conditions" [EN 62841]
safety-related function	"Control, protection, and monitoring functions which are intended to reduce the risk of fire, electric shock, or injury to persons." [ANSI/UL 1998]
safety-related control functions	A "control function ... that is intended to maintain the safe condition of the machine or prevent an immediate increase of the risk(s)" [IEC 60204-32 section

	3.62]
	note: not all are safety critical functions.
signal	Often has an active and deactivated state; forms can include a digital logic signal (which may be active high, or active low), an analog signal, some logical state conveyed by a communication method, etc.
single event upset	An ionizing particle flipped a bit or transistor state
single wire debug	An electrical debugging interface for the ARM Cortex microcontrollers.
software development lifecycle	“conceptual structure spanning the life of the software from definition of its requirements to its release” [ISO/IEC 12207 3.11]
software risk analysis	A risk analysis applied to the software
software safety requirement	A safety requirement applied to the function or operation of software
test monitoring	“the provision of independent means such as watchdog devices and comparators which are tested at start up or periodically during operation” [IEC 60730-1]
test report	A report of test outcomes describing how a product performs under test.
test requirement	A requirement that defines what a test must do for a product must pass the test.
test specification	A requirements specification that describes a set of tests intended to check that the product meets its requirements. This may be in the form of test requirements – what the tests are to do – and test procedures.
to be determined	The information is not known as of the writing but will need to be known.
traceability	Ability to follow the steps from output back to original sources. For products, this allows tracing all of the product’s design and features back to the original documents approved by the company. For information, this allows tracing to measurements, methodology and standards.
trace matrix	A tool that is used to identify high level requirements that are not realized by a low-level requirement or design element; and low-level requirements or design requirements that are not driven by a high-level requirement.
validation	Check that the product meets the user’s specification when the item is used as an element of the product
verification	Checking that an item meets its specification
watchdog reset	A microcontroller reset triggered by the expiration of a watchdog timer.
watchdog timer	A hardware timer that automatically resets the microcontroller if the software is unable to periodically service it.
white-box testing	Testing technique focusing on testing functional requirements (and other specifications), with an examination of the internal structure or workings of the item.

APPENDIX B

Product Standards

This appendix provides further, supplemental discussion of standards.

77. STANDARDS

I did not provide a definition of “standard” earlier. Circular No A-119 provides a useful definition of *technical standard*, being that a standard that includes:

1. [The] common and repeated use of rules, conditions, guidelines or characteristics for products or related processes and production methods, and related management systems practices[; and]
2. The definition of terms;
classification of components;
delineation of procedures;
specification of dimensions, materials, performance, designs, or operations;
measurement of quality and quantity in describing materials, processes, products, systems, services, or practices;
test methods and sampling procedures; or
descriptions of fit and measurements of size or strength.

*OMB Circular No A-119, Revised
OMB (US
Government) 1998
Feb 10*

77.1. OTHER IMPORTANT SOFTWARE SAFETY STANDARDS

DO-178C is the aerospace industry’s software quality standard. It employs five levels (instead of 3) and in descending order of concern (as opposed to the IEC 60730’s & 62304 ascending order):

- Level A for Catastrophic
- Level B for Hazard/Severe
- Level C for Major
- Level D for Minor
- Level E for no effect

*DO-178C, Software
Considerations in
Airborne Systems and
Equipment
Certification, RTCA,
Inc. 2012 Jan 5*

NASA-STD-8719.13 is NASA’s software assurance standard. It classifies software criticality in descending level of concern, based on its role and/or complexity. This classification is based on MIL-STD-882C (the last revision to have such a classification).

*NASA-STD-8719.12.,
NASA Software Safety
Standard, Rev C 2013-
5-7*

- Category IA. “Partial or total autonomous control of safety-critical functions by software[; or] Complex system with multiple subsystems, interacting parallel processors, or multiple interfaces[; or] Some or all safety-critical software functions are time critical exceeding response time of other systems or human operator[; or] Failure of the software, or a failure to prevent an event, leads directly to a hazard's occurrence.”
- Category IIA & IIB. “Control of hazard by software but other safety systems can partially mitigate. Software detects hazards, notifies system of need for safety actions.[or]

Moderately complex with few subsystems and/or a few interfaces, no parallel processing[; or] Some hazard control actions may be time critical but do not exceed time needed for adequate human operator or automated system response.[; or] Software failures will allow, or fail to prevent, the hazard's occurrence. “

- Category IIIA & IIIB. “Several non-software mitigating systems prevent hazard if software malfunction[; or] Redundant and independent sources of safety-critical information[; or] Somewhat complex system, limited number of interfaces[; or] Mitigating systems can respond within any time critical period[; or] Software issues commands over potentially hazardous hardware systems, subsystems or components requiring human action to complete the control function.”
- Category IV. “No control over hazardous hardware. No safety-critical data generated for a human operator. Simple system with only 2-3 subsystems, limited number of interfaces. Not time-critical.”

NASA-STD-8739.8 is NASA’s software quality standard. It classifies software criticality in descending level of concern, but based on a classification of intended use rather than hazard:

NASA-STD-8739.8,
“Software Assurance
Standard” NASA
Technical Standard
8739.8 2004, 2004 Jul
28

- Class A Human Rated
- Class B Non-Human Space rated
- Class C Mission support software
- Class D Analysis and Distribution software
- Class E Development support

78. PRODUCT STANDARDS

78.1. TYPES OF ISO SAFETY & PRODUCT STANDARDS

ISO 12100-1:2003 proposes organizing standards into a hierarchy of how broadly or specifically they apply.

- *Basic safety standards* (type A), give generic concepts & principles applicable to all machinery of a class. (ISO 12100 is itself a type A standard)
- *Generic safety standards* address wide range of machinery, but focus on a narrow area of safety (type-B),
 - Type B1 are those that focus on safety “aspect” – some safe operating region often defined along a physical dimension
 - Type B2 are those that focus on safeguards or mechanisms
- Standards for groups or a particular machine (type C) are the narrowest

78.2. TYPES OF IEC SAFETY STANDARDS

IEC safety standards are similarly grouped, from broadest to narrowest:

- *Basic safety publications* give general safety provisions, generic concepts & principles applicable to many products.
- *Group safety publications* address all safety aspects of a specific group of products
- *Product publication* for “a specific product or group of related products” [IEC 2011]

An extra, informal, variant is that a country (or region) may adopt the standards, modifying them in the process. This is important as these are the ones *recognized* (accepted) for the country or region.

78.3. PRODUCT STANDARDS

The table below summarizes how several safety standards adapt software safety-related material from other standards:

Std	Adapts	Type	Sector	Notes
EN/ISO 13849	IEC 61508	B1	machine control	<i>“Safety of machinery - Safety-related Parts of Control Systems” Uses PL risk</i>
ISO 26262	IEC 61508	Group	Automotive	<i>“Road Vehicles Functional Safety” Applies ASIL to automotive electrical/electronic systems</i>
EN 50128:2011		Group	Railway	<i>“Railway applications. Communication, signaling and processing systems.” (includes software)</i>
EN 60601		Group	Medical	Medical device product requirements
UL 61010				<i>Safety Requirements for Electrical Equipment for Measurement, Control, and Laboratory Use - Part 1: General Requirements, 2015 May 11</i>
IEC 61508	DIN 12950	Basic		Adapted risk assessment from DIN 12950
IEC 61511	IEC 61508	Group	Industrial process	<i>“Functional safety - Safety instrumented systems for the process industry sector.”</i>
IEC 61513:2001	IEC 61508			<i>“Nuclear power plants - Instrumentation and control for systems important to safety - General requirements for”</i>
IEC/EN 62061	IEC 61508	Group	Machinery	<i>“Safety of machinery: Functional safety of electrical, electronic and programmable electronic control systems,”</i>
IEC 62279	IEC 61508		Railway	
IEC 62841	IEC 60730	Group	Garden appliances	<i>“Electric motor-operated hand-held tools, transportable tools and lawn and garden machinery - Safety - Part 1: General requirements”</i>

Table 45: Safety standards and where they adapt from

79. REFERENCES AND RESOURCES

IEC, *Basic Safety Publications*, 2011

IEC, *Basic Safety Publications: Tools*

APPENDIX C

Floating-point precision

This appendix summarizes the limits of precision employing a floating-point representation. Floats have some corner cases, and loss of precision.

- There is a -0 with floats. IEEE 754 requires that zeros be signed.
- Floats can have signed infinity (+INF, and -INF)
- Floats can be NAN; there are several different encodings for NAN. (The exponent is zero, and significand is non-zero)
- Division by zero can throw exception, and/or give a NAN as a result.
- Division by non-zero numbers can also give a NAN, such as *denormals*.
- Due to subtleties of precision and other factors, two floating point values must not be compared for equality or inequality using == or !=.
- Floats are not *associative*. The order of addition matters. Adding numbers in different orders can give differing results.
- Float values can be correctly sorted by treating the format as 32-bit integers.

Parameter	Value
maximum value	3.402823×10^{38}
minimum value	-3.402823×10^{38}

Table 46: Float range

From	To	Precision
-16777216	16777216	can be exactly represented
-33554432	-16777217	rounded to a multiple of two
16777217	33554432	rounded to a multiple of two
-2^{n+1}	$-2^n - 1$	rounded to a multiple of 2^{n-23} ; $n > 22$
$2^n + 1$	2^{n+1}	rounded to a multiple of 2^{n-23} ; $n > 22$
$-\infty$	2^{128}	rounded to -INF
2^{128}	∞	rounded to +INF

Table 47: Accuracy of integer values represented as a float

APPENDIX D

Bug Report Template¹⁴

This Appendix describes the best means in which to file a bug report. A useful bug report is written in simple, jargon free language, and structured using the inverted hierarchy.

"The horror of that moment," the King went on, "I shall never, never forget!"
"You will, though," the Queen said, "if you don't make a memorandum of it"—
Lewis Carroll, Through the Looking Glass

80. OUTLINE OF A PROPER BUG REPORT

12 words	1 : Bug Header Information
1-5 words	1.1 : Product
2 words	1.2 : Classification
1-3 words	1.3 : Reproducibility
	1.4 : Version/Build Number
2 words	1.5 : Area of bug
< 20 words	2 : Bug Title & Description
< 20 words	2.1 : Title
	2.2 : Description
	2.3 : Requirements that are of interest or are relevant
	3 : Additional Information To Provide (General)
	3.1 : Configuration Information
	3.2 : Crashing Issues
	3.3 : Application resets
	3.4 : Hanging/Performance Issues
	3.5 : Screen shots, Scope Capture,
	4 : Contact Information
	5 : Product-specific Additional Information

The remainder of the

81. BUG HEADER INFORMATION

1.1: Product:

PC Programmer, Handheld, OurPeripheral, Implant, Telemetry Module, etc, whether it is a first run engineering board, a second run engineering board, a first run production board, a second run production board, etc

Include details such as the part number, or board assembly and serial number

1.2: Classification:

¹⁴ This appendix is adapted from Apple's bug reporting form, as well as many others.

Classify the bug appropriately (partly by its *manifestation*) so that we can properly prioritize the problem:

Method of manifestation is the observable effect

- Crash/Hang/Data Loss: Bugs which cause a machine to crash, resulting in an irrecoverable hang, or loss of data.
- Performance: Issues that reduce the performance or responsiveness of an application.
- Usability: A cosmetic issue, or an issue with the usability of an application.
- Serious bug: Functionality is greatly affected and has no workaround.
- Other bug: A bug that has a workaround.
- Unexpected behaviour: a bug that not only has a work around
- Feature (new): Request for a new feature
- Enhancement: Request for an enhancement to an existing feature.

1.3: Reproducibility

1 to 3 words

Let us know how frequently you can reproduce this problem.

1.4: Version/Build Number:

Provide the version of firmware / software you are using. (If it is an engineering change to a release version, please note that)

1.5 Area of bug:

2 words

This is how the bug manifests itself, or where it has the observable effect:

- Communication
- Therapy Behaviour
- Input to output logic behaviour
- Preferences
- Recharge
- Incorrect or inaccurate results: input/output is wrong, or provides inaccurate information
- Corruption – data is corrupted, altered, lost or destroyed
- Responsiveness, Speed or Performance degradation, efficiency defects
- Power: poor battery life, high power consumption, degradation, efficiency defects
- Increased resource usage in other areas
- Other device behaviour
- It crashes my Handheld / OurPeripheral / Telemetry Module / LabPC / Display Unit

82. BUG TITLE AND DESCRIPTION

2.1: Problem Report Title:

<20 words

The ideal problem title is clear, concise, succinct and informative. It should include the following:

- Build or version of the firmware on which the problem occurred
- Verb describing the action that occurred
- Explanation of the situation which was happening at the time that the problem occurred

- In case of a crash or hang, include the symbol name

The title should also:

- Be objective and clear (and refrain from using idiomatic speech/colloquialisms/slang)
- Include keywords or numbers from any error messages you may be receiving
- Not employ vague terms such as “failed”, “useless”, “crashed”, “observed” etc....

The following examples demonstrate the difference between a non-functional title and a functional title:

Example 1:

Non-functional title:	Handheld Crashed.
Functional title:	Handheld gave a watchdog reset while performing a lead impedance measurement

Example 2:

Non-functional title:	Failed test
Functional title:	OurPeripheral return error ErrOutOfSpace when performing recharge test.

2.2: Description:

The description includes:

- A Summary
- Steps to Reproduce
- Expected Results
- Actual Results
- Workaround, and
- Regression/Isolation
- Relevant requirements.

Summary:

Recap the problem title and be explicit in providing more descriptive summary information.

Provide what happened, what you were doing when it happened, and why you think it's a problem. If you receive an error message, provide the content of the error message (or an approximation of it).

Provide specifics and avoid vague language or colloquialisms. Instead of using descriptive words or phrases when something “looks bad,” “has issues,” “is odd,” “is wrong,” “is acting up,” or “is failing,” be concise and describe how something is looking or acting, why you believe there is a problem, and provide any error messages that will support the problem being reported.

Example 1:

Non-functional description:	When printing, nothing happens. Application doesn't work.
Functional description:	Print Menu item enabled, print dialog box appears, print button enabled, but progress dialog box doesn't appear.

Example 2:

Non-functional description:	Handheld is slow.
Functional description:	Handheld is slow when incrementing therapy amplitude (provide durations)

If there is a clear safety implication, specify it (otherwise do not).

Steps to Reproduce:

Describe the step-by-step process to reproduce the bug, including any non-default preferences/installation, and the system configuration information. Note: It is better to include too much information than not enough, as this reduces the amount of back-and-forth communications. Note: Be very specific and be sure to provide details, as opposed to high-level actions. Test cases with clear & concise steps to reproduce that will enable us to reproduce this and fix.

When does the problem occur? For example:

- Does it occur after power on?
- Does it occur after unlock?
- Does it occur after power off and lock?

Important points to note when providing steps to reproduce are:

- Include information about any preferences that have been changed from the defaults.

Expected Results:

Describe what you expected to happen when performing the steps to reproduce.

Actual Results:

Explain what actually occurred.

With error codes try to include the text name of the error code

Bad:	error 0x12
Good:	ErrParameterOutOfRange (0x12)

Workaround:

If you have found a workaround for this problem, describe it.

Regression/Isolation:

Note any other configurations in which this issue was reproducible. Include details if it is new to this build, or no regression testing was done.

If there are other steps that are similar to those above, but do not create an undesired outcome, please note those. We can use this information to help resolve the issue.

2.3: Requirements that are of interest or are relevant

83. ADDITIONAL INFORMATION REQUIREMENTS (GENERAL)

Reports from developers should include:

- The hardware configuration
- The “preferences” configuration

- The device bonding or pairing configuration
- The “manufacturing data” configuration
- The embedded device(s) configuration
- If reporting an error dialog message or UI bug, provide screen shots
- Log file

Reports from developers should include

- A complete enumeration of the Revision Ids of the source files

Reports from test stations should include:

- The software / firmware version
- Event trace (e.g. log of the connection). Please provide the *smallest* trace possible that captures the issue. As traces may contain a lot of spurious information that doesn't pertain to the issue at hand, it is vital to the bug solving effort to remove distracting volume.

The generation of this information can be done in an automated fashion.

3.2: Crashing Issues:

A crash might include a NMI, Watchdog, Stack Underflow, Stack Overflow, memory fault, bus fault, usage fault, or Hard Fault. Extra information is essential. Please give us:

- The fault register values
- Call stack trace (if possible)

In addition to all the above, provide any information regarding what you were doing around the time of the problem.

NOTE: If you're able to reproduce the crash the exact same way each time and the ____ looks identical in every instance, only one crash report is required. In instances where the crash doesn't look identical, file separate reports with one crash log submitted per bug.

3.4: Hanging/Performance Issues:

If you are experiencing a “hang” (includes freeze, slow data transfer), a sample of the application while it is in the hung state is required.

3.5: Screen shots, Scope Traces and Waveform capture:

SCREEN SHOTS. Provide a screen shot when it will help clarify the bug report. In addition to providing any screen shots to error or dialog messages, be sure to also type the text of the error/dialog message you're seeing in the description of the bug report (so that the contents of the message are searchable. If there are steps involved, a sequence of screen shots, or a movie is always appreciated. Be sure to write down the steps associated with each screen shot.

SCOPE TRACE. When working with electrical signals, please provide scope trace or screen shot of the oscilloscope. Please provide a diagram of the setup, and a description where in the diagram or schematic the signals were measured.

84. CONTACT INFORMATION

Be sure to include the contact information of who found the bug. Although this sounds implicit in an email or trouble tracking system (e.g. ClearQuest, Jira), too often the bug

reporter is different than the one who found it. By including the contact information we'll be able to correspond with them as we investigate the issue.

85. PRODUCT-SPECIFIC ADDITIONAL INFORMATION

When submitting a bug report against certain tools, be sure to provide the following additional information:

- Build number & version. Put the build number at the beginning of your title as such:

1.5.0_06-112: Title Here

If your setup is non-standard, indicate that in the bug report.

Handheld Power Management (sleep/wake) issues:

- Be aware of what is plugged into the Handheld

When submitting a bug report involved a sealed in the can device, be sure to provide:

- Whether the battery is connected or not
- Was it in saline?
- Were leads attached?
- Which version of firmware?
- Was an OurPeripheral being used – which version?

APPENDIX E

Types of Defects

This Appendix describes a system of categorizing bugs.

86. OVERVIEW

The analysis of a bug is intended to gather information about its causes and underlying defects (there may be many) and provide a basis to disposition or prioritize repairs.

Bugs are classified along four dimensions by

1. Method of manifestation.
2. Type of Defect
3. Implication
4. Means of testing

The bug analysis should try included a number of attributes about how the bug manifests itself. And include a chain of analysis to other potential underlying defects.

Defect is the design or implementation mistake
Method of manifestation is the observable effect

87. CLASSIFYING THE TYPE OF DEFECT

The types of defects include:

- Hardware problem
- Hardware misuse
- Storage / access partition violation
- Resource allocation issues
- Arithmetic, numerical bug
- Logic errors
- Syntax errors
- Improper use of API's – violates how an API should be used, including calling sequence, parameter range, etc. Errors in interacting with others in calls, commands, macros, variable settings, control blocks, etc.
- State errors
- Concurrency
- Interaction issues
- Graphic errors
- Security issue – disclosure, alteration/destruction/insertion

Various sources were used in the preparation of this. "A comparative study of industrial static analysis tools (Extend Version)" Par Emanuelsoon, Ulf Nilsson, January 7 2008

87.1. HARDWARE PROBLEM

Defect is the design or implementation mistake

- Missing component
- Component incorrectly mounted
- Component broken
 - Unable to communicate
 - Does not pass self test
 - Does not operate correctly.

87.2. HARDWARE MISUSE

- Power is too high, too low, or off
- Power transition is too fast
- Truncated addresses
- Stack overrun

87.3. STORAGE / ACCESS PARTITION VIOLATIONS

STORAGE / ACCESS PARTITION VIOLATION may have attributes of the storage violation:

- Type of access: read, write
- Location of the segment, and access: stack, or heap
- The boundary violated: above or below the segment/partition.
- How far outside of the segment was the access?
- How much data is affected with the access?
- Stride: were the access violations in a large continuous span, or were there gaps between the accesses?

An access violation can be classified into one of:

- NULL pointer dereference
 - Is a pointer possibly NULL before its use? Is it checked before use?
 - Is it checked for NULL *after* its use?
- Wild pointer dereference
- Pointer arithmetic error
 - Pointer does not point to a meaningful location
 - Pointer points outside of the bounds of its referent.
- Improper memory allocation
- Using memory that has not been initialized
 - Array cell being dereferenced in a fetch (or fetch-n-modify) operation has not been initialized.
 - Pointer being dereferenced has not been initialized (a variation on the use of a variable that has not been initialized)
- Aliasing
 - Two pointers to the same region. Especially without proper *volatile*.

- Pointer to variable storage. Especially without proper *volatile*.
 - Pointer to an array is assigned to point to second, smaller array
- Access (segmentation) violation – using something not allowed to
 - Buffer overflow / overrun
 - Array is indexed outside of its upper or lower bound.
 - Pointer points outside of the bounds of its referent.
 - Possible causes may include pointer arithmetic errors
- Access alignment violation – e.g. having something on a odd address that must be align on 16 byte boundary
- Reference of pointer being dereferenced in a fetch (or fetch-n-modify) operation has not been initialized.
- Function pointer does not point to a function – or points to a function with a different signature.
- Casting an integer in a pointer or pointer-union when it is smaller / larger
- Use of arrays (especially large arrays) on stack. This can happen when returning a struct, or array
- Use of large strings on stack. This can happen when returning a struct, or array
- Return of a pointer to the local stack

Possible causes of these

- Earlier access violation
- Uninitialized value, variable or field used as pointer
- *Arithmetic issues*, for potential sources of erroneous index and pointer calculations
 - Conversion created incorrect value. Check implicit and explicit values for proper widening and conversion.
- Input value wrong, out of range, or does not meet implicit constraints
- String or other data structure missing a termination, e.g. a NULL terminator
- The allocation was smaller than the amount of data to process

Possible fixes and mitigations

- For large strings and arrays passed on stack, pass a pointer to the array
- Add parameter checking and return a value
- Employ sentinel values, and canaries to detect inconsistencies and misuse earlier

87.4. RESOURCE AND REFERENCE MANAGEMENT ISSUES

RESOURCE AND REFERENCE MANAGEMENT ISSUES includes leaks and resources that are not released when they are no longer used:

- Use resource after free
- Double free
- Mismatch array new / delete
- Memory leak (use more memory over time)
 - Constructor / Destructor leaks

- Bad deletion of arrays
- Temporary files
- Resource – esp. memory and file handle – leaks
- Database connection leaks
- Custom memory and network resource leaks

87.5. ARITHMETIC, NUMERICAL BUG & INCORRECT CALCULATIONS

Calculation bugs can include:

- Relying on operator precedence or not understanding operator precedence.
- Overflow or underflow
- Invalid use of negative variables
- Loss of precision. These can come from using the wrong size type or casting to an inappropriate type:
 - Underflow – a number too small
 - Overflow – bigger than can be represented, dropping the most significant bits
 - Truncation – dropping the least significant bits
- Inadequate precision, accuracy, or resolution of type
- Computation is inaccurate. Accuracy issues relate from the formulae used.
- Numerically unstable algorithm
 - Using an IIR with an order higher than 2
 - PID lacks anti-windup (e.g. timers)
 - PID lacks dead-band dampening
- Equality check is incorrect
 - Check for literal zero rather than within epsilon around zero
 - Check equal to NaN, rather than using `isnan()`
- Basic inappropriate values for an operation
 - Using a Not-A-Number
 - Driving by zero
 - Performing an operation, such as `logarithm` and `sqrt()`, on a negative number
 - Shift left by more than the size of the target
 - Shift operand is negative
 - Shift LHS is negative

87.6. ERRORS IN LOGIC

Errors in logical can include:

- Illegal values to operations
- Not checking taint or validating values properly
- Wrong order of parameters in a call
- Variables that have not been initialized
- Dead code cause by logical errors

- Under run – not sending enough on time
- Macros
- Dynamic-link and loading bugs
- Infinite loop / loss of forward progression; a procedure or loop does not terminate.
- Typo between variable and procedure names
- Error in internal check
- *See API misuse*

Logical errors can have three sub-classes of defects:

- Syntax errors
- Unused results
- Incorrect calculation

UNUSED RESULTS. Unreachable code (dead code) may indicate a logical or syntax error. Data that is computed but not used may also indicate logical errors or misspellings. Data stored via a pointer but is not used may indicate a problem.

87.7. API OR COMPONENT INTERFACE MISUSE

Interface Misuse – violates how an API should be used, including calling sequence, parameter range, etc. Errors in interacting with others in calls, commands, macros, variable settings, control blocks, etc. A description of the interface should be concise, but provide enough information to understand the intended used and limitations

- STL usage errors
- API error handling
- Misuse of `sprintf`, other `varargs`, and `argv`

87.8. ERROR HANDLING

- Uncaught fault / exception.
- Inadequate fault / exception handling.
- Not checking return values
- Not checking error values

87.9. SYNTAX ERRORS

SYNTAX ERRORS may produce some of the logical errors above:

- Use of the comma operator
- Misplacement of “;”, especially in conditional statements
- Forgotten breaks.
- The use of variables that were not initialized with values
- Return statements without defined value – either the return is implicit, no value is specified, or the return accesses a variable that has not been initialized.
- Return of a pointer to the local stack

MISRA has recommended these checks

- Inconsistent return values for input

87.10. STATE ERRORS

- Results in wrong state
- Transition from state A to state B is not allowed
- Does not handle event in given state
- Handles event incorrectly in given state.

87.11. CONCURRENCY

- Deadlocks
- Double locking
- Missing lock releases
- Release order does not match acquisition order of other thread means dead lock, etc. (Aka reversed order of clocking)
 - Static / dynamic analysis should check the lock order (for several locks)
- Blocking call misuse
- Associate variable/register/object access with specific locks
- Lock contention

87.12. INTERACTION ISSUES

- Thread prioritizations
- Contention for resources (including, but not limited to lock contention)
- Data rate is incorrect / mismatch
- Differing process rates
- Sourcing events faster than they can be processed
- Long communication and processing pipelines
- Timing violation, too soon / too late
 - Timer incorrectly set
 - Timer stopped
 - Timer reset
- Sequence of operation is incorrect
 - Wrong command sent
 - Missing command
- Wrong response is sent
- Sent to wrong party
- Format is wrong
- Length is wrong
- Misinterpreted
- Ignored command or response
- Mis-estimated state of other party

- Redundant interaction

87.13. GRAPHIC ERRORS

- Position incorrect
- Size incorrect / truncated
- Shape incorrect
- Parent / child relationship is incorrect
- Incorrect sibling order / tab order
- Color is wrong
- Text is wrong
- Graphic mismatch / pixels not refreshed
- Pixels not being refreshed / dirty rectangle issue
- Item is not visible when it should be
- Item is visible when it should not be

87.14. SECURITY VULNERABILITY

- Temporary files. Not using secure temporary files, file names.
- Missing / insufficient validation of malicious data and string input (*see also taint checking*)
 - SQL injection attacks
- Cross-site scripting attacks
- Format string vulnerabilities
- Faulty permission models – not a bug with access checks, but many with wrong arrangement of access controls (it's very hard to do bottom up)
- Incorrect use of `chroot`, `access`, and `chmod`.
- Bad passwords
- Dynamic-link and loading bugs
- Spoofing
- Race conditions and other concurrency issues
- Poor encryption
- Command injection
- Not checking values or their origins
- Race conditions with system calls

88. RESOURCES

ANSI/AAMI SW91:2018 – *Classification of defects in health software*

An interesting standard classifying defects giving each an identifier (coding), it includes a mapping of other standards (such as FDA, and TR80002-1) to its classification. Worth a look.

APPENDIX F

Code-Complete Requirements Review Checklists

Adapted from

**STEVEN C.
MCCONNELL,**
*CODE COMPLETE,
2ND ED.*

Source: <https://github.com/janosgyerik/software-construction-notes/tree/master/checklists-all>

This material is copied and/or adapted from the Code Complete 2 Website at cc2e.com. This material is Copyright (c) 1993-2004 Steven C. McConnell. Permission is hereby given to copy, adapt, and distribute this material as long as this notice is included on all such materials and the materials are not sold, licensed, or otherwise distributed for commercial gain.

89. CHECKLIST: REQUIREMENTS

89.1. SPECIFIC FUNCTIONAL REQUIREMENTS

- ☐ Are all the inputs to the system specified, including their source, accuracy, range of values, and frequency?
- ☐ Are all the outputs from the system specified, including their destination, accuracy, range of values, frequency, and format?
- ☐ Are all output formats specified for web pages, reports, and so on?
- ☐ Are all the external hardware and software interfaces specified?
- ☐ Are all the external communication interfaces specified, including handshaking, error-checking, and communication protocols?
- ☐ Are all the tasks the user wants to perform specified?
- ☐ Is the data used in each task and the data resulting from each task specified?

89.2. SPECIFIC NON-FUNCTIONAL (QUALITY) REQUIREMENTS

- ☐ Is the expected response time, from the user's point of view, specified for all necessary operations?
- ☐ Are other timing considerations specified, such as processing time, data-transfer rate, and system throughput?

This material is copied and/or adapted from the Code Complete 2 Website at cc2e.com. This material is Copyright (c) 1993-2004 Steven C. McConnell. Permission is hereby given to copy, adapt, and distribute this material as long as this notice is included on all such materials and the materials are not sold, licensed, or otherwise distributed for commercial gain.

- ☐ Is the level of security specified?
- ☐ Is the reliability specified, including the consequences of software failure, the vital information that needs to be protected from failure, and the strategy for error detection and recovery?
- ☐ Is maximum memory specified?
- ☐ Is the maximum storage specified?
- ☐ Is the maintainability of the system specified, including its ability to adapt to changes in specific functionality, changes in the operating environment, and changes in its interfaces with other software?
- ☐ Is the definition of success included? Of failure?

89.3. REQUIREMENTS QUALITY

- ☐ Are the requirements written in the user's language? Do the users think so?
- ☐ Does each requirement avoid conflicts with other requirements?
- ☐ Are acceptable trade-offs between competing attributes specified—for example, between robustness and correctness?
- ☐ Do the requirements avoid specifying the design?
- ☐ Are the requirements at a fairly consistent level of detail? Should any requirement be specified in more detail? Should any requirement be specified in less detail?
- ☐ Are the requirements clear enough to be turned over to an independent group for construction and still be understood?
- ☐ Is each item relevant to the problem and its solution? Can each item be traced to its origin in the problem environment?
- ☐ Is each requirement testable? Will it be possible for independent testing to determine whether each requirement has been satisfied?
- ☐ Are all possible changes to the requirements specified, including the likelihood of each change?

89.4. REQUIREMENTS COMPLETENESS

- ☐ Where information isn't available before development begins, are the areas of incompleteness specified?
- ☐ Are the requirements complete in the sense that if the product satisfies every requirement, it will be acceptable?
- ☐ Are you comfortable with all the requirements? Have you eliminated requirements that are impossible to implement and included just to appease your customer or your boss?

APPENDIX G

Code-Complete Design Review Checklists

Adapted from

STEVEN C.
MCCONNELL,
CODE COMPLETE,
2ND ED.

Source: <https://github.com/janosgyerik/software-construction-notes/tree/master/checklists-all>

90. CHECKLIST: ARCHITECTURE

90.1. SPECIFIC ARCHITECTURAL TOPICS

This material is copied and/or adapted from the Code Complete 2 Website at cc2e.com. This material is Copyright (c) 1993-2004 Steven C. McConnell. Permission is hereby given to copy, adapt, and distribute this material as long as this notice is included on all such materials and the materials are not sold, licensed, or otherwise distributed for commercial gain.

- ☐ Is the overall organization of the program clear, including a good architectural overview and justification?
- ☐ Are major building blocks well defined, including their areas of responsibility and their interfaces to other building blocks?
- ☐ Are all the functions listed in the requirements covered sensibly, by neither too many nor too few building blocks?
- ☐ Are the most critical classes described and justified?
- ☐ Is the data design described and justified?
- ☐ Is the database organization and content specified?
- ☐ Are all key business rules identified and their impact on the system described?
- ☐ Is a strategy for the user interface design described?
- ☐ Is the user interface modularized so that changes in it won't affect the rest of the program?
- ☐ Is a strategy for handling I/O described and justified?
- ☐ Are resource-use estimates and a strategy for resource management described and justified?
- ☐ Are the architecture's security requirements described?
- ☐ Does the architecture set space and speed budgets for each class, subsystem, or functionality area?
- ☐ Does the architecture describe how scalability will be achieved?
- ☐ Does the architecture address interoperability?
- ☐ Is a strategy for internationalization/localization described?
- ☐ Is a coherent error-handling strategy provided?

This material is copied and/or adapted from the Code Complete 2 Website at cc2e.com. This material is Copyright (c) 1993-2004 Steven C. McConnell. Permission is hereby given to copy, adapt, and distribute this material as long as this notice is included on all such materials and the materials are not sold, licensed, or otherwise distributed for commercial gain.

- ☐ Is the approach to fault tolerance defined (if any is needed)?
- ☐ Has technical feasibility of all parts of the system been established?
- ☐ Is an approach to overengineering specified?
- ☐ Are necessary buy-vs.-build decisions included?
- ☐ Does the architecture describe how reused code will be made to conform to other architectural objectives?
- ☐ Is the architecture designed to accommodate likely changes?
- ☐ Does the architecture describe how reused code will be made to conform to other architectural objectives?

90.2. GENERAL ARCHITECTURAL QUALITY

- ☐ Does the architecture account for all the requirements?
- ☐ Is any part over- or under-architected? Are expectations in this area set out explicitly?
- ☐ Does the whole architecture hang together conceptually?
- ☐ Is the top-level design independent of the machine and language that will be used to implement it?
- ☐ Are the motivations for all major decisions provided?
- ☐ Are you, as a programmer who will implement the system, comfortable with the architecture?

90.3. CHECKLIST: UPSTREAM PREREQUISITES

- ☐ Have you identified the kind of software project you're working on and tailored your approach appropriately?
- ☐ Are the requirements sufficiently well-defined and stable enough to begin construction (see the requirements checklist for details)?
- ☐ Is the architecture sufficiently well defined to begin construction (see the architecture checklist for details)?
- ☐ Have other risks unique to your particular project been addressed, such that construction is not exposed to more risk than necessary?

91. CHECKLIST: MAJOR CONSTRUCTION PRACTICES

91.1. CODING

- ☐ Have you defined coding conventions for names, comments, and formatting?
- ☐ Have you defined specific coding practices that are implied by the architecture, such as how error conditions will be handled, how security will be addressed, and so on?
- ☐ Have you identified your location on the technology wave and adjusted your approach to match? If necessary, have you identified how you will program into the language rather than being limited by programming in it?

91.2. TEAMWORK

- ☐ Have you defined an integration procedure, that is, have you defined the specific steps a programmer must go through before checking code into the master sources?

This material is copied and/or adapted from the Code Complete 2 Website at cc2e.com. This material is Copyright (c) 1993-2004 Steven C. McConnell. Permission is hereby given to copy, adapt, and distribute this material as long as this notice is included on all such materials and the materials are not sold, licensed, or otherwise distributed for commercial gain.

- ☐ Will programmers program in pairs, or individually, or some combination of the two?

91.3. QUALITY ASSURANCE

- ☐ Will programmers write test cases for their code before writing the code itself?
- ☐ Will programmers write unit tests for their code regardless of whether they write them first or last?
- ☐ Will programmers step through their code in the debugger before they check it in?
- ☐ Will programmers integration-test their code before they check it in?
- ☐ Will programmers review or inspect each others' code?

91.4. TOOLS

- ☐ Have you selected a revision control tool?
- ☐ Have you selected a language and language version or compiler version?
- ☐ Have you decided whether to allow use of non-standard language features?
- ☐ Have you identified and acquired other tools you'll be using editor, refactoring tool, debugger, test framework, syntax checker, and so on?

92. CHECKLIST: DESIGN IN CONSTRUCTION

92.1. DESIGN PRACTICES

- ☐ Have you iterated, selecting the best of several attempts rather than the first attempt?
- ☐ Have you tried decomposing the system in several different ways to see which way will work best?
- ☐ Have you approached the design problem both from the top down and from the bottom up?
- ☐ Have you prototyped risky or unfamiliar parts of the system, creating the absolute minimum amount of throwaway code needed to answer specific questions?
- ☐ Has your design been reviewed, formally or informally, by others?
- ☐ Have you driven the design to the point that its implementation seems obvious?
- ☐ Have you captured your design work using an appropriate technique such as a Wiki, email, flipcharts, digital camera, UML, CRC cards, or comments in the code itself?

92.2. DESIGN GOALS

- ☐ Does the design adequately address issues that were identified and deferred at the architectural level?
- ☐ Is the design stratified into layers?
- ☐ Are you satisfied with the way the program has been decomposed into subsystems, packages, and classes?
- ☐ Are you satisfied with the way the classes have been decomposed into routines?
- ☐ Are classes designed for minimal interaction with each other?
- ☐ Are classes and subsystems designed so that you can use them in other systems?
- ☐ Will the program be easy to maintain?
- ☐ Is the design lean? Are all of its parts strictly necessary?

This material is copied and/or adapted from the Code Complete 2 Website at cc2e.com. This material is Copyright (c) 1993-2004 Steven C. McConnell. Permission is hereby given to copy, adapt, and distribute this material as long as this notice is included on all such materials and the materials are not sold, licensed, or otherwise distributed for commercial gain.

- ☐ Does the design use standard techniques and avoid exotic, hard-to-understand elements?
- ☐ Overall, does the design help minimize both accidental and essential complexity?

93. CHECKLIST: CLASS QUALITY

93.1. ABSTRACT DATA TYPES

- ☐ Have you thought of the classes in your program as Abstract Data Types and evaluated their interfaces from that point of view?

93.2. ABSTRACTION

- ☐ Does the class have a central purpose?
- ☐ Is the class well named, and does its name describe its central purpose?
- ☐ Does the class's interface present a consistent abstraction?
- ☐ Does the class's interface make obvious how you should use the class?
- ☐ Is the class's interface abstract enough that you don't have to think about how its services are implemented? Can you treat the class as a black box?
- ☐ Are the class's services complete enough that other classes don't have to meddle with its internal data?
- ☐ Has unrelated information been moved out of the class?
- ☐ Have you thought about subdividing the class into component classes, and have you subdivided it as much as you can?
- ☐ Are you preserving the integrity of the class's interface as you modify the class?

93.3. ENCAPSULATION

- ☐ Does the class minimize accessibility to its members?
- ☐ Does the class avoid exposing member data?
- ☐ Does the class hide its implementation details from other classes as much as the programming language permits?
- ☐ Does the class avoid making assumptions about its users, including its derived classes?
- ☐ Is the class independent of other classes? Is it loosely coupled?

93.4. INHERITANCE

- ☐ Is inheritance used only to model “is a” relationships?
- ☐ Does the class documentation describe the inheritance strategy?
- ☐ Do derived classes adhere to the Liskov Substitution Principle?
- ☐ Do derived classes avoid “overriding” non-overridable routines?
- ☐ Are common interfaces, data, and behavior as high as possible in the inheritance tree?
- ☐ Are inheritance trees fairly shallow?
- ☐ Are all data members in the base class private rather than protected?

93.5. OTHER IMPLEMENTATION ISSUES

- ☐ Does the class contain about seven data members or fewer?

This material is copied and/or adapted from the Code Complete 2 Website at cc2e.com. This material is Copyright (c) 1993-2004 Steven C. McConnell. Permission is hereby given to copy, adapt, and distribute this material as long as this notice is included on all such materials and the materials are not sold, licensed, or otherwise distributed for commercial gain.

- ☐ Does the class minimize direct and indirect routine calls to other classes?
- ☐ Does the class collaborate with other classes only to the extent absolutely necessary?
- ☐ Is all member data initialized in the constructor?
- ☐ Is the class designed to be used as deep copies rather than shallow copies unless there's a measured reason to create shallow copies?

93.6. LANGUAGE-SPECIFIC ISSUES

- ☐ Have you investigated the language-specific issues for classes in your specific programming language?

94. CHECKLIST: THE PSEUDOCODE PROGRAMMING PROCESS

- ☐ Have you checked that the prerequisites have been satisfied?
- ☐ Have you defined the problem that the class will solve?
- ☐ Is the high level design clear enough to give the class and each of its routines a good name?
- ☐ Have you thought about how to test the class and each of its routines?
- ☐ Have you thought about efficiency mainly in terms of stable interfaces and readable implementations, or in terms of meeting resource and speed budgets?
- ☐ Have you checked the standard libraries and other code libraries for applicable routines or components?
- ☐ Have you checked reference books for helpful algorithms?
- ☐ Have you designed each routine using detailed pseudocode?
- ☐ Have you mentally checked the pseudocode? Is it easy to understand?
- ☐ Have you paid attention to warnings that would send you back to design (use of global data, operations that seem better suited to another class or another routine, and so on)?
- ☐ Did you translate the pseudocode to code accurately?
- ☐ Did you apply the PPP recursively, breaking routines into smaller routines when needed?
- ☐ Did you document assumptions as you made them?
- ☐ Did you remove comments that turned out to be redundant?
- ☐ Have you chosen the best of several iterations, rather than merely stopping after your first iteration?
- ☐ Do you thoroughly understand your code? Is it easy to understand?

95. CHECKLIST: A QUALITY-ASSURANCE PLAN

- ☐ Have you identified specific quality characteristics that are important to your project?
- ☐ Have you made others aware of the projects quality objectives?
- ☐ Have you differentiated between external and internal quality characteristics?
- ☐ Have you thought about the ways in which some characteristics may compete with or complement others?

- ☐ Does your project call for the use of several different error-detection techniques suited to finding several different kinds of errors?
- ☐ Does your project include a plan to take steps to assure software quality during each stage of software development?
- ☐ Is the quality measured in some way so that you can tell whether its improving or degrading?
- ☐ Does management understand that quality assurance incurs additional costs up front in order to save costs later?

96. CHECKLIST: EFFECTIVE PAIR PROGRAMMING

- ☐ Do you have a coding standard to support pair programming that's focused on programming rather than on philosophical coding-style discussions?
- ☐ Are both partners participating actively?
- ☐ Are you avoiding pair programming everything, instead selecting the assignments that will really benefit from pair programming?
- ☐ Are you rotating pair assignments and work assignments regularly?
- ☐ Are the pairs well matched in terms of pace and personality?
- ☐ Is there a team leader to act as the focal point for management and other people outside the project?

97. CHECKLIST: TEST CASES

- ☐ Does each requirement that applies to the class or routine have its own test case?
- ☐ Does each element from the design that applies to the class or routine have its own test case?
- ☐ Has each line of code been tested with at least one test case? Has this been verified by computing the minimum number of tests necessary to exercise each line of code?
- ☐ Have all defined-used data-flow paths been tested with at least one test case?
- ☐ Has the code been checked for data-flow patterns that are unlikely to be correct, such as defined-defined, defined-exited, and defined-killed?
- ☐ Has a list of common errors been used to write test cases to detect errors that have occurred frequently in the past?
- ☐ Have all simple boundaries been tested – maximum, minimum, and off-by-one boundaries?
- ☐ Have compound boundaries been tested – that is, combinations of input data that might result in a computed variable that is too small or too large?
- ☐ Do test cases check for the wrong kind of data – for example, a negative number of employees in a payroll program?
- ☐ Are representative, middle-of-the-road values tested?
- ☐ Is the minimum normal configuration tested?
- ☐ Is the maximum normal configuration tested?
- ☐ Is compatibility with old data tested? And are old hardware, old versions of the operating system, and interfaces with old versions of other software tested?

- ☐ Do the test cases make hand-checks easy?

98. CHECKLIST: DEBUGGING REMINDERS

98.1. TECHNIQUES FOR FINDING DEFECTS

- ☐ Use all the data available to make your hypothesis
- ☐ Refine the test cases that produce the error
- ☐ Exercise the code in your unit test suite
- ☐ Use available tools
- ☐ Reproduce the error several different ways
- ☐ Generate more data to generate more hypotheses
- ☐ Use the results of negative tests
- ☐ Brainstorm for possible hypotheses
- ☐ Narrow the suspicious region of the code
- ☐ Be suspicious of classes and routines that have had defects before
- ☐ Check code that's changed recently
- ☐ Expand the suspicious region of the code
- ☐ Integrate incrementally
- ☐ Check for common defects
- ☐ Talk to someone else about the problem
- ☐ Take a break from the problem
- ☐ Set a maximum time for quick and dirty debugging
- ☐ Make a list of brute force techniques, and use them

98.2. TECHNIQUES FOR SYNTAX ERRORS

- ☐ Don't trust line numbers in compiler messages
- ☐ Don't trust compiler messages
- ☐ Don't trust the compilers second message
- ☐ Divide and conquer
- ☐ Find extra comments and quotation marks

98.3. TECHNIQUES FOR FIXING DEFECTS

- ☐ Understand the problem before you fix it
- ☐ Understand the program, not just the problem
- ☐ Confirm the defect diagnosis
- ☐ Relax
- ☐ Save the original source code
- ☐ Fix the problem, not the symptom
- ☐ Change the code only for good reason

This material is copied and/or adapted from the Code Complete 2 Website at cc2e.com. This material is Copyright (c) 1993-2004 Steven C. McConnell. Permission is hereby given to copy, adapt, and distribute this material as long as this notice is included on all such materials and the materials are not sold, licensed, or otherwise distributed for commercial gain.

- ☐ Make one change at a time
- ☐ Check your fix
- ☐ Look for similar defects

98.4. GENERAL APPROACH TO DEBUGGING

- ☐ Do you use debugging as an opportunity to learn more about your program, mistakes, code quality, and problem-solving approach?
- ☐ Do you avoid the trial-and-error, superstitious approach to debugging?
- ☐ Do you assume that errors are your fault?
- ☐ Do you use the scientific method to stabilize intermittent errors?
- ☐ Do you use the scientific method to find defects?
- ☐ Rather than using the same approach every time, do you use several different techniques to find defects?
- ☐ Do you verify that the fix is correct?
- ☐ Do you use compiler warnings?

99. CHECKLIST: CODE-TUNING STRATEGY

99.1. OVERALL PROGRAM PERFORMANCE

- ☐ Have you considered improving performance by changing the program requirements?
- ☐ Have you considered improving performance by modifying the program's design?
- ☐ Have you considered improving performance by modifying the class design?
- ☐ Have you considered improving performance by avoiding operating system interactions?
- ☐ Have you considered improving performance by avoiding I/O?
- ☐ Have you considered improving performance by using a compiled language instead of an interpreted language?
- ☐ Have you considered improving performance by using compiler optimizations?
- ☐ Have you considered improving performance by switching to different hardware?
- ☐ Have you considered code tuning only as a last resort?

99.2. CODE-TUNING APPROACH

- ☐ Is your program fully correct before you begin code tuning?
- ☐ Have you measured performance bottlenecks before beginning code tuning?
- ☐ Have you measured the effect of each code-tuning change?
- ☐ Have you backed out the code-tuning changes that didn't produce the intended improvement?
- ☐ Have you tried more than one change to improve performance of each bottleneck, i.e., iterated?

100. CHECKLIST: CONFIGURATION MANAGEMENT

100.1. GENERAL

- ☐ Is your software-configuration-management plan designed to help programmers and minimize overhead?
- ☐ Does your SCM approach avoid overcontrolling the project?
- ☐ Do you group change requests, either through informal means such as a list of pending changes or through a more systematic approach such as a change-control board?
- ☐ Do you systematically estimate the effect of each proposed change?
- ☐ Do you view major changes as a warning that requirements development isn't yet complete?

100.2. TOOLS

- ☐ Do you use version-control software to facilitate configuration management?
- ☐ Do you use version-control software to reduce coordination problems of working in teams?

100.3. BACKUP

- ☐ Do you back up all project materials periodically?
- ☐ Are project backups transferred to off-site storage periodically?
- ☐ Are all materials backed up, including source code, documents, graphics, and important notes?
- ☐ Have you tested the backup-recovery procedure?

101. CHECKLIST: INTEGRATION

101.1. INTEGRATION STRATEGY

- ☐ Does the strategy identify the optimal order in which subsystems, classes, and routines should be integrated?
- ☐ Is the integration order coordinated with the construction order so that classes will be ready for integration at the right time?
- ☐ Does the strategy lead to easy diagnosis of defects?
- ☐ Does the strategy keep scaffolding to a minimum?
- ☐ Is the strategy better than other approaches?
- ☐ Have the interfaces between components been specified well? (Specifying interfaces isn't an integration task, but verifying that they have been specified well is.)

101.2. DAILY BUILD AND SMOKE TEST

- ☐ Is the project building frequently – ideally, daily to support incremental integration?
- ☐ Is a smoke test run with each build so that you know whether the build works?
- ☐ Have you automated the build and the smoke test?

- ☐ Do developers check in their code frequently – going no more than a day or two between check-ins?
- ☐ Is a broken build a rare occurrence?
- ☐ Do you build and smoke test the software even when you're under pressure?

102. CHECKLIST: PROGRAMMING TOOLS

- ☐ Do you have an effective IDE?
- ☐ Does your IDE support outline view of your program; jumping to definitions of classes, routines, and variables; source code formatting; brace matching or begin-end matching; multiple file string search and replace; convenient compilation; and integrated debugging?
- ☐ Do you have tools that automate common refactorings?
- ☐ Are you using version control to manage source code, content, requirements, designs, project plans, and other project artifacts?
- ☐ If you're working on a very large project, are you using a data dictionary or some other central repository that contains authoritative descriptions of each class used in the system?
- ☐ Have you considered code libraries as alternatives to writing custom code, where available?
- ☐ Are you making use of an interactive debugger?
- ☐ Do you use make or other dependency-control software to build programs efficiently and reliably?
- ☐ Does your test environment include an automated test framework, automated test generators, coverage monitors, system perturbers, diff tools, and defect tracking software?
- ☐ Have you created any custom tools that would help support your specific project's needs, especially tools that automate repetitive tasks?
- ☐ Overall, does your environment benefit from adequate tool support?

APPENDIX H

Design Review Rubric

This appendix describes the rating of design.

103. DOCUMENTATION

103.1. READABILITY RUBRIC

					<i>Table 48: Readability rubric</i>
Trait	Exceptional	Acceptable	Unsatisfactory	Needs improvement	
<i>Process</i>	There is a clear process to guide design requirements and choices	There rules of thumb, and senior team members are mature guides.	There is no process. Members duplicate previous projects	There is no process. People make it up as they go along	
<i>Follows guides / process</i>	Process & style guidelines are followed correctly.	Process & style guidelines are almost always followed correctly.	Process & style guidelines are not followed. Style guide may be inadequate.	Does not follow process or does not match style guide; style guide may not exist.	
<i>Organization</i>	The documentation is exceptionally well organized	The documentation is logically organized.	The documentation is poorly organized	The documentation is disorganized	
<i>Readability</i>	The documentation is very easy to follow, understandable, is clean, and has no errors	The documentation is easy to read. Minor issues with consistent naming, or general organization.	The documentation is readable only by someone who knows what it is supposed to be doing. At least one major issue with names, or organization.	The documentation is poorly organized and very difficult to read. Major problems with at names and organization.	
<i>Diagrams</i>	Diagrams are clear and help understanding	Diagrams are mostly clear and do not sacrifice understanding	Diagrams are mostly confusing, overwrought, or junk	No diagrams used	
<i>Naming</i>	All names follow naming conventions, are meaningful or expressive, and defined. Glossary is complete.	Names are mostly consistent in style and expressive. Isolated cases may be overly terse or ambiguous. No glossary	Names are often cryptic or overly terse, ambiguous or misleading. No glossary.	Names are cryptic; items may be referred to by multiple different names or phrases. No glossary is given.	

This material is copied and/or adapted from the Code Complete 2 Website at cc2e.com. This material is Copyright (c) 1993-2004 Steven C. McConnell. Permission is hereby given to copy, adapt, and distribute this material as long as this notice is included on all such materials and the materials are not sold, licensed, or otherwise distributed for commercial gain.

103.2. ORGANIZATION AND CLARITY

Trait	Exceptional	Acceptable	Unsatisfactory	Needs improvement	Table 49: <i>Documentation organization and clarity rubric</i>
<i>Documentation</i>	The documentation is well written and clearly explains what the design is accomplishing and how, at an appropriate level of detail. All required and most optional elements are present and follow the prescribed format.	The documentation is not compelling; consists of embedded comment and some simple header documentation that is somewhat useful in understanding the documentation. All files, procedures, and structures are given an overview statement.	The documentation is simply comments embedded in the code and pretty printed. Does little to help the reader understand the design.	No documentation.	
<i>Overview statement</i>	The overview is given and explains what the documentation is accomplishing.	The overview is given but is minimal and is only somewhat useful in understanding the documentation.	The overview is not given or is not helpful in understanding what the documentation is to accomplish.	No overview is given.	
<i>Top-Down Design</i>	Top-down design method followed and written in appropriate detail.	Top-down method followed, but level of detail is too vague or too exact.	Top-down design method attempted, but poorly executed.	No design.	
<i>Modularization & Generalization</i>	The description is broken into well thought out elements that are of an appropriate length, scope and independence.	Documentation elements are generally well planned and executed. Some documentation is repeated. Individual elements are often, but not always, written in a way that invites reuse.	Documentation elements are not well thought out, are used in a somewhat arbitrary fashion, or do not improve clarity. Elements are seldom written in a way that invites reuse.		
<i>Reusability</i>	Individual elements were developed in a manner that actively invites reuse in other projects.	Most of the documentation could be reused in other projects.	Some parts of the documentation could be reused in other projects.	The documentation is not organized for reusability.	
<i>Design & Diagrams</i>	A design tool or diagram is correctly used	A design or diagram tool is used but does not entirely match text	A design or diagram tool is used but is incorrect.	No design or diagram tool is used.	
<i>Identification</i>	All identifying information is shown in the documentation	Some identifying information is shown.	Only a small portion of identifying information is shown, and/or is not correct.	No identifying information is shown.	

This material is copied and/or adapted from the Code Complete 2 Website at cc2e.com. This material is Copyright (c) 1993-2004 Steven C. McConnell. Permission is hereby given to copy, adapt, and distribute this material as long as this notice is included on all such materials and the materials are not sold, licensed, or otherwise distributed for commercial gain.

104. DESIGN

Trait	Exceptional	Acceptable	Unsatisfactory	Needs improvement
<i>Overall design</i>	The design is elegant, complete system	The design lacks some critical design components; simpler than comparable products	The design lacks many critical design components, is not simpler than comparable products	The design is lacking most or all design components, or is excessively complex
<i>Understanding</i>	Shows thorough understanding of the mission, the components, underlying techniques and science	Shows moderate understanding of the mission, the components, underlying techniques and science	Shows minimal understanding of the mission, the components, underlying techniques and science	Can't describe what the design will do, shows little knowledge of why some components are employed or understanding of what they do
<i>Design, & Structure</i>	The design proceeds in a clear and logical manner. Structures are used correctly. The most appropriate algorithms are used.	The design is mostly clear and logical. Structures are used correctly. Reasonable algorithms are employed.	The design isn't as clear or logical as it should be. Structures are occasionally used incorrectly. Portions are clearly inefficient or unnecessarily complicated.	The design is sparse or appears to be patched together. Requires significant effort to comprehend.
<i>Modularization & Generalization</i>	The design is broken into well thought out components that are of an appropriate scale, scope and independence.	Components are generally well planned and executed. Individual components are often, but not always, written in a way that invites reuse.	Components are not well thought out, are used in a somewhat arbitrary fashion, or do not improve clarity. Elements are seldom written in a way that invites reuse.	
<i>Cohesion</i>	All the components look like they belong together.	Most of the components look like they belong together.	Some of the components look like they belong together.	Few components look like they belong together.
<i>Reusability</i>	Individual components were designed in a manner that actively invites reuse in other projects.	Most of the components could be reused in other projects.	Some parts of the design could be reused in other projects.	The design is not organized for reusability.
<i>Efficiency</i>	The design is extremely efficient, using the best approach in every case.	The design is efficient at completing most tasks	The design uses poorly chosen approaches in at least one place. For example, the documentation is brute force	Many things in the design could have been accomplished in an easier, faster, or otherwise better fashion.

Table 50:
Implementation rubric

Trait	Exceptional	Acceptable	Unsatisfactory	Needs improvement
<i>Correctness</i>	Prioritization properly based on Rate Monotonic Analysis. Performs error checking in all cases. Appropriately bounded time checks are used in all cases. Resources are appropriately sized.	Has potential or obvious deadlocks. Some operations do not use time limits or use limits that are inappropriate. Does not check for error/lack of resources in some case. Has prioritization, based on ad hoc experience, not on analysis. Mutexes correctly used. Semaphores may overflow, or not wake task	Has obvious deadlocks. Does not use time limits on operations. Doesn't check for error, or lack of resources. Resource sizing is not based on analysis. Has prioritization, based on ad hoc experience, not on analysis. Semaphores or mutexes misused.	Has obvious deadlocks. Does not use time limits to operations. Doesn't check for error/lack of resources. Resource sizing is not based on analysis. No prioritization, not based on analysis
<i>Problem Prevention</i>	Communication / resource utilization has effective (or best in class) collision avoidance algorithms	Communication / resource utilization has some collision avoidance algorithm(s), but it is not always effective (or best in class)	Communication / resource utilization has poorly thought-out collision avoidance approach	Communication / resource utilization has no collision avoidance algorithm
	Has fallback on collision, reducing further errors in all cases	Has fallback on collision, reducing further errors in most cases	Has fallback on collision, but fails to significantly reduce collisions	Has no fallback on collision
<i>Safety</i>	Controls have been identified from analysis such as SIL or FMEA. Device handles error/exception circumstances correctly. Device engages safe conditions in all cases. Internal state is monitored. External state is monitored. Self-checks are performed correctly. Memory and other internal protection are employed.	Internal state, such as values and Buffers are checked. Output monitoring is employed. Self-test is not performed.	Some safe bounds are used. Some value/range checking is employed. Some output monitoring is employed.	No requirements, no analysis, no action.

APPENDIX I

Code-Complete Code Review Checklists

Adapted from

**STEVEN C.
MCCONNELL,**
*CODE COMPLETE,
2ND ED.*

This material is copied and/or adapted from the Code Complete 2 Website at cc2e.com. This material is Copyright (c) 1993-2004 Steven C. McConnell. Permission is hereby given to copy, adapt, and distribute this material as long as this notice is included on all such materials and the materials are not sold, licensed, or otherwise distributed for commercial gain.

Source: <https://github.com/janosgyerik/software-construction-notes/tree/master/checklists-all>

105. CHECKLIST: EFFECTIVE INSPECTIONS

- ☐ Do you have checklists that focus reviewer attention on areas that have been problems in the past?
- ☐ Is the emphasis on defect detection rather than correction?
- ☐ Are inspectors given enough time to prepare before the inspection meeting, and is each one prepared?
- ☐ Does each participant have a distinct role to play?
- ☐ Does the meeting move at a productive rate?
- ☐ Is the meeting limited to two hours?
- ☐ Has the moderator received specific training in conducting inspections?
- ☐ Is data about error types collected at each inspection so that you can tailor future checklists to your organization?
- ☐ Is data about preparation and inspection rates collected so that you can optimize future preparation and inspections?
- ☐ Are the action items assigned at each inspection followed up, either personally by the moderator or with a re-inspection?
- ☐ Does management understand that it should not attend inspection meetings?

106. CHECKLIST: HIGH-QUALITY ROUTINES

106.1. BIG-PICTURE ISSUES

- ☐ Is the reason for creating the routine sufficient?
- ☐ Have all parts of the routine that would benefit from being put into routines of their own been put into routines of their own?

This material is copied and/or adapted from the Code Complete 2 Website at cc2e.com. This material is Copyright (c) 1993-2004 Steven C. McConnell. Permission is hereby given to copy, adapt, and distribute this material as long as this notice is included on all such materials and the materials are not sold, licensed, or otherwise distributed for commercial gain.

- ☐ Is the routine's name a strong, clear verb-plus-object name for a procedure or a description of the return value for a function?
- ☐ Does the routine's name describe everything the routine does?
- ☐ Have you established naming conventions for common operations?
- ☐ Does the routine have strong, functional cohesion – doing one and only one thing and doing it well?
- ☐ Do the routines have loose coupling – are the routine's connections to other routines small, intimate, visible, and flexible?
- ☐ Is the length of the routine determined naturally by its function and logic, rather than by an artificial coding standard?

106.2. PARAMETER-PASSING ISSUES

- ☐ Does the routine's parameter list, taken as a whole, present a consistent interface abstraction?
- ☐ Are the routine's parameters in a sensible order, including matching the order of parameters in similar routines?
- ☐ Are interface assumptions documented?
- ☐ Does the routine have seven or fewer parameters?
- ☐ Is each input parameter used?
- ☐ Is each output parameter used?
- ☐ Does the routine avoid using input parameters as working variables?
- ☐ If the routine is a function, does it return a valid value under all possible circumstances?

107. CHECKLIST: DEFENSIVE PROGRAMMING

107.1. GENERAL

- ☐ Does the routine protect itself from bad input data?
- ☐ Have you used assertions to document assumptions, including preconditions and postconditions?
- ☐ Have assertions been used only to document conditions that should never occur?
- ☐ Does the architecture or high-level design specify a specific set of error handling techniques?
- ☐ Does the architecture or high-level design specify whether error handling should favor robustness or correctness?
- ☐ Have barricades been created to contain the damaging effect of errors and reduce the amount of code that has to be concerned about error processing?
- ☐ Have debugging aids been used in the code?
- ☐ Has information hiding been used to contain the effects of changes so that they won't affect code outside the routine or class that is changed?
- ☐ Have debugging aids been installed in such a way that they can be activated or deactivated without a great deal of fuss?
- ☐ Is the amount of defensive programming code appropriate – neither too much nor too little?
- ☐ Have you used offensive programming techniques to make errors difficult to overlook during development?

107.2. EXCEPTIONS

- ☐ Has your project defined a standardized approach to exception handling?

This material is copied and/or adapted from the Code Complete 2 Website at cc2e.com. This material is Copyright (c) 1993-2004 Steven C. McConnell. Permission is hereby given to copy, adapt, and distribute this material as long as this notice is included on all such materials and the materials are not sold, licensed, or otherwise distributed for commercial gain.

- ☐ Have you considered alternatives to using an exception?
- ☐ Is the error handled locally rather than throwing a non-local exception if possible?
- ☐ Does the code avoid throwing exceptions in constructors and destructors?
- ☐ Are all exceptions at the appropriate levels of abstraction for the routines that throw them?
- ☐ Does each exception include all relevant exception background information?
- ☐ Is the code free of empty catch blocks? (Or if an empty catch block truly is appropriate, is it documented?)

107.3. SECURITY ISSUES

- ☐ Does the code that checks for bad input data check for attempted buffer overflows, SQL injection, html injection, integer overflows, and other malicious inputs?
- ☐ Are all error-return codes checked?
- ☐ Are all exceptions caught?
- ☐ Do error messages avoid providing information that would help an attacker break into the system?

108. CHECKLIST: GENERAL CONSIDERATIONS IN USING DATA

108.1. INITIALIZING VARIABLES

- ☐ Does each routine check input parameters for validity?
- ☐ Does the code declare variables close to where they're first used?
- ☐ Does the code initialize variables as they're declared, if possible?
- ☐ Does the code initialize variables close to where they're first used, if it isn't possible to declare and initialize them at the same time?
- ☐ Are counters and accumulators initialized properly and, if necessary, reinitialized each time they are used?
- ☐ Are variables reinitialized properly in code that's executed repeatedly?
- ☐ Does the code compile with no warnings from the compiler?
- ☐ If your language uses implicit declarations, have you compensated for the problems they cause?

108.2. OTHER GENERAL ISSUES IN USING DATA

- ☐ Do all variables have the smallest scope possible?
- ☐ Are references to variables as close together as possible – both from each reference to a variable to the next and in total live time?
- ☐ Do control structures correspond to the data types?
- ☐ Are all the declared variables being used?
- ☐ Are all variables bound at appropriate times, that is, striking a conscious balance between the flexibility of late binding and the increased complexity associated with late binding?
- ☐ Does each variable have one and only one purpose?
- ☐ Is each variable's meaning explicit, with no hidden meanings?

109. CHECKLIST: NAMING VARIABLES

109.1. GENERAL NAMING CONSIDERATIONS

- ☐ Does the name fully and accurately describe what the variable represents?
- ☐ Does the name refer to the real-world problem rather than to the programming-language solution?
- ☐ Is the name long enough that you don't have to puzzle it out?
- ☐ Are computed-value qualifiers, if any, at the end of the name?
- ☐ Does the name use Count or Index instead of Num? Naming Specific Kinds Of Data
- ☐ Are loop index names meaningful (something other than i, j, or k if the loop is more than one or two lines long or is nested)?
- ☐ Have all “temporary” variables been renamed to something more meaningful?
- ☐ Are boolean variables named so that their meanings when they're True are clear?
- ☐ Do enumerated-type names include a prefix or suffix that indicates the category – for example, Color for Color Red, Color Green, Color Blue, and so on?
- ☐ Are named constants named for the abstract entities they represent rather than the numbers they refer to?

109.2. NAMING CONVENTIONS

- ☐ Does the convention distinguish among local, class, and global data?
- ☐ Does the convention distinguish among type names, named constants, enumerated types, and variables?
- ☐ Does the convention identify input-only parameters to routines in languages that don't enforce them?
- ☐ Is the convention as compatible as possible with standard conventions for the language?
- ☐ Are names formatted for readability? Short Names
- ☐ Does the code use long names (unless it's necessary to use short ones)?
- ☐ Does the code avoid abbreviations that save only one character?
- ☐ Are all words abbreviated consistently?
- ☐ Are the names pronounceable?
- ☐ Are names that could be mispronounced avoided?
- ☐ Are short names documented in translation tables?

109.3. COMMON NAMING PROBLEMS: HAVE YOU AVOIDED...

- ☐ ...names that are misleading?
- ☐ ...names with similar meanings?
- ☐ ...names that are different by only one or two characters?
- ☐ ...names that sound similar?
- ☐ ...names that use numerals?
- ☐ ...names intentionally misspelled to make them shorter?
- ☐ ...names that are commonly misspelled in English?
- ☐ ...names that conflict with standard library-routine names or with predefined variable names?

This material is copied and/or adapted from the Code Complete 2 Website at cc2e.com. This material is Copyright (c) 1993-2004 Steven C. McConnell. Permission is hereby given to copy, adapt, and distribute this material as long as this notice is included on all such materials and the materials are not sold, licensed, or otherwise distributed for commercial gain.

- ☐ ...totally arbitrary names?
- ☐ ...hard-to-read characters?

110. CHECKLIST: FUNDAMENTAL DATA

110.1. NUMBERS IN GENERAL

- ☐ Does the code avoid magic numbers?
- ☐ Does the code anticipate divide-by-zero errors?
- ☐ Are type conversions obvious?
- ☐ If variables with two different types are used in the same expression, will the expression be evaluated as you intend it to be?
- ☐ Does the code avoid mixed-type comparisons?
- ☐ Does the program compile with no warnings?

110.2. INTEGERS

- ☐ Do expressions that use integer division work the way they're meant to?
- ☐ Do integer expressions avoid integer-overflow problems?

110.3. FLOATING-POINT NUMBERS

- ☐ Does the code avoid additions and subtractions on numbers with greatly different magnitudes?
- ☐ Does the code systematically prevent rounding errors?
- ☐ Does the code avoid comparing floating-point numbers for equality?

110.4. CHARACTERS AND STRINGS

- ☐ Does the code avoid magic characters and strings?
- ☐ Are references to strings free of off-by-one errors?
- ☐ Does C code treat string pointers and character arrays differently?
- ☐ Does C code follow the convention of declaring strings to be length constant+1?
- ☐ Does C code use arrays of characters rather than pointers, when appropriate?
- ☐ Does C code initialize strings to NULLs to avoid endless strings?
- ☐ Does C code use strncpy() rather than strcpy()? And strncat() and strncmp()?

110.5. BOOLEAN VARIABLES

- ☐ Does the program use additional boolean variables to document conditional tests?
- ☐ Does the program use additional boolean variables to simplify conditional tests?

110.6. ENUMERATED TYPES

- ☐ Does the program use enumerated types instead of named constants for their improved readability, reliability, and modifiability?
- ☐ Does the program use enumerated types instead of boolean variables when a variable's use cannot be completely captured with TRUE and FALSE?

This material is copied and/or adapted from the Code Complete 2 Website at cc2e.com. This material is Copyright (c) 1993-2004 Steven C. McConnell. Permission is hereby given to copy, adapt, and distribute this material as long as this notice is included on all such materials and the materials are not sold, licensed, or otherwise distributed for commercial gain.

- ☐ Do tests using enumerated types test for invalid values?
- ☐ Is the first entry in an enumerated type reserved for “invalid”?
- ☐ Named Constants
- ☐ Does the program use named constants for data declarations and loop limits rather than magic numbers?
- ☐ Have named constants been used consistently – not named constants in some places, literals in others?

110.7. ARRAYS

- ☐ Are all array indexes within the bounds of the array?
- ☐ Are array references free of off-by-one errors?
- ☐ Are all subscripts on multidimensional arrays in the correct order?
- ☐ In nested loops, is the correct variable used as the array subscript, avoiding loop-index cross talk?

110.8. CREATING TYPES

- ☐ Does the program use a different type for each kind of data that might change?
- ☐ Are type names oriented toward the real-world entities the types represent rather than toward programming language types?
- ☐ Are the type names descriptive enough to help document data declarations?
- ☐ Have you avoided redefining predefined types?
- ☐ Have you considered creating a new class rather than simply redefining a type?

111. CHECKLIST: CONSIDERATIONS IN USING UNUSUAL DATA TYPES

111.1. STRUCTURES

- ☐ Have you used structures instead of naked variables to organize and manipulate groups of related data?
- ☐ Have you considered creating a class as an alternative to using a structure?

111.2. GLOBAL DATA

- ☐ Are all variables local or class-scope unless they absolutely need to be global?
- ☐ Do variable naming conventions differentiate among local, class, and global data?
- ☐ Are all global variables documented?
- ☐ Is the code free of pseudo-global data-mammoth objects containing a mishmash of data that's passed to every routine?
- ☐ Are access routines used instead of global data?
- ☐ Are access routines and data organized into classes?
- ☐ Do access routines provide a level of abstraction beyond the underlying data-type implementations?
- ☐ Are all related access routines at the same level of abstraction?

111.3. POINTERS

- ☐ Are pointer operations isolated in routines?

This material is copied and/or adapted from the Code Complete 2 Website at cc2e.com. This material is Copyright (c) 1993-2004 Steven C. McConnell. Permission is hereby given to copy, adapt, and distribute this material as long as this notice is included on all such materials and the materials are not sold, licensed, or otherwise distributed for commercial gain.

- ☐ Are pointer references valid, or could the pointer be dangling?
- ☐ Does the code check pointers for validity before using them?
- ☐ Is the variable that the pointer references checked for validity before it's used?
- ☐ Are pointers set to NULL after they're freed?
- ☐ Does the code use all the pointer variables needed for the sake of readability?
- ☐ Are pointers in linked lists freed in the right order?
- ☐ Does the program allocate a reserve parachute of memory so that it can shut down gracefully if it runs out of memory?
- ☐ Are pointers used only as a last resort, when no other method is available?

112. CHECKLIST: ORGANIZING STRAIGHT LINE CODE

- ☐ Does the code make dependencies among statements obvious?
- ☐ Do the names of routines make dependencies obvious?
- ☐ Do parameters to routines make dependencies obvious?
- ☐ Do comments describe any dependencies that would otherwise be unclear?
- ☐ Have housekeeping variables been used to check for sequential dependencies in critical sections of code?
- ☐ Does the code read from top to bottom?
- ☐ Are related statements grouped together?
- ☐ Have relatively independent groups of statements been moved into their own routines?

113. CHECKLIST: CONDITIONALS

113.1. IF-THEN STATEMENTS

- ☐ Is the nominal path through the code clear?
- ☐ Do if-then tests branch correctly on equality?
- ☐ Is the else clause present and documented?
- ☐ Is the else clause correct?
- ☐ Are the if and else clauses used correctly – not reversed?
- ☐ Does the normal case follow the if rather than the else?
- ☐ if-then-else-if Chains
- ☐ Are complicated tests encapsulated in boolean function calls?
- ☐ Are the most common cases tested first?
- ☐ Are all cases covered?
- ☐ Is the if-then-else-if chain the best implementation – better than a case statement?
- ☐ case Statements
- ☐ Are cases ordered meaningfully?
- ☐ Are the actions for each case simple-calling other routines if necessary?
- ☐ Does the case statement test a real variable, not a phony one that's made up solely to use and abuse the case statement?
- ☐ Is the use of the default clause legitimate?

This material is copied and/or adapted from the Code Complete 2 Website at cc2e.com. This material is Copyright (c) 1993-2004 Steven C. McConnell. Permission is hereby given to copy, adapt, and distribute this material as long as this notice is included on all such materials and the materials are not sold, licensed, or otherwise distributed for commercial gain.

- ☐ Is the default clause used to detect and report unexpected cases?
- ☐ In C, C++, or Java, does the end of each case have a break?

114. CHECKLIST: LOOPS

114.1. LOOP SELECTION AND CREATION

- ☐ Is a while loop used instead of a for loop, if appropriate?
- ☐ Was the loop created from the inside out?

114.2. ENTERING THE LOOP

- ☐ Is the loop entered from the top?
- ☐ Is initialization code directly before the loop?
- ☐ If the loop is an infinite loop or an event loop, is it constructed cleanly rather than using a kludge such as for i = 1 to 9999?
- ☐ If the loop is a C++, C, or Java for loop, is the loop header reserved for loop-control code?

114.3. INSIDE THE LOOP

- ☐ Does the loop use { and } or their equivalent to prevent problems arising from improper modifications?
- ☐ Does the loop body have something in it? Is it nonempty?
- ☐ Are housekeeping chores grouped, at either the beginning or the end of the loop?
- ☐ Does the loop perform one and only one function – as a well-defined routine does?
- ☐ Is the loop short enough to view all at once?
- ☐ Is the loop nested to three levels or less?
- ☐ Have long loop contents been moved into their own routine?
- ☐ If the loop is long, is it especially clear?

114.4. LOOP INDEXES

- ☐ If the loop is a for loop, does the code inside it avoid monkeying with the loop index?
- ☐ Is a variable used to save important loop-index values rather than using the loop index outside the loop?
- ☐ Is the loop index an ordinal type or an enumerated type – not floating point?
- ☐ Does the loop index have a meaningful name?
- ☐ Does the loop avoid index cross talk?

114.5. EXITING THE LOOP

- ☐ Does the loop end under all possible conditions?
- ☐ Does the loop use safety counters – if you've instituted a safety-counter standard?
- ☐ Is the loop's termination condition obvious?
- ☐ If break or continue are used, are they correct?

115. CHECKLIST: UNUSUAL CONTROL STRUCTURES

115.1. RETURN

- ☐ Does each routine use return only when necessary?
- ☐ Do returns enhance readability?

115.2. RECURSION

- ☐ Does the recursive routine include code to stop the recursion?
- ☐ Does the routine use a safety counter to guarantee that the routine stops?
- ☐ Is recursion limited to one routine?
- ☐ Is the routine's depth of recursion within the limits imposed by the size of the program's stack?
- ☐ Is recursion the best way to implement the routine? Is it better than simple iteration?

115.3. GOTO

- ☐ Are gotos used only as a last resort, and then only to make code more readable and maintainable?
- ☐ If a goto is used for the sake of efficiency, has the gain in efficiency been measured and documented?
- ☐ Are gotos limited to one label per routine?
- ☐ Do all gotos go forward, not backward?
- ☐ Are all goto labels used?

116. CHECKLIST: TABLE DRIVEN METHODS

- ☐ Have you considered table-driven methods as an alternative to complicated logic?
- ☐ Have you considered table-driven methods as an alternative to complicated inheritance structures?
- ☐ Have you considered storing the table's data externally and reading it at run time so that the data can be modified without changing code?
- ☐ If the table cannot be accessed directly via a straightforward array index (as in the Age example), have you put the access-key calculation into a routine rather than duplicating the index calculation in the code?

117. CHECKLIST: CONTROL STRUCTURE ISSUES

- ☐ Do expressions use True and False rather than 1 and 0?
- ☐ Are boolean values compared to True and False implicitly?
- ☐ Are numeric values compared to their test values explicitly?
- ☐ Have expressions been simplified by the addition of new boolean variables and the use of boolean functions and decision tables?
- ☐ Are boolean expressions stated positively?
- ☐ Do pairs of braces balance?
- ☐ Are braces used everywhere they're needed for clarity?
- ☐ Are logical expressions fully parenthesized?
- ☐ Have tests been written in number-line order?

This material is copied and/or adapted from the Code Complete 2 Website at cc2e.com. This material is Copyright (c) 1993-2004 Steven C. McConnell. Permission is hereby given to copy, adapt, and distribute this material as long as this notice is included on all such materials and the materials are not sold, licensed, or otherwise distributed for commercial gain.

- ☐ Do Java tests use `a.equals(b)` style instead of `a == b` when appropriate?
- ☐ Are null statements obvious?
- ☐ Have nested statements been simplified by retesting part of the conditional, converting to if-then-else or case statements, moving nested code into its own routine, converting to a more object-oriented design, or improved in some other way?
- ☐ If a routine has a decision count of more than 10, is there a good reason for not redesigning it?

118. REFACTORING

118.1. REASONS TO REFACTOR

- ☐ Code is duplicated
- ☐ A routine is too long
- ☐ A loop is too long or too deeply nested
- ☐ A class has poor cohesion
- ☐ A class interface does not provide a consistent level of abstraction
- ☐ A parameter list has too many parameters
- ☐ Changes within a class tend to be compartmentalized
- ☐ Changes require parallel modifications to multiple classes
- ☐ Inheritance hierarchies have to be modified in parallel
- ☐ Related data items that are used together are not organized into classes
- ☐ A routine uses more features of another class than of its own class
- ☐ A primitive data type is overloaded
- ☐ A class doesn't do very much
- ☐ A chain of routines passes tramp data
- ☐ A middle man object isn't doing anything
- ☐ One class is overly intimate with another
- ☐ A routine has a poor name
- ☐ Data members are public
- ☐ A subclass uses only a small percentage of its parents' routines
- ☐ Comments are used to explain difficult code
- ☐ Global variables are used
- ☐ A routine uses setup code before a routine call or takedown code after a routine call
- ☐ A program contains code that seems like it might be needed someday

118.2. DATA LEVEL REFACTORINGS

- ☐ Replace a magic number with a named constant
- ☐ Rename a variable with a clearer or more informative name
- ☐ Move an expression inline
- ☐ Replace an expression with a routine

This material is copied and/or adapted from the Code Complete 2 Website at cc2e.com. This material is Copyright (c) 1993-2004 Steven C. McConnell. Permission is hereby given to copy, adapt, and distribute this material as long as this notice is included on all such materials and the materials are not sold, licensed, or otherwise distributed for commercial gain.

- ☐ Introduce an intermediate variable
- ☐ Convert a multi-use variable to a multiple single-use variables
- ☐ Use a local variable for local purposes rather than a parameter
- ☐ Convert a data primitive to a class
- ☐ Convert a set of type codes to a class
- ☐ Convert a set of type codes to a class with subclasses
- ☐ Change an array to an object
- ☐ Encapsulate a collection
- ☐ Replace a traditional record with a data class

118.3. STATEMENT LEVEL REFACTORINGS

- ☐ Decompose a boolean expression
- ☐ Move a complex boolean expression into a well-named boolean function
- ☐ Consolidate fragments that are duplicated within different parts of a conditional
- ☐ Use break or return instead of a loop control variable
- ☐ Return as soon as you know the answer instead of assigning a return value within nested if-then-else statements
- ☐ Replace conditionals with polymorphism (especially repeated case statements)
- ☐ Create and use null objects instead of testing for null values
- ☐ Routine Level Refactorings
- ☐ Extract a routine
- ☐ Move a routine's code inline
- ☐ Convert a long routine to a class
- ☐ Substitute a simple algorithm for a complex algorithm
- ☐ Add a parameter
- ☐ Remove a parameter
- ☐ Separate query operations from modification operations
- ☐ Combine similar routines by parameterizing them
- ☐ Separate routines whose behavior depends on parameters passed in
- ☐ Pass a whole object rather than specific fields
- ☐ Pass specific fields rather than a whole object
- ☐ Encapsulate downcasting

118.4. CLASS IMPLEMENTATION REFACTORINGS

- ☐ Change value objects to reference objects
- ☐ Change reference objects to value objects
- ☐ Replace virtual routines with data initialization
- ☐ Change member routine or data placement
- ☐ Extract specialized code into a subclass

This material is copied and/or adapted from the Code Complete 2 Website at cc2e.com. This material is Copyright (c) 1993-2004 Steven C. McConnell. Permission is hereby given to copy, adapt, and distribute this material as long as this notice is included on all such materials and the materials are not sold, licensed, or otherwise distributed for commercial gain.

- ☐ Combine similar code into a superclass

118.5. CLASS INTERFACE REFACTORINGS

- ☐ Move a routine to another class
- ☐ Convert one class to two
- ☐ Eliminate a class
- ☐ Hide a delegate
- ☐ Replace inheritance with delegation
- ☐ Replace delegation with inheritance
- ☐ Remove a middle man
- ☐ Introduce a foreign routine
- ☐ Introduce a class extension
- ☐ Encapsulate an exposed member variable
- ☐ Remove Set() routines for fields that cannot be changed
- ☐ Hide routines that are not intended to be used outside the class
- ☐ Encapsulate unused routines
- ☐ Collapse a superclass and subclass if their implementations are very similar

118.6. SYSTEM LEVEL REFACTORINGS

- ☐ Duplicate data you can't control
- ☐ Change unidirectional class association to bidirectional class association
- ☐ Change bidirectional class association to unidirectional class association
- ☐ Provide a factory routine rather than a simple constructor
- ☐ Replace error codes with exceptions or vice versa

118.7. CHECKLIST: REFACTORING SAFELY

- ☐ Is each change part of a systematic change strategy?
- ☐ Did you save the code you started with before beginning refactoring?
- ☐ Are you keeping each refactoring small?
- ☐ Are you doing refactorings one at a time?
- ☐ Have you made a list of steps you intend to take during your refactoring?
- ☐ Do you have a parking lot so that you can remember ideas that occur to you mid-refactoring?
- ☐ Have you retested after each refactoring?
- ☐ Have changes been reviewed if they are complicated or if they affect mission-critical code?
- ☐ Have you considered the riskiness of the specific refactoring, and adjusted your approach accordingly?
- ☐ Does the change enhance the program's internal quality rather than degrading it?
- ☐ Have you avoided using refactoring as a cover for code and fix or as an excuse for not rewriting bad code?

119. CHECKLIST: CODE-TUNING TECHNIQUES

119.1. IMPROVE BOTH SPEED AND SIZE

- ☐ Substitute table lookups for complicated logic
- ☐ Jam loops
- ☐ Use integer instead of floating-point variables
- ☐ Initialize data at compile time
- ☐ Use constants of the correct type
- ☐ Precompute results
- ☐ Eliminate common subexpressions
- ☐ Translate key routines to assembler

119.2. IMPROVE SPEED ONLY

- ☐ Stop testing when you know the answer
- ☐ Order tests in case statements and if-then-else chains by frequency
- ☐ Compare performance of similar logic structures
- ☐ Use lazy evaluation
- ☐ Unswitch loops that contain if tests
- ☐ Unroll loops
- ☐ Minimize work performed inside loops
- ☐ Use sentinels in search loops
- ☐ Put the busiest loop on the inside of nested loops
- ☐ Reduce the strength of operations performed inside loops
- ☐ Change multiple-dimension arrays to a single dimension
- ☐ Minimize array references
- ☐ Augment data types with indexes
- ☐ Cache frequently used values
- ☐ Exploit algebraic identities
- ☐ Reduce strength in logical and mathematical expressions
- ☐ Be wary of system routines
- ☐ Rewrite routines in line

120. CHECKLIST: LAYOUT

120.1. GENERAL

- ☐ Is formatting done primarily to illuminate the logical structure of the code?
- ☐ Can the formatting scheme be used consistently?
- ☐ Does the formatting scheme result in code that's easy to maintain?
- ☐ Does the formatting scheme improve code readability?

This material is copied and/or adapted from the Code Complete 2 Website at cc2e.com. This material is Copyright (c) 1993-2004 Steven C. McConnell. Permission is hereby given to copy, adapt, and distribute this material as long as this notice is included on all such materials and the materials are not sold, licensed, or otherwise distributed for commercial gain.

120.2. CONTROL STRUCTURES

- ☐ Does the code avoid doubly indented begin-end or {} pairs?
- ☐ Are sequential blocks separated from each other with blank lines?
- ☐ Are complicated expressions formatted for readability?
- ☐ Are single-statement blocks formatted consistently?
- ☐ Are case statements formatted in a way that's consistent with the formatting of other control structures?
- ☐ Have gotos been formatted in a way that makes their use obvious?

120.3. INDIVIDUAL STATEMENTS

- ☐ Is white space used to make logical expressions, array references, and routine arguments readable?
- ☐ Do incomplete statements end the line in a way that's obviously incorrect?
- ☐ Are continuation lines indented the standard indentation amount?
- ☐ Does each line contain at most one statement?
- ☐ Is each statement written without side effects?
- ☐ Is there at most one data declaration per line?

120.4. COMMENTS

- ☐ Are the comments indented the same number of spaces as the code they comment?
- ☐ Is the commenting style easy to maintain?

120.5. ROUTINES

- ☐ Are the arguments to each routine formatted so that each argument is easy to read, modify, and comment?
- ☐ Are blank lines used to separate parts of a routine?

120.6. CLASSES, FILES AND PROGRAMS

- ☐ Is there a one-to-one relationship between classes and files for most classes and files?
- ☐ If a file does contain multiple classes, are all the routines in each class grouped together and is the class clearly identified?
- ☐ Are routines within a file clearly separated with blank lines?
- ☐ In lieu of a stronger organizing principle, are all routines in alphabetical sequence?

121. CHECKLIST: GOOD COMMENTING TECHNIQUE

121.1. GENERAL

- ☐ Can someone pick up the code and immediately start to understand it?
- ☐ Do comments explain the code's intent or summarize what the code does, rather than just repeating the code?
- ☐ Is the Pseudocode Programming Process used to reduce commenting time?
- ☐ Has tricky code been rewritten rather than commented?

This material is copied and/or adapted from the Code Complete 2 Website at cc2e.com. This material is Copyright (c) 1993-2004 Steven C. McConnell. Permission is hereby given to copy, adapt, and distribute this material as long as this notice is included on all such materials and the materials are not sold, licensed, or otherwise distributed for commercial gain.

- ☐ Are comments up to date?
- ☐ Are comments clear and correct?
- ☐ Does the commenting style allow comments to be easily modified?

121.2. STATEMENTS AND PARAGRAPHS

- ☐ Does the code avoid endline comments?
- ☐ Do comments focus on why rather than how?
- ☐ Do comments prepare the reader for the code to follow?
- ☐ Does every comment count? Have redundant, extraneous, and self-indulgent comments been removed or improved?
- ☐ Are surprises documented?
- ☐ Have abbreviations been avoided?
- ☐ Is the distinction between major and minor comments clear?
- ☐ Is code that works around an error or undocumented feature commented?

121.3. DATA DECLARATIONS

- ☐ Are units on data declarations commented?
- ☐ Are the ranges of values on numeric data commented?
- ☐ Are coded meanings commented?
- ☐ Are limitations on input data commented?
- ☐ Are flags documented to the bit level?
- ☐ Has each global variable been commented where it is declared?
- ☐ Has each global variable been identified as such at each usage, by a naming convention, a comment, or both?
- ☐ Are magic numbers replaced with named constants or variables rather than just documented?

121.4. CONTROL STRUCTURES

- ☐ Is each control statement commented?
- ☐ Are the ends of long or complex control structures commented or, when possible, simplified so that they don't need comments?

121.5. ROUTINES

- ☐ Is the purpose of each routine commented?
- ☐ Are other facts about each routine given in comments, when relevant, including input and output data, interface assumptions, limitations, error corrections, global effects, and sources of algorithms?

121.6. FILES, CLASSES, AND PROGRAMS

- ☐ Does the program have a short document such as that described in the Book Paradigm that gives an overall view of how the program is organized?
- ☐ Is the purpose of each file described?

This material is copied and/or adapted from the Code Complete 2 Website at cc2e.com. This material is Copyright (c) 1993-2004 Steven C. McConnell. Permission is hereby given to copy, adapt, and distribute this material as long as this notice is included on all such materials and the materials are not sold, licensed, or otherwise distributed for commercial gain.

- ☐ Are the author's name, email address, and phone number in the listing?

122. CHECKLIST: SELF-DOCUMENTING CODE

122.1. CLASSES

- ☐ Does the class's interface present a consistent abstraction?
- ☐ Is the class well named, and does its name describe its central purpose?
- ☐ Does the class's interface make obvious how you should use the class?
- ☐ Is the class's interface abstract enough that you don't have to think about how its services are implemented?
- ☐ Can you treat the class as a black box?

122.2. ROUTINES

- ☐ Does each routine's name describe exactly what the routine does?
- ☐ Does each routine perform one well-defined task?
- ☐ Have all parts of each routine that would benefit from being put into their own routines been put into their own routines?
- ☐ Is each routine's interface obvious and clear?

122.3. DATA NAMES

- ☐ Are type names descriptive enough to help document data declarations?
- ☐ Are variables named well?
- ☐ Are variables used only for the purpose for which they're named?
- ☐ Are loop counters given more informative names than i, j, and k?
- ☐ Are well-named enumerated types used instead of makeshift flags or boolean variables?
- ☐ Are named constants used instead of magic numbers or magic strings?
- ☐ Do naming conventions distinguish among type names, enumerated types, named constants, local variables, class variables, and global variables?

122.4. DATA ORGANIZATION

- ☐ Are extra variables used for clarity when needed?
- ☐ Are references to variables close together?
- ☐ Are data types simple so that they minimize complexity?
- ☐ Is complicated data accessed through abstract access routines (abstract data types)?

122.5. CONTROL

- ☐ Is the nominal path through the code clear?
- ☐ Are related statements grouped together?
- ☐ Have relatively independent groups of statements been packaged into their own routines?
- ☐ Does the normal case follow the if rather than the else?

This material is copied and/or adapted from the Code Complete 2 Website at cc2e.com. This material is Copyright (c) 1993-2004 Steven C. McConnell. Permission is hereby given to copy, adapt, and distribute this material as long as this notice is included on all such materials and the materials are not sold, licensed, or otherwise distributed for commercial gain.

- ☐ Are control structures simple so that they minimize complexity?
- ☐ Does each loop perform one and only one function, as a well-defined routine would?
- ☐ Is nesting minimized?
- ☐ Have boolean expressions been simplified by using additional boolean variables, boolean functions, and decision tables?

122.6. LAYOUT

- ☐ Does the program's layout show its logical structure?

122.7. DESIGN

- ☐ Is the code straightforward, and does it avoid cleverness?
- ☐ Are implementation details hidden as much as possible?
- ☐ Is the program written in terms of the problem domain as much as possible rather than in terms of computer-science or programming-language structures?

APPENDIX J

Code Review Rubric

This appendix describes the rating of source code workmanship.

Note: This was inspired by numerous sources including the First Lego League Coaches Handbook, and school grading rubrics.

123. SOFTWARE READABILITY RUBRIC

					Table 51: Readability rubric
Trait	Exceptional	Acceptable	Unsatisfactory	Needs improvement	
<i>Coding Style</i>	Coding style is checked at code reviews; automated tools are used to ensure consistent formatting; audits		Program manager has ensured that the development team has followed a set of coding standards	No check that software team has and is following coding standards	
<i>Consistency</i>	Coding style guidelines are followed correctly.	Coding style guidelines are almost always followed correctly.	Coding style guidelines are not followed. Style guide may be inadequate.	Does not match style guide; style guide may not exist.	
<i>Organization</i>	The code is exceptionally well organized	The code is logically organized.	The code is poorly organized	The code is disorganized	
<i>Readability</i>	The code is clean, very easy to follow, understandable, is easy to maintain, and has no errors.	The code is easy to read. Minor issues with consistent indentation, use of whitespace, variable naming, or general organization.	The code is readable only by someone who knows what it is supposed to be doing. At least one major issue with indentation, whitespace, variable names, or organization.	The code is poorly organized and very difficult to read. Major problems with at three or four of the readability subcategories.	
<i>Indentation / white spaces</i>	Indentation and whitespace follow coding style and is not distracting.	Minor issues with consistent indentation, use of whitespace.	At least one major issue with indentation, whitespace.	The code is poorly organized and very difficult to read.	
<i>Naming</i>	All names follow naming conventions, are meaningful or expressive without being verbose, and documented. Data dictionary is complete.	Names are mostly consistent in style and expressive. Isolated cases may be verbose, overly terse or ambiguous. No data dictionary	Names are occasionally verbose, but often are cryptic or overly terse, ambiguous or misleading. No data dictionary.	Variable names are cryptic, and no data dictionary is shown.	

124. SOFTWARE COMMENTS & DOCUMENTATION

Trait	Exceptional	Acceptable	Unsatisfactory	Needs improvement
<i>Comments</i>	Code is well-commented.	One or two places that could benefit from comments are missing them or the code is overly commented.	File header missing, lack of comments or meaningful comments.	No file header or comments present.
<i>Initial Comments</i>	Initial comments are complete. Internal documentation is complete and well suited to the program	Initial comments are complete but internal documentation is in some small fashion inadequate.	Initial comments are incomplete or internal documentation is inadequate.	No internal documentation
<i>Coding Comments</i>	Every line is commented. Comments clarify meaning.	Many comments are present, in correct format. Comments usually clarify meaning. Unhelpful comments may exist.	Some comments exist but are frequently unhelpful or occasionally misleading; may use an incorrect format. Complicated lines or sections of code uncommented or lacking meaningful comments. Comments do not help the reader understand the code.	No comments
<i>Documentation</i>	The documentation is well written and clearly explains what the code is accomplishing and how, at an appropriate level of detail. All required and most optional elements are present and follow the prescribed format.	The documentation is not compelling; consists of code comments and simple header documentation that is somewhat useful in understanding the code. All files, procedures, and structures are given an overview statement.	The documentation is simply comments embedded in the code with some header comments separating routines. Does little to help the reader understand the implementation.	No documentation. There might be comments embedded in the code with some simple header comments separating routines. Does not help the reader understand the implementation.
<i>Overview statement</i>	The overview is given and explains what the code is accomplishing.	The overview is given but is minimal and is only somewhat useful in understanding the code.	The overview is not given or is not helpful in understanding what the code is to accomplish.	No overview is given.
<i>Top-Down Design</i>	Top-down design method followed and written in appropriate detail.	Top-down method followed, but level of detail is too vague or too exact.	Top-down design method attempted, but poorly executed.	No design.
<i>Design & Diagrams</i>	A design tool or diagram is correctly used	A design or diagram tool is used but does not entirely match code	A design or diagram tool is used but is incorrect.	No design or diagram tool is used.

Table 52: Comments and documentation rubric

<i>Identification</i>	All identifying information is shown in the documentation	Some identifying information is shown.	Only a small portion of identifying information is shown, and/or is not correct.	No identifying information is shown.
-----------------------	---	--	--	--------------------------------------

125. IMPLEMENTATION

Trait	Exceptional	Acceptable	Unsatisfactory	Needs improvement
<i>Syntax/runtime /logic errors</i>	The program contains no errors.	The program has no major errors.	Program executes but has errors.	Program does not execute.
<i>Modularization & Generalization</i>	Program is broken into well thought out elements that are of an appropriate length, scope and independence.	Code elements are generally well planned and executed. Some code is repeated that should be encapsulated. Individual elements are often, but not always, written in a way that invites code reuse.	Code elements are not well thought out, are used in a somewhat arbitrary fashion, or do not improve program clarity. Elements are seldom written in a way that invites code reuse.	
<i>Reusability</i>	Individual elements were developed in a manner that actively invites reuse in other projects.	Most of the routines could be reused in other programs.	Some parts of the code could be reused in other programs.	The code is not organized for reusability.
<i>Design, & Structure</i>	Program is designed in a clear and logical manner. Control structures are used correctly. The most appropriate algorithms are used, in a manner that does not sacrifice readability or understanding	Program is mostly clear and logical. Control structures are used correctly. Reasonable algorithms are implemented, in a manner that does not sacrifice readability or understanding	Program isn't as clear or logical as it should be. Control structures are occasionally used incorrectly. Steps that are clearly inefficient or unnecessarily long are used.	The code is huge and appears to be patched together. Requires significant effort to comprehend.
<i>Emulation</i>	has a whole system emulation	can emulate significant parts, individually	in concept could emulate	no emulation
<i>Efficiency</i>	The code is extremely efficient, using the best approach in every case.	The code is efficient at completing most tasks	Code uses poorly chosen approaches in at least one place. For example, the code is brute force	Many things in the code could have been accomplished in an easier, faster, or otherwise better fashion.
<i>Consistency</i>	Program behaves in a consistent, predictable fashion, even for complex tasks	Mostly predictable	Somewhat unpredictable	unpredictable
Trait	Exceptional	Acceptable	Unsatisfactory	Needs improvement

Table 53:
Implementation rubric

<i>Operating bounds</i>	The code and design have been reviewed by independent experts for arithmetic issues. Appropriate analysis tools have been used. A sizable body of test cases and tests has been applied against the code.	The code and design have been reviewed by independent experts for resource arithmetic issues.	The Designer has ensured that the implementation is not vulnerable to arithmetic issues.	No one has checked for arithmetic issues
	The code and design have been reviewed by independent experts for buffer overflow issues. Appropriate analysis tools have been used. A sizable body of test cases and tests has been applied against the code.	The code and design have been reviewed by independent experts for buffer overflow issues.	The Designer has ensured that the implementation is not vulnerable to buffer overflow issues.	No one has checked for overflow issues
	The code and design have been reviewed by independent experts for resource exhaustion. Appropriate analysis tools have been used. A sizable body of test cases and tests has been applied against the code.	The code and design have been reviewed by independent experts for resource exhaustion issues.	The Designer has ensured that the implementation is not vulnerable to resource exhaustion issues.	No one has checked for overflow issues
	The code and design have been reviewed by independent experts for race conditions. Appropriate analysis tools have been used. A sizable body of test cases and tests has been applied against the code.	The code and design have been reviewed by independent experts for race conditions	The Designer has ensured that the implementation is not vulnerable to race conditions.	No one has checked for race conditions

Trait	Exceptional	Acceptable	Unsatisfactory	Needs improvement
<i>Correctness</i>	Prioritization properly based on Rate Monotonic Analysis. Performs error checking in all cases. Appropriately bounded time checks are used in all cases. Resources are appropriately sized.	Has potential or obvious deadlocks. Some operations do not use time limits or uses limits that are too long. Does not check for error / lack of resources in some case. Has prioritization, based on ad hoc experience, not on analysis. Mutexes correctly used. Semaphores may overflow, or not wake task	Has obvious deadlocks. Does not use time limits on operations. Doesn't check for error, or lack of resources. Resource sizing is not based on analysis. Has prioritization, based on ad hoc experience, not on analysis. Semaphores or mutexes misused.	Has obvious deadlocks. Does not use time limits to operations. Doesn't check for error/lack of resources. Resource sizing is not based on analysis. No prioritization, not based on analysis
<i>Problem Prevention</i>	Communication / resource utilization has effective (or best in class) collision avoidance algorithms	Communication / resource utilization has some collision avoidance algorithm(s), but it is not always effective (or best in class)	Communication / resource utilization has poorly thought-out collision avoidance approach	Communication / resource utilization has no collision avoidance algorithm
	Has fallback on collision, reducing further errors in all cases	Has fallback on collision, reducing further errors in most cases	Has fallback on collision, but fails to significantly reduce collisions	Has no fallback on collision
<i>Safety</i>	Controls have been identified from analysis such as SIL or FMEA. Device handles error / exception circumstances correctly. Device engages safe conditions in all cases. Internal state is monitored. External state is monitored. Self-checks are performed correctly. Memory and other internal protection are employed.	Internal state, such as values and Buffers are checked. Output monitoring is employed. Self-test is not performed.	Some safe bounds are used. Some value/range checking is employed. Some output monitoring is employed.	No requirements, no analysis, no action.

126. ERROR HANDLING

Trait	Exceptional	Acceptable	Unsatisfactory	Needs improvement
<i>Robustness</i>	Program handles erroneous or unexpected input gracefully; action is taken without surprises	All obvious error conditions are checked for and appropriate action is taken.	Some, but not sufficient portion of, obvious error conditions are checked for with an appropriate action is taken.	Many obvious error conditions are not checked. Or, if checked, appropriate action is not taken.
<i>PID Control</i>	Is stable and free of oscillation (low and high frequency) for all manner of conditions and disturbances	Is stable and free of oscillation for most conditions and disturbances; may have some high-pitch whine or oscillation for boundary conditions	Is occasionally approximately correct, frequently has oscillation or is easily disturbed	Has high oscillation, high degree of error.
<i>Testing</i>	Testing is complete without being redundant. All boundary cases are considered and tested.	All key items are tested, but testing may be redundant. Nearly all boundary cases are considered and tested.	Testing was done but is not sufficiently complete. Most boundary cases are considered and tested.	Testing has not been done

Table 54: Error handling rubric

127. BEHAVIOUR

Trait	Exceptional	Acceptable	Unsatisfactory	Needs improvement
<i>Analysis of comm and IPC network</i>	Full structural analysis of all software systems as a network.	Structural analysis of only IPC, or communication section.	structural analysis of one unit software	no structural analysis
<i>Communication overload</i>	Handles overload in a graceful fashion, with predictable / defined behaviour, including honoring time bounds, priority order of responses to messages, and dropping messages & disabling services.	Thrashes on overload. Inefficient slow responses	Communication fails; does not hold safe state; is not responsive; crashes or sends erroneous behaviour. Runs out of resources. Critical behaviours are missed.	crashes on overload
<i>Interrupt / Event overload</i>	Handles overload in a graceful fashion, with predictable / defined behaviour, including honoring time bounds, priority order of responses and dropping messages & disabling services.	Thrashes on overload. Inefficient slow responses	Communication fails; does not hold safe state; is not responsive; crashes or sends erroneous behaviour. Runs out of resources. Critical behaviours are missed.	crashes on overload

Table 55: Behaviour rubric

APPENDIX K

ARM Cortex-M Specifics

This appendix provides detailed technical tips specific to ARM Cortex-M processors, that is too low-level for a detailed design document. Instead, this type of information would go into technical notes and the software implementation “doxygen.”

128. MICROCONTROLLER SPECIFIC DETAILED DESIGN ELEMENTS

This section covers design features specific to Cortex-M based microcontrollers.

128.1. ATOMICITY

On the Cortex-M processors, loads and stores are atomic *only* if:

- It is an 8-bit transaction, or
- It is a 16-bit transaction to an address aligned 16-bits, or
- It is a 32-bit transaction to an address aligned 32-bits

Normally the compiler takes care of this of this alignment. The exceptions – which will void the atomics – are if

- a compiler option has been used to change padding or alignment
- The variable was specified with an address
- The structs are “packed” or otherwise had their alignment changed.

This means that volatiles are not read or written atomically on the Cortex-M unless all of the conditions mentioned are followed. Compare and swap techniques or disabling interrupts must be used when modifying memory shared with an interrupt routine

128.2. MEMORY BARRIERS

Memory barriers are a necessary mechanism to force the commit of memory access before next step. Specifically, it ensures that data has been moved from any cache / buffer to the destination, and blocks execution until that has been done.

- Some instruction cores have write buffers – the Cortex-M0 does not.
- The microcontroller may have a system level cache (outside of instruction engine)
- There may be queue or buffer between the processor and the memory mapped peripheral (esp. external peripherals attached to the memory bus). *Note: the memory region often should also be marked as non-cacheable.*

- There may be a queue or buffer between the processor and the event bus.

Memory barriers should be employed

- Before wait for event/interrupt (sleep)
- In the construction of IPC mechanisms, e.g. mutexes and semaphores

The barrier CMSIS wrappers are:

```
_DAB()
_DSB()
_ISB()
```

128.3. A NOTE ON ARM CORTEX-M0 PROCESSORS

The ARM Cortex-M0 instruction core cannot do:

- Compare and swap (LDREX/STREX)
- Atomic writes or increments
- Bit-banding
- Detecting that debugger is attached

The techniques below are still (largely) applicable but will have Cortex-M0 specific adaptations.

128.4. HARDWARE EXCEPTIONS

Exceptions, and faults, are a type of error detected by the processor at run-time. By supplying the appropriate handling procedure, the software can signal an error condition. The handlers can preserve the call stack, key register values, and key global variables. This may be helpful for identifying what was going on.

128.5. DIGITAL INPUTS AND OUTPUTS

The majority of microcontrollers have “Input Data Registers” and “Output Data registers” per port. Save the data register, and the masks (for the relevant ones to access), and possibly any index substitution index from internal reference to the data register and pin.

No microcontroller I’ve seen has more than 32 pins per port; most keep to 16 or fewer.

128.6. BITBAND

Cortex-M3 and above processors have bit-banding. This can be leveraged for simplifying IO. It can create a pointer to a single IO pin. For instance, for the chip select on I²C or SPI communication. (Assuming that the hardware peripheral doesn’t already handle the chip select).

128.7. PROCEDURE BLIP

One useful technique is to have procedures raise a digital output line when they enter and lower it when they exit. This can be used to:

- Validate that key procedures execute when stimulated
- Measure the duration of interrupt or other procedure

- Check that procedures execution timing holds, even under load or high events
- To demonstrate the regularity of procedure execution
- To demonstrate regularity of events, such as CPU timers, and interrupt servicing.

The design is simple. Create a variable for each potential procedure of interest, defined like so:

```
uint32_t volatile* XYZ_blip= &XYZ_null;
```

In side of each procedure – called XYZ_procedure() here – have the following template:

```
void XYZ_procedure()
{
    XYZ_blip[0] = 1;
    ... do work ....
    XYZ_blip[0] = 0;
}
```

When the procedure it will set the value at the destination of the pointer to 1, and when it exit it will set the value at the destination of the pointer to 0. The probe effect is minimal: the procedure executes the same code no matter what XYZ_blip points to. Both steps take only an instruction or two; there are no conditions, branches or other variations.

To cause the procedure to blip a digital output pin:

1. Ensure that the GPIO is configured to be an output
2. Set XYZ_blip to point to the bitband address for the pins bit in the digital output register of the target port.

Note: multiple procedures can drive the same output pin.

The disable the procedure blip:

1. Set XYZ_blip to point to XYZ_null. This way the procedure only stores to a dummy variable.

The execution time of the procedure is the same whether or not the probe is enabled, and the overhead is negligible.

Note: as stated above, Cortex-M0 based (and non-Cortex) processors do not have banding. The above technique can be adapted in a straightforward manner to those processors.

128.8. FIND-FIRST SET BIT

Finding the first set bit in O(1) time is an important utility procedure. It is used to find, for example, the highest queued item in a bit list. Cortex-M3 and above include a “count left zeros” instruction which will tell one the highest bit set in a 32-bit word:

```
highest bit set = 32- clz(x)
```

However, the usual convention is that bit 0 is the highest priority and bit 32 is the lowest. This convention allows working with longer bit queues by using a hierarchy. To find the right most bit set, one could (but should not do):

```
FFS(x) = 32- clz(x&(-x))
```

This takes several instructions. Finding the highest priority is often performed in a time critical procedure, such as PendSV exception handler to switch tasks. The next option is to employ the ARM “reverse bits” instruction:

```
arm_clz(arm_rbit(xx))
gcc:
__builtin_clz(__builtin_bswap32(x))
```

This is better, but still 1 instruction slower than need be.

The fix is to reverse the bits when they are set in the mask:

```
mask |= 0x80000000uL >>idx;
```

On the ARM that takes the same number of instructions as:

```
mask |= 1uL << idx;
```

128.9. INTERRUPT PRIORITIZATION

The ARM Cortex-M microcontrollers have a prioritizable interrupt controller. Many processors can have as few as four levels of prioritization. Others can have a great range of prioritization. The diagram gives some idea of how higher interrupts & exceptions can interrupt lower ones.

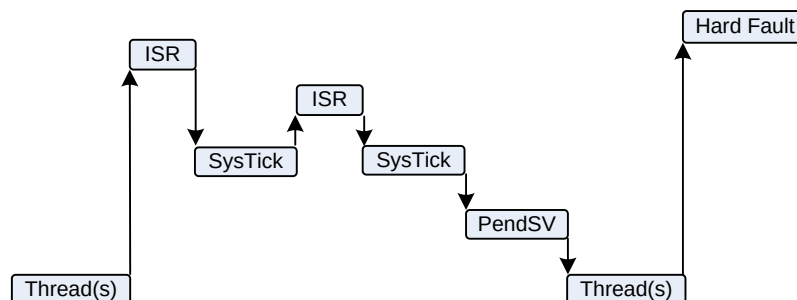


Figure 45: Prioritized interrupts and exceptions

The *Hard fault* exception (and other similar faults, such as NMI, etc.) is at the highest priority, and is fixed in the hardware. Interrupts cannot occur within these handlers. If this is invoked, the software (and/or hardware) has failed. The software design should place the hardware in safe state but take no complex actions.

PendSV is an exception at the lowest priority, in that it is invoked infrequently – only when a thread, timer (in systick), and other IPC object in the interrupt changes the CPU’s ready-to-run list.

The *System Tick* is an exception that occurs regularly. It is (in this design guide) at a priority lower than all of the interrupts. This is done to service the interrupts with lower latency, preserving the quality of their function. It is the same priority (or higher than) *PendSV*’s priority. If it were at a priority lower than *PendSV*, the regular switching of tasks would be a much higher cost.

The low priority of the system tick handler serves an integrity role: this is how interrupt overload is detected. A watchdog timer – such as the *windowed watchdog timer* on the STM32 product family – will be serviced (or partly serviced) in the system tick routine. If the system tick routine can’t execute regularly either because of interrupt overload, or someone having disabled interrupts for too long, the watchdog servicing will be inhibited, triggering the microcontroller to reset, and proceed to the fail-safe state.

Note: by partially serviced can mean that the watchdog timer in question can only be reset if the system tick has hand-shaked with some other thread. By requiring all the parties to handshake (or other demonstration of liveliness) the watchdog timer can detect failure to service those parties in a timely fashion.

129. REFERENCES AND RESOURCES

ARM, DDI0403, “ARM v7-M Architecture Reference Manual,” Rev E.b, 2014-Dec

ARM, DDI0406, “ARM Architecture Reference Manual, ARM v7-A and ARM v7-R edition” Rev C.c, 2014-May

ARM, DDI0419, “ARMv6-M Architecture Reference Manual,” Rev D 2017-May

ARM, DDI 0432C, “Cortex-M0: Technical Reference Manual” r0p0 Rev C 2009 Nov 30

ARM, DDI 0439B, “Cortex-M4: Technical Reference Manual,” Rev r0p0 2009-2010

ARM, DUI 0497A “Cortex-M0 Devices: Generic User Guide” Rev A 2009 Oct 8

ARM, QRC0011, “ARMv6-M Instruction Set Quick Reference Guide,” Rev L 2007 March

Keil, “Using Cortex-M3 and Cortex-M4 Fault Exceptions,” Application Note 209. 2010

CMSIS “Cortex Microcontroller Software Interface Standard,” Version: 1.10 - 24. 2009 Feb

Doulos, “Getting started with CMSIS” 2009

129.1. MEMORY PROTECTION

Atmel, “AT02346: Using the MPU on Atmel Cortex-M3 / Cortex-M4 based Microcontrollers,” 2013

ST Micro, DocID029037, “AN4838 Managing memory protection unit (MPU) in STM32 MCUs”, Rev 1, 2016 Mar

APPENDIX L

Hardware Firmware Integration Tests

130. TESTS

This appendix offers basic tests of the software units, starting with the most fundamental units. The tests are intended to check that the software module function as expected. Note: many of these tests require external stimulation or instrumentation; these tests are not intended to be in the “Test” build configuration.

- Basic input or outputs
- Time based behaviour
- Basic function of the module
- Signal processing qualities

and can be employed as a hardware test:

- Signals stuck. e.g. stuck high or stuck low
- Signals shorted together
- Signals that are open

The digital input tests:

- Test 1: Test CPU input with a line high
- Test 2: Test CPU input with a line low

The digital output tests include:

- Test 3: Test CPU output with a line high
- Test 4: Test CPU output with a line low

The analog input tests include:

- Test 5: Test CPU input with a line high
- Test 6: Test CPU input with a line midrange
- Test 7: Test CPU input with a line low

This analog output tests include:

- Test 8: Test CPU output with a line high
- Test 9: Test CPU output with a line midrange
- Test 10: Test CPU output with a line low

“This application has requested the Runtime to terminate it in an unusual way.”
– An actual Microsoft error message

This polynomial correction tests include:

- Test 11: Test CPU input with a line high
- Test 12: Test CPU input with a line midrange
- Test 13: Test CPU input with a line low

This IIR signal processing tests include:

- Test 14: Inject a stable signal
- Test 15: Inject a signal with a fast-rising pulse
- Test 16: Inject a signal with a fast-rising step
- Test 17: Inject a signal with a fast-descending pulse
- Test 18: Inject a signal with a fast-descending step

This debounce module tests include:

- Test 19: Check that a rising edge is passed thru
- Test 20: Check that a falling edge is passed thru
- Test 21: Check that rising-edge bounces are rejected
- Test 22: Check that falling-edge bounces are rejected

130.1. TEST 1: TEST CPU INPUT WITH A LINE HIGH

The basic test is:

1. Set the input high to the pin, using an external tool
2. Read the digital input (using diagnostic tool).

Stretch: This should be done with all input lines. All lines should be read to check that their states do not become high unexpectedly. If a line is in the wrong state, this may indicate a hardware short.

130.2. TEST 2: TEST CPU INPUT WITH A LINE LOW

The basic test is:

1. Set the input low to the pin, using an external tool
2. Read the digital input (using diagnostic tool).

Stretch: This should be done with all input lines. All lines should be read to check that their states do not become high unexpectedly. If a line is in the wrong state, this may indicate a hardware short.

It integrates several elements together that are, in other approaches, *separate* documentation efforts. The testing is often separate, later pass. This is included here for several reasons. Control flow errors: how did it get to the wrong spot? Bug in control flow implementation? Individual values right, but altogether not right. Wrong implementation of control flow.

130.3. TEST 3: TEST CPU OUTPUT WITH A LINE HIGH

The basic test is:

1. With the diagnostic tool, have the software set the output high
2. Using an external tool, read the digital pin. Confirm that it is high.

Stretch: This should be done with all output lines. All lines should be read to check that their states do not become high unexpectedly. If a line is in the wrong state, this may indicate a software problem, or a hardware short.

130.4. TEST 4: TEST CPU OUTPUT WITH A LINE LOW

The basic test is:

1. With the diagnostic tool, have the software set the output low
2. Using an external tool, read the digital pin. Confirm that it is low.

Stretch: This should be done with all output lines. All lines should be read to check that their states do not become high unexpectedly. If a line is in the wrong state, this may indicate a software problem, or a hardware short.

130.5. TEST 5: TEST CPU INPUT WITH A LINE HIGH

The basic test is:

1. Set the input high to the pin, using an external tool
2. Read the analog input (using diagnostic tool).

Stretch: This should be done with all input lines. All lines should be read to check that their states do not become high unexpectedly. If a line is in the wrong state, this may indicate a hardware short.

130.6. TEST 6: TEST CPU INPUT WITH A LINE MIDGRANGE

The basic test is:

1. Set the input to the mid range value, using an external tool
2. Read the analog input (using diagnostic tool).

Stretch: This should be done with all input lines. All lines should be read to check that their states do not become low unexpectedly. If a line is in the wrong state, this may indicate a hardware short.

130.7. TEST 7: TEST CPU INPUT WITH A LINE LOW

The basic test is:

1. Set the input low to the pin, using an external tool
2. Read the analog input (using diagnostic tool).

Stretch: This should be done with all input lines. All lines should be read to check that their states do not become high unexpectedly. If a line is in the wrong state, this may indicate a hardware short.

130.8. TEST 8: TEST CPU OUTPUT WITH A LINE HIGH

The basic test is:

1. Set the output high to the pin, using an external tool
2. Read the analog output (using external tool).

Stretch: This should be done with all output lines. All lines should be read to check that their states do not become high unexpectedly. If a line is in the wrong state, this may indicate a software problem, or a hardware short.

130.9. TEST 9: TEST CPU OUTPUT WITH A LINE MIDRANGE

The basic test is:

1. Set the output midrange to the pin, using an external tool
2. Read the analog output (using an external tool).

Stretch: This should be done with all output lines. All lines should be read to check that their states are at the commanded level, $\pm$ a range. If a line is in the wrong state, this may indicate a software problem, or a hardware short.

130.10. TEST 10: TEST CPU OUTPUT WITH A LINE LOW

The basic test is:

1. Set the output low to the pin, using an external tool
2. Read the analog output (using an external tool).

Stretch: This should be done with all output lines. All lines should be read to check that their states do not become high unexpectedly. If a line is in the wrong state, this may indicate a software problem, or a hardware short.

130.11. TEST 11: TEST CPU INPUT WITH A LINE HIGH

The basic test is:

1. Set the input high to the pin, using an external tool
2. Read the conversion results (using diagnostic tool).

Stretch: This should be done with all input lines. All lines should be read to check that their states do not become high unexpectedly. If a line is in the wrong state, this may indicate a hardware short.

130.12. TEST 12: TEST CPU INPUT WITH A LINE MIDGRANGE

The basic test is:

1. Set the input low to the mid range value, using an external tool
2. Read the conversion results (using diagnostic tool).

Stretch: This should be done with all input lines. All lines should be read to check that their states do not become low unexpectedly. If a line is in the wrong state, this may indicate a hardware short.

130.13. TEST 13: TEST CPU INPUT WITH A LINE LOW

The basic test is:

1. Set the input low to the pin, using an external tool
2. Read the conversion results (using diagnostic tool).

Stretch: This should be done with all input lines. All lines should be read to check that their states do not become high unexpectedly. If a line is in the wrong state, this may indicate a hardware short.

130.14. TEST 14: INJECT A STABLE SIGNAL

The basic test is:

1. Route the filter output to an analog output test point.
2. Using an external tool, set the input to the pin to a known, stable voltage
3. Read the filter results (using diagnostic tool). The reported voltage should match the injected voltage, after the input divider.
4. Check that the output signal is stable (i.e. no self-induced noise)

Stretch: This should be done with all input lines. All lines should be read to check that their states do not become high unexpectedly. If a line is in the wrong state, this may indicate a hardware short.

130.15. TEST 15: INJECT A SIGNAL WITH A FAST RISING PULSE

The basic test is:

1. Route the filter output to an analog output test point.
2. Using an external tool, set the input to the pin to a known, stable voltage
3. Read the filter results (using diagnostic tool). The reported voltage should match the injected voltage, after the input divider.
4. Check that the output signal is stable (i.e. no self-induced noise)
5. Inject a fast-rising pulse, returning to the prior level
6. Read the filter results (using diagnostic tool). The reported voltage should match the prior injected voltage, after the input divider.
7. Check that the output signal is stable (i.e. no self-induced noise as a response), and that blip negligible pass thru.

Stretch: This should be done with all input lines. All lines should be read to check that their states did not have an unexpected pulse. If a line did have a pulse, this may indicate a coupling or error with the filter implementation.

130.16. TEST 16: INJECT A SIGNAL WITH A FAST RISING STEP

The basic test is:

1. Route the filter output to an analog output test point.
2. Using an external tool, set the input to the pin to a known, stable voltage
3. Read the filter results (using diagnostic tool). The reported voltage should match the injected voltage, after the input divider.

4. Check that the output signal is stable (i.e. no self-induced noise)
5. Inject a fast-rising step to a higher voltage level
6. Read the filter results (using diagnostic tool). Within TBD msecs, the reported voltage should match the new injected voltage, after the input divider.
7. Check that the output signal is stable (i.e. no self-induced noise as a response), and that blip negligible pass thru.

Stretch: This should be done with all input lines. All lines should be read to check that their states did not have an unexpected pulse. If a line did have a pulse, this may indicate a coupling or error with the filter implementation.

130.17. TEST 17: INJECT A SIGNAL WITH A FAST DESCENDING PULSE

The basic test is:

1. Route the filter output to an analog output test point.
2. Using an external tool, set the input to the pin to a known, stable voltage
3. Read the filter results (using diagnostic tool). The reported voltage should match the injected voltage, after the input divider.
4. Check that the output signal is stable (i.e. no self-induced noise)
5. Inject a fast-descending pulse, returning to the prior level
6. Read the filter results (using diagnostic tool). The reported voltage should match the prior injected voltage, after the input divider.
7. Check that the output signal is stable (i.e. no self-induced noise as a response), and that blip negligible pass thru.

Stretch: This should be done with all input lines. All lines should be read to check that their states did not have an unexpected pulse. If a line did have a pulse, this may indicate a coupling or error with the filter implementation.

130.18. TEST 18: INJECT A SIGNAL WITH A FAST DESCENDING STEP

The basic test is:

1. Route the filter output to an analog output test point.
2. Using an external tool, set the input to the pin to a known, stable voltage
3. Read the filter results (using diagnostic tool). The reported voltage should match the injected voltage, after the input divider.
4. Check that the output signal is stable (i.e. no self-induced noise)
5. Inject a fast-descending step to a lower voltage level
6. Read the filter results (using diagnostic tool). Within TBD msecs, the reported voltage should match the new injected voltage, after the input divider.
7. Check that the output signal is stable (i.e. no self-induced noise as a response), and that blip negligible pass thru.

Stretch: This should be done with all input lines. All lines should be read to check that their states did not have an unexpected pulse. If a line did have a pulse, this may indicate a coupling or error with the filter implementation.

130.19. TEST 19: CHECK THAT RISING EDGE IS PASSED

The basic test is:

1. Set the input signal low
2. Check that the output signal is low
3. Set the input signal high
4. Check that the output signal is high within TBD ms.

130.20. TEST 20: CHECK THAT FALLING EDGE IS PASSED

The basic test is:

1. Set the input signal high
2. Wait TBD ms
3. Check that the output signal is high
4. Set the input signal low
5. Check that the output signal is low within TBD ms.

130.21. TEST 21: CHECK THAT RISING-EDGE BOUNCES ARE REJECTED

The basic test is:

1. Set the input signal low
2. Check that the output signal is low
3. Set the input signal high
4. Check that the output signal is high within TBD ms.
5. Set the input signal low
6. Check that the output signal is high
7. Raise signal within TBD ms
8. Check that the output signal is high

130.22. TEST 22: CHECK THAT FALLING-EDGE BOUNCES ARE REJECTED

The basic test is:

1. Set the input signal high
2. Check that the output signal is high
3. Set the input signal low
4. Check that the output signal is low within TBD ms.
5. Set the input signal high
6. Check that the output signal is low
7. Set the input signal low within TBD ms
8. Check that the output signal is low

References & Resources

Note: most references appear in the margins, significant references will appear at the end of their respective chapter.

131. REFERENCE DOCUMENTATION AND RESOURCES

131.1. OVERALL SOFTWARE CRAFTSMANSHIP

McConnell, Steve *“Code Complete”* 2ed 2004

IEEE Computer Society, *SWEBOK Guide to the Software Engineering Body of Knowledge*, version 3, 2014

IEEE Std 1044-2009 *IEEE Standard Classification for Software Anomalies*, IEEE-SA Standards Board, 2009 Nov 9

131.2. SOFTWARE SAFETY

Joint Software Systems Safety Committee, *“Software System Safety Handbook,”* 2000-Dec

Joint Software Systems Safety Engineering Workgroup, *“Joint Software Systems Safety Engineering Handbook,”* Rev 1 2010-Aug-27

While both cover much the same material – although the second has more material. I prefer the style of the earlier edition.

MOD Defence Standard 0058 *Requirements for Safety Related Software in Defence Equipment*. 1996 UK Ministry of Defence

MOD Interim Defence Standard 08-58 *Issues 1: HAZOP Studies on Systems Containing Programmable Electronics* 1996 UK Ministry of Defence

SAE ARP 4761 *Guidelines and methods for conducting the safety assessment process on Civil Airborne Systems and Equipment*. 1996 Society of Automotive Engineers.

UCRL-ID-122514, Lawrence, J Dennis *“Software Safety Hazard Analysis”* Rev 2, U.S. Nuclear Regulatory Commission, 1995-October

131.3. OTHER

ISO/IEC/IEEE 60559:2011 *“Information technology – Microprocessor Systems – Floating-Point arithmetic”*

Mikitjuk et al, V.G. Mikitjuk, V.N. Yarmolik, A.J. van de Goor, *RAM Testing Algorithm for Detection Linked Coupling Faults*, IEEE 1996

“Optimally, what is needed is something that can be added to airplanes and other systems which weighs nothing, yet is very costly, and violates none of the physical laws of the universe, such as the law of gravitation or the laws of thermodynamic.

This might appear to be an insurmountable challenge; however, as a result of the traditional ingenuity characteristic of system designers, it can be reported with confidence that such an ingredient has already been found.

It is called software.”

– Norman Ralph Augustine, (1970s)